International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Case Studies: Investigating Rhion Possession Evidence

Student Name: Kafayat Omolara Animashawun
Student ID: 2025/FWSD/11317
Instructor Name: Aminu Idris
Date of Submission: 26/10/2026

EXECUTIVE SUMMARY

This investigation was conducted as part of the ICDFA Basic Computer Skills for Digital Forensics programme, under the case study titled *Investigating Rhion Possession Evidence*. The objective was to examine the supplied evidence files, identify relevant digital artefacts, and document findings using forensic analysis tools in a Kali Linux environment.

The investigation involved verifying the integrity of the supplied archive, examining the USB disk image, identifying and extracting filesystem artefacts, analysing unallocated data, recovering files based on recognised file signatures, and examining three network packet captures. The network analysis focused on HTTP communications, FTP file-transfer activity, and interactive Telnet sessions.

The USB image was identified as a FAT16 filesystem. Two allocated text files were identified and extracted for examination. A review of unallocated data located two GIF-signature candidates that were subsequently carved and structurally examined. Network analysis identified web requests, FTP upload activity involving several filenames, and readable data exchanged during three Telnet TCP streams. The packet captures also contained an HTTP transaction requesting and receiving a file named rhino.exe, with response data consistent with a Windows executable.

These findings establish the presence of relevant filesystem and network artefacts within the supplied evidence. However, filenames and protocol activity alone do not establish

the complete contents, purpose, execution, or real-world attribution of the associated activities. The findings are therefore presented according to the evidence examined and the limitations of the analysis.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment was to apply digital forensic techniques to a supplied USB disk image and associated network packet captures in order to identify, extract, and interpret potentially relevant digital artefacts. The investigation aimed to demonstrate the use of forensic tools for filesystem examination, file recovery, integrity verification, and network traffic analysis while maintaining a clear distinction between observed evidence and conclusions drawn from that evidence.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the investigation were to:

1. Establish an organised evidence and working-directory structure for the investigation.
2. Verify the integrity of the supplied archive and calculate hashes for relevant evidence files.
3. Identify the filesystem structure and examine allocated and potentially recoverable artefacts within the USB disk image.
4. Extract and document selected files and their associated metadata.
5. Examine unallocated data for recognised file signatures and assess recovered file structures.
6. Identify the types, timelines, endpoints, and protocols represented in the supplied packet captures.
7. Examine HTTP requests and responses, FTP control and data communications, and Telnet session data.
8. Preserve relevant command outputs and analytical results for reporting.
9. Document findings, methodological limitations, and conclusions without making unsupported claims about individual identity or activity.

METHODOLOGY

The investigation followed a structured, command-line-based forensic workflow in Kali Linux. An evidence directory and separate investigation output locations were established to organise the supplied materials and generated results. The downloaded archive was checked for integrity before its contents were extracted, and SHA-256 hashes were calculated to support identification and subsequent comparison of the evidence files.

The USB disk image was examined using filesystem identification and forensic analysis utilities. The filesystem structure, allocated entries, file metadata, and selected file contents were reviewed. Relevant text files were extracted using forensic tools, while unallocated data was examined for readable strings and recognised file signatures. Candidate GIF files were carved and evaluated using file-identification and structural-analysis utilities without relying on visual inspection of their contents.

The three network packet captures were examined using Wireshark command-line utilities, including capinfos and tshark. Capture metadata, protocol distributions, timestamps, endpoints, HTTP transactions, FTP control and data streams, and Telnet communications were reviewed. Relevant outputs were saved for documentation and further interpretation. Findings were correlated across timestamps, stream identifiers, endpoints, and protocol details where appropriate.

Throughout the investigation, conclusions were limited to what the available artefacts and tool outputs supported. The examination did not establish the identity of a real-world operator, prove that the requested executable was executed, or independently verify the contents of every file referenced in network traffic.

TOOLS AND RESOURCES USED

The following tools and resources were used during the investigation:

Tool or resource	Purpose
Kali Linux	Digital forensic examination environment
Terminal and shell utilities	Directory management, command execution, and output organisation

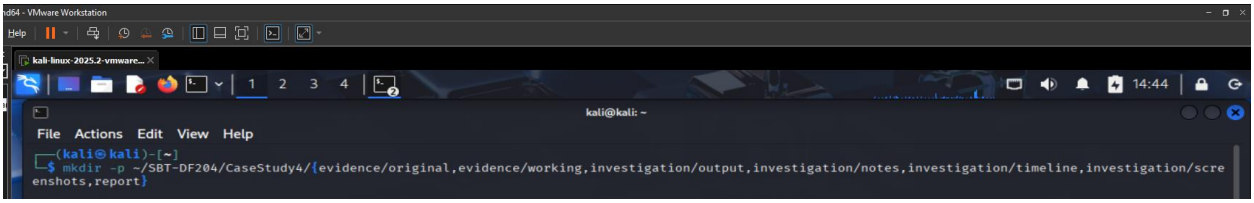
sha256sum	Calculation of SHA-256 hashes
unzip	Archive extraction and integrity testing
file	Identification of file types and signatures
mmls and fsstat	Examination of disk layout and filesystem metadata
fls and istat	Filesystem entry listing and metadata examination
icat	Extraction of file content from the disk image
blkls	Extraction of unallocated filesystem data
strings, grep, xxd, and awk	Text, signature, hexadecimal, and structured-output examination
Foremost	File carving based on recognised signatures
ImageMagick identify and giftext	Structural examination of recovered GIF candidates
capinfos	Packet-capture metadata and statistics
tshark	Protocol, endpoint, timestamp, and session analysis
Supplied USB image and PCAP files	Primary evidence sources

The tools were used to examine the supplied evidence and generate supporting artefacts. Tool output was interpreted in context rather than treated as conclusive proof of activity beyond what the evidence demonstrated.

Creation of the Case Directory Structure

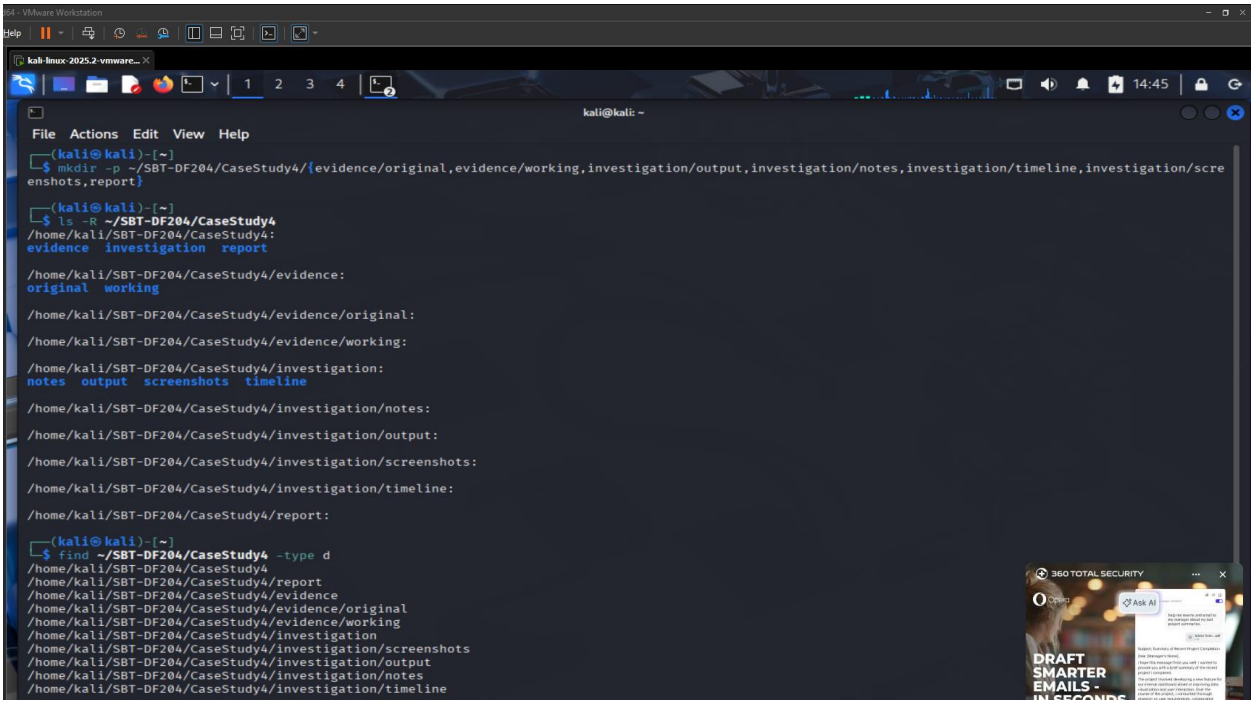
The case directory structure was created by executing the mkdir -p command in the Kali Linux terminal. The command established the main CaseStudy4 directory under the user's home directory, together with separate subdirectories for evidence preservation, working copies, investigation outputs, examination notes, timeline development, screenshots, and report preparation. The evidence/original directory was designated for the preserved original course evidence, while evidence/working was reserved for verified copies to be used during forensic examination. The investigation/output directory was created to store relevant tool outputs, recovered files, and reconstructed network objects. The investigation/notes and investigation/timeline directories were established to

maintain examination notes and develop a chronological account of material events. The investigation/screenshots directory was designated for screenshots documenting the procedures and results, while the report directory was created to hold the draft and final forensic reports. This separation provides an organised structure for managing the different stages of the investigation and maintaining links between the evidence examined, the procedures performed, and the findings reported.



Verification of the Directory Structure

Following the creation of the directory structure, the ls -R ~/SBT-DF204/CaseStudy4 command was executed to display the case directory and its subdirectories recursively. The output confirmed the presence of the main evidence, investigation, and report directories, together with their expected subdirectories. An additional verification was performed using the find ~/SBT-DF204/CaseStudy4 -type d command, which returned the paths of the case directory and all its subdirectories. The results of both commands confirmed that the required directory hierarchy had been created successfully. At the time of verification, the directories were empty, indicating that evidence acquisition and substantive forensic examination had not yet commenced. These checks established that the investigation workspace was ready for the subsequent stages of the examination.



Assessment of Available Storage Capacity

An assessment of the available storage capacity was performed using the df -h ~ command, which reports filesystem capacity and utilisation in a human-readable format for the filesystem containing the user's home directory. The command output identified /dev/sda1 as the relevant filesystem, with a reported total capacity of 79 GB, approximately 69 GB used, and 6.0 GB available. The filesystem utilisation was reported as 93%, indicating that the available storage was limited relative to the potential requirements of the investigation. This assessment was necessary because the supplied USB disk image, recovered files, reconstructed FTP and HTTP objects, and intermediate examination outputs may require substantial storage. Attempting to acquire a large evidence package without first confirming the available capacity could interrupt the acquisition process or prevent subsequent examination activities from being completed. Consequently, the size of the supplied evidence package must be established before downloading the materials. If the available capacity is insufficient, an alternative storage location or an appropriately expanded virtual disk will need to be considered before proceeding.

```
┌──(kali㉿kali)-[~]
└─$ df -h ~
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        79G   69G  6.0G  93% /

┌──(kali㉿kali)-[~]
└─$
```

Evidence Acquisition

The evidence package, DFRWS2005-RODEO.zip, was acquired from the frankwxu/digital-forensics-lab GitHub repository under the Illegal_Possession_Images/USB_image directory. The acquisition was performed from the Kali Linux terminal using the wget utility. The server returned an HTTP 200 OK response, and the download completed successfully, with 3,614,418 bytes recorded as saved. The archive was stored in the designated evidence/original directory within the Case Study 4 workspace. This directory was selected to retain the received evidence package separately from the working files that would be used for subsequent examination. The source used for acquisition was documented because it was an alternative to the initial OneDrive link, which had returned an HTTP 403 Forbidden response.

```
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ wget -O DFRWS2005-RODEO.zip \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/USB_image/DFRWS2005-RODEO.zip'
--2026-10-26 15:05:12--  https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/USB_image/DFRWS2005-RODEO.zip
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3614418 (3.4M) [application/zip]
Saving to: 'DFRWS2005-RODEO.zip'

DFRWS2005-RODEO.zip                 100%[═══════════════════════════════════════════════════════════════════════════>]   3.45M   568KB/s    in 5.6s

2026-10-26 15:05:18 (633 KB/s) - 'DFRWS2005-RODEO.zip' saved [3614418/3614418]
```

Evidence File Identification

Following acquisition, the Linux file utility was used to identify the format of DFRWS2005-RODEO.zip. The output identified the file as a ZIP archive created using a Unix-compatible ZIP format and reported that the compressed archive used the DEFLATE method. The utility also reported an uncompressed size of 259,506,176 bytes and a last-modified timestamp of 30 April 2004, 20:29:48. These details provided an initial technical description of the acquired package and helped establish the expected format before further verification. The timestamp was treated as archive metadata rather than evidence of the date on which the package was acquired.

```
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ file DFRWS2005-RODEO.zip
DFRWS2005-RODEO.zip: Zip archive data, made by v2.3 UNIX, extract using at least v2.0, last modified Apr 30 2004 20:29:48, uncompressed size 259506176, method=deflate
```

Cryptographic Hash Calculation

The SHA-256 hash of DFRWS2005-RODEO.zip was calculated using the Linux sha256sum utility to establish a reference value for subsequent integrity checks. The resulting hash was e83459a800baed6a3c37551d413ab59fe8dac54c879ed5408001cfed171a0aaa. This value was recorded in the investigation notes as the hash of the downloaded archive. Recalculating the hash at a later stage will allow the investigator to determine whether the archive has changed since the reference value was recorded. However, the hash alone does not prove that the downloaded package is identical to the version originally supplied by the course instructor.



Archive Integrity Verification

The archive was tested using the unzip -t command to determine whether its compressed contents could be read without reported data errors. The test returned an OK result for each of the four entries: RHINOUSB.dd, rhino.log, rhino2.log, and rhino3.log. The final output stated that no errors were detected in the compressed data. This result indicates that the archive passed the utility's compressed-data integrity test and was suitable for proceeding to controlled extraction. The original ZIP file remained in the evidence/original directory, separate from the working copies intended for examination.

Evidence Package Inventory

The contents of the acquired archive were listed using the unzip -l command without extracting or modifying the files. The inventory identified four evidence items. RHINOUSB.dd had an uncompressed size of 259,506,176 bytes, rhino.log had a size of 3,187,907 bytes, rhino2.log had a size of 292,604 bytes, and rhino3.log had a size of 226,094 bytes. The combined uncompressed size was reported as 263,212,781 bytes. The disk image will form the basis of the disk and file-system examination, while the three log files will be examined as network-related artefacts. At this stage, the files were

inventoried only; no conclusions about their contents, user activity, or the identity of any individual were drawn.

```
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ file DFRWS2005-RODEO.zip
DFRWS2005-RODEO.zip: Zip archive data, made by v2.3 UNIX, extract using at least v2.0, last modified Apr 30 2004 20:29:48, uncompressed size 259506176, me
thod=deflate

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ sha256sum DFRWS2005-RODEO.zip
e83459a800baed6a3c37551d413ab59fe8dac54c879ed5408001cfed171a0aaa  DFRWS2005-RODEO.zip

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ unzip -t DFRWS2005-RODEO.zip
Archive:  DFRWS2005-RODEO.zip
    testing: RHINOUSB.dd            OK
    testing: rhino.log              OK
    testing: rhino2.log             OK
    testing: rhino3.log             OK
```

Evidence Extraction and Working-Copy Preparation

Following the successful integrity test of the acquired archive and the assessment of available storage, DFRWS2005-RODEO.zip was extracted into the dedicated working directory at /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO.

```
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ df -h ~
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        79G   69G  5.9G  93% /

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ mkdir -p ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ unzip ~/SBT-DF204/CaseStudy4/evidence/original/DFRWS2005-RODEO.zip \
-d ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
Archive:  /home/kali/SBT-DF204/CaseStudy4/evidence/original/DFRWS2005-RODEO.zip
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/RHINOUSB.dd
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino.log
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino2.log
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino3.log
```

The extraction was completed using the Linux unzip utility, which reported successful extraction of all four entries: RHINOUSB.dd, rhino.log, rhino2.log, and rhino3.log. The original ZIP archive was retained separately in the evidence/original directory.

```
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ mkdir -p ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ unzip ~/SBT-DF204/CaseStudy4/evidence/original/DFRWS2005-RODEO.zip \
-d ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
Archive:  /home/kali/SBT-DF204/CaseStudy4/evidence/original/DFRWS2005-RODEO.zip
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/RHINOUSB.dd
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino.log
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino2.log
  inflating: /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino3.log

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ ls -lh ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/
total 252M
-rw-rw-r-- 1 kali kali 286K Apr 28  2004 rhino2.log
-rw-rw-r-- 1 kali kali 221K Apr 28  2004 rhino3.log
-rw-rw-r-- 1 kali kali 3.1M Apr 26  2004 rhino.log
-rw-r--r-- 1 kali kali 248M Apr 30  2004 RHINOUSB.dd

┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$ sha256sum ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/*
41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde  /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino2.log
7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46  /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino3.log
64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2  /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/rhino.log
ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65  /home/kali/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO/RHINOUSB.dd

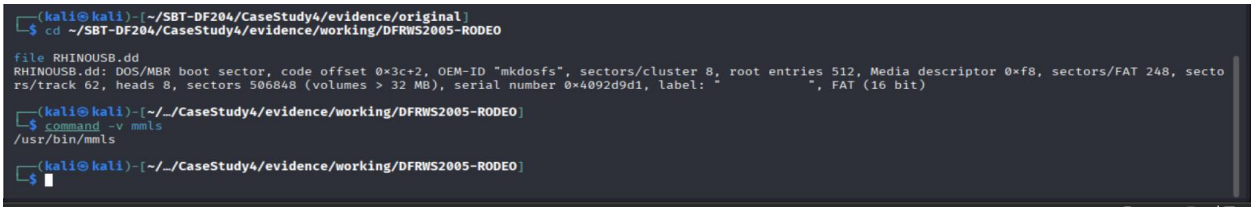
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy4/evidence/original]
└─$
```

A subsequent directory listing confirmed that the extracted files were present. The disk image, RHINOUSB.dd, was displayed as approximately 248 MiB, while the three log files

were displayed as approximately 3.1 MiB, 286 KiB, and 221 KiB respectively. SHA-256 hashes were then calculated for each extracted file using the sha256sum utility. These hash values establish reference points for checking the integrity of the working artefacts throughout the investigation. At this stage, the files had been extracted and inventoried; their substantive contents had not yet been analyzed, and no conclusions about the events under investigation were drawn.

Preliminary Identification of the USB Disk Image

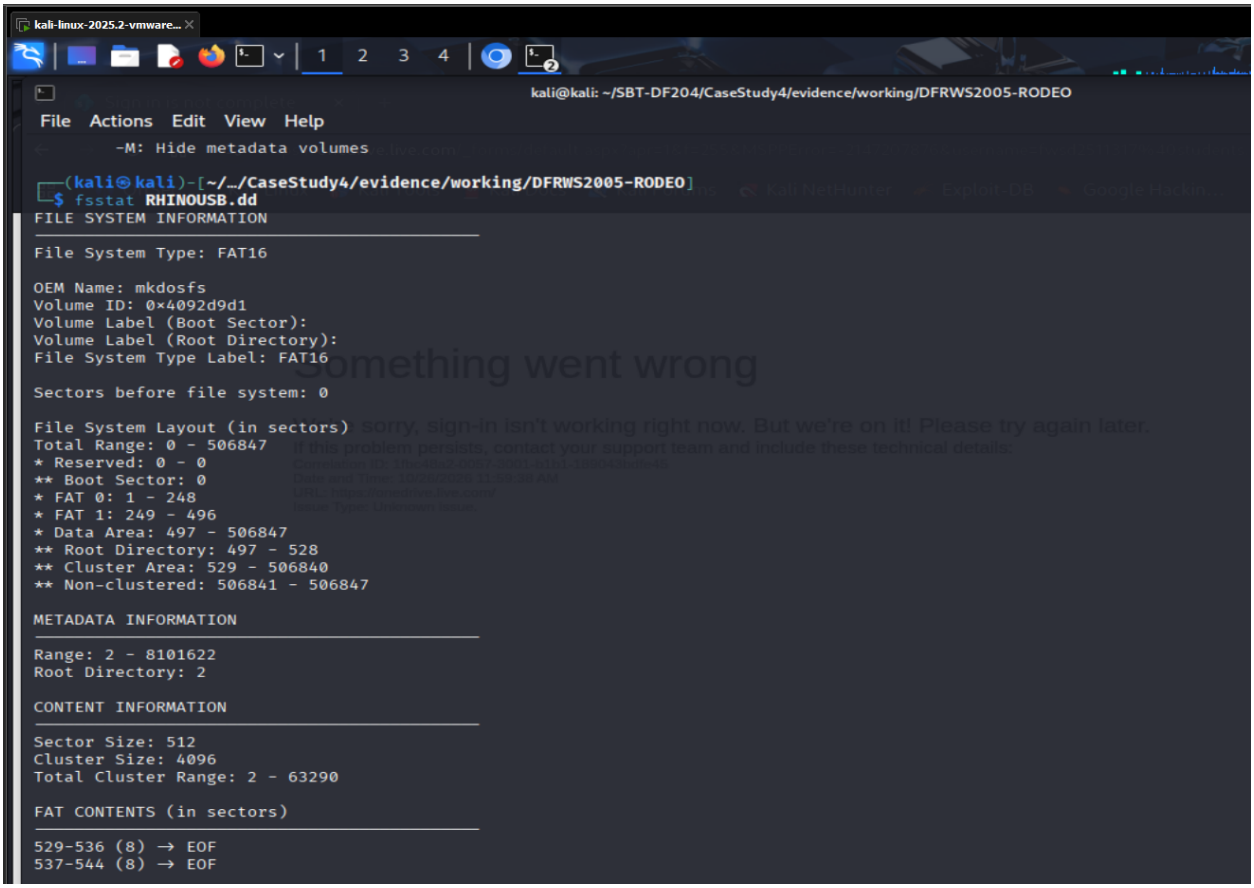
The file utility was used to examine RHINOUSB.dd, the disk image extracted from the evidence archive. The output identified a DOS/MBR boot-sector structure and reported FAT16 filesystem characteristics, including eight sectors per cluster, 512 root-directory entries, and a FAT size of 248 sectors. The image was also associated with the OEM identifier mkdosfs and the volume serial number 0x4092d9d1. These findings provide an initial technical description of the image; however, the partition layout and filesystem boundaries must be examined separately before further analysis. No files have been recovered or interpreted at this stage.



Preliminary identification of the USB disk image using the file utility.

Filesystem Structure and Volume Analysis

The Sleuth Kit utilities mmls and fsstat were used to investigate the structure of RHINOUSB.dd. The mmls command returned exit status 1 without displaying a partition layout. Subsequent examination with fsstat successfully identified a FAT16 filesystem, reporting zero sectors before the filesystem and a total sector range of 0–506847. The reported sector size was 512 bytes, while the cluster size was 4096 bytes. The filesystem layout placed the two FAT copies in sectors 1–248 and 249–496, respectively, with the data area beginning at sector 497. The root directory occupied sectors 497–528. These results indicate that filesystem analysis can proceed using sector zero as the starting offset. The failure of mmls to display a partition layout is recorded separately and is not treated as proof that the image is invalid.



FAT16 filesystem structure and sector layout identified using fsstat.

The key findings are:

- Filesystem: FAT16
- Volume ID: 0x4092d9d1 - matches the ID reported by file
- Sectors before filesystem: 0
- Sector size: 512 bytes
- Cluster size: 4,096 bytes (4 KiB)
- Filesystem sector range: 0–506847
- Data area: sectors 497–506847
- Root directory: sectors 497–528
- Image size: approximately 248 MiB

Initial Filesystem Directory Listing

The fls utility from The Sleuth Kit was used to recursively enumerate directory entries in RHINOUSB.dd. The output was saved to investigation/output/usb_file_listing.txt for documentation and subsequent analysis. The command returned six entries: two regular

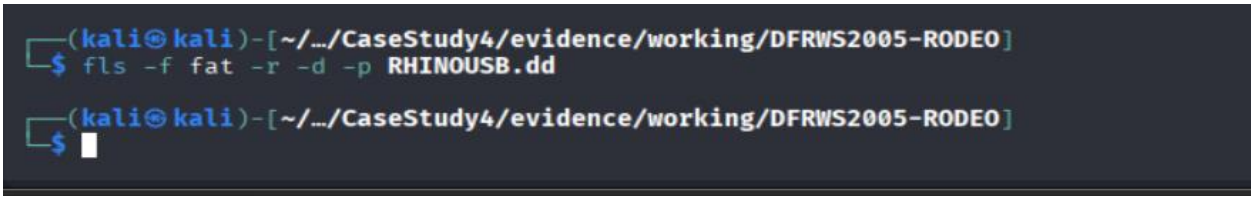
files, gumbo1.txt and gumbo2.txt, and four special filesystem entries identified as $MBR, $FAT1, $FAT2, and $OrphanFiles. The regular files were recorded as the initial visible file entries in the filesystem listing. The special entries represent filesystem or forensic-analysis structures and should not be interpreted as ordinary user-created files. Further examination is required to determine whether deleted directory entries or orphaned file data are present.



Initial recursive directory listing of the FAT16 USB image using fls.

Examination of Deleted Filesystem Entries

The fls utility was executed with the -d option to identify directory entries marked as deleted within the FAT16 filesystem. The command returned no entries. This indicates that no deleted directory entries were reported by this particular examination. However, the result does not establish that deleted content is absent from the disk image, because file data may remain in unallocated clusters or may no longer have an intact directory entry. Further examination of filesystem allocation metadata and orphaned or unallocated data is therefore required before reaching a conclusion about potential file recovery.



Filesystem Structure and Allocation Metadata

The fsstat utility was used to examine the filesystem metadata of RHINOUSB.dd. The output identified the filesystem as FAT16, with volume ID 0x4092d9d1, a sector size of 512 bytes, and a cluster size of 4096 bytes. The reported filesystem range was sectors 0–506847, with zero sectors preceding the filesystem. The layout showed the reserved boot sector at sector 0, the first FAT copy in sectors 1 - 248, the second FAT copy in sectors 249 - 496, and the data area beginning at sector 497. The root directory was

reported in sectors 497 - 528, while the cluster area extended across sectors 529–506840.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ fls -f fat -r -p -o 0 RHINOUSB.dd
r/r 4:  gumbo1.txt
r/r 6:  gumbo2.txt
v/v 8101619:    $MBR
v/v 8101620:    $FAT1
v/v 8101621:    $FAT2
V/V 8101622:    $OrphanFiles
```

These results provide a structural reference for subsequent filesystem examination and potential recovery work. The FAT contents displayed by fsstat included two allocated cluster chains, each consisting of eight sectors and terminating at end-of-file. This output alone does not establish the complete contents or identity of the associated files. Further analysis is required before drawing conclusions about deleted content, orphaned data, or the significance of any recovered artefacts.

```
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODE
File  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ fsstat RHINOUSB.dd
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT16

OEM Name: mkdosfs
Volume ID: 0x4092d9d1
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: FAT16

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 506847
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 248
* FAT 1: 249 - 496
* Data Area: 497 - 506847
** Root Directory: 497 - 528
** Cluster Area: 529 - 506840
** Non-clustered: 506841 - 506847

METADATA INFORMATION
--------------------------------------------
Range: 2 - 8101622
Root Directory: 2

CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 2 - 63290

FAT CONTENTS (in sectors)
--------------------------------------------
529-536 (8) → EOF
537-544 (8) → EOF
```

Examination of File Metadata and Directory Entries

The istat utility was used to examine the metadata associated with the two regular files identified in the FAT16 filesystem. Directory entry 4 was associated with GUMBO1.TXT, which was reported as an allocated file with a size of 2,815 bytes and the File and Archive attributes. Its recorded written and created timestamps were both 30 April 2004 at 18:11:20 EDT, while the access date was recorded as 30 April 2004 at 00:00:00 EDT. The sector list included sectors 529–534, followed by zero entries.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ istat -f fat RHINOUSB.dd 4
Directory Entry: 4
Allocated
File Attributes: File, Archive
Size: 2815
Name: GUMBO1.TXT

Directory Entry Times:
Written:        2004-04-30 18:11:20 (EDT)
Accessed:       2004-04-30 00:00:00 (EDT)
Created:        2004-04-30 18:11:20 (EDT)

Sectors:
529 530 531 532 533 534 0 0
```

Directory entry 6 was associated with GUMBO2.TXT, also reported as allocated with the File and Archive attributes. Its recorded size was 1,293 bytes. The written and created timestamps were both 30 April 2004 at 18:11:24 EDT, and the access date was recorded as 30 April 2004 at 00:00:00 EDT. Its sector list included sectors 537–539, followed by zero entries.

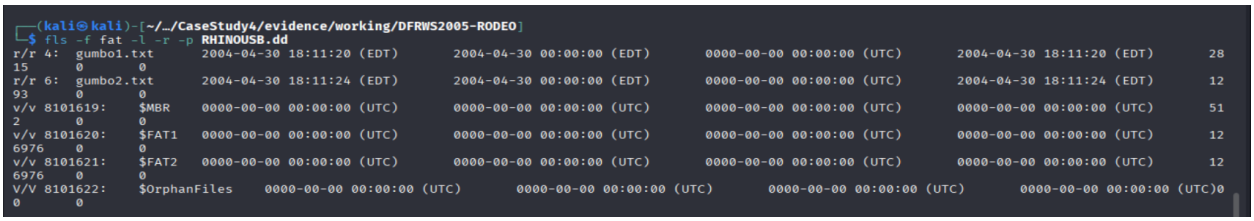
```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ istat -f fat RHINOUSB.dd 6
Directory Entry: 6
Allocated
File Attributes: File, Archive
Size: 1293
Name: GUMBO2.TXT

Directory Entry Times:
Written:        2004-04-30 18:11:24 (EDT)
Accessed:       2004-04-30 00:00:00 (EDT)
Created:        2004-04-30 18:11:24 (EDT)

Sectors:
537 538 539 0 0 0 0 0
```

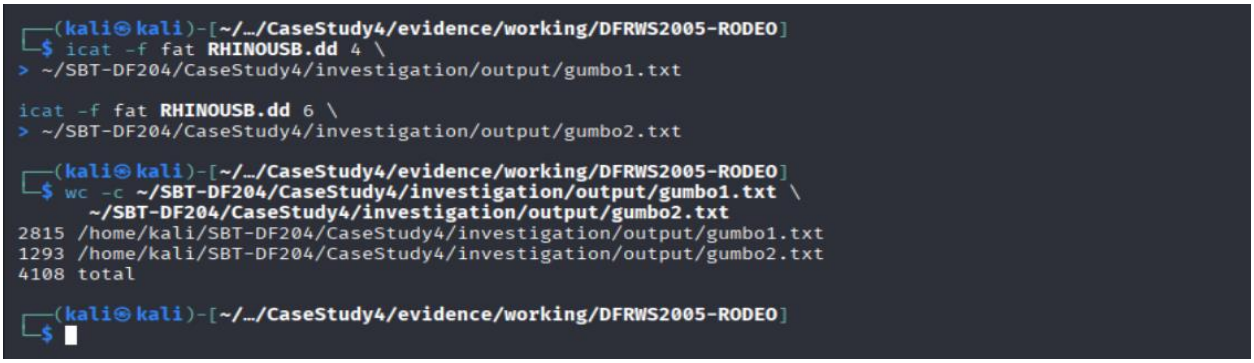
The fls -l output independently displayed both files, their recorded timestamps, and their sizes, supporting the metadata observations obtained with istat. The four-second difference between the recorded written timestamps is a useful point for subsequent timeline correlation. These timestamps represent filesystem metadata and should not, in isolation, be treated as proof of when a person created, accessed, or intentionally handled

the files. The files were reported as allocated at the time of examination; no conclusions about their content or relevance can be drawn from metadata alone.



3.8 Extraction and Verification of Text Files

The icat utility was used to extract the contents associated with directory entries 4 and 6 from the FAT16 disk image. The extracted files were saved as gumbo1.txt and gumbo2.txt in the investigation output directory, leaving the disk image unchanged. The wc -c command was subsequently used to verify the byte counts of the extracted files. gumbo1.txt was measured at 2,815 bytes, while gumbo2.txt was measured at 1,293 bytes, giving a combined total of 4,108 bytes. Both extracted sizes matched the values reported by istat during the earlier metadata examination. This agreement supports the completeness of the extraction by byte count, although a cryptographic hash comparison between the extracted files and independently extracted reference data would provide an additional integrity check.



File Type Identification and Cryptographic Hashing

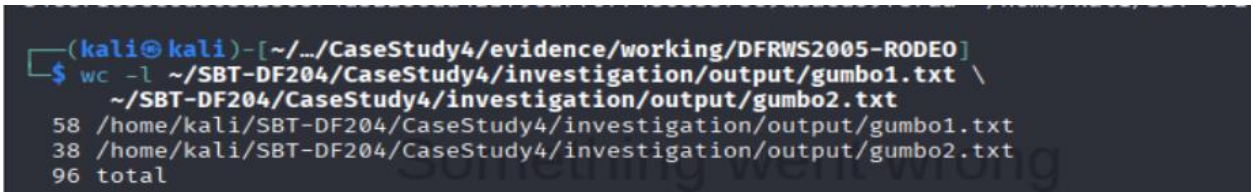
The file utility was used to identify the formats of the two extracted text files. gumbo1.txt was classified as non-ISO extended-ASCII text with very long lines, while gumbo2.txt was identified as ISO-8859 text. These classifications describe the byte patterns detected by the utility and do not, by themselves, establish the meaning or evidential significance of the contents.

SHA-256 hashes were then calculated for both extracted files. The hash for gumbo1.txt was ecb06cec73218f66d19cccfd1e45ab7144b02eb5344d9ede14e1c8428d76713c, while the hash for gumbo2.txt was 84cc7105cc6d663d230574d32256dd42bf96d7f077456e357ec9daacd59f87aa. These values provide reference points for verifying whether the extracted files change during later analysis. The hashes and file-type results were recorded before examining the text contents.

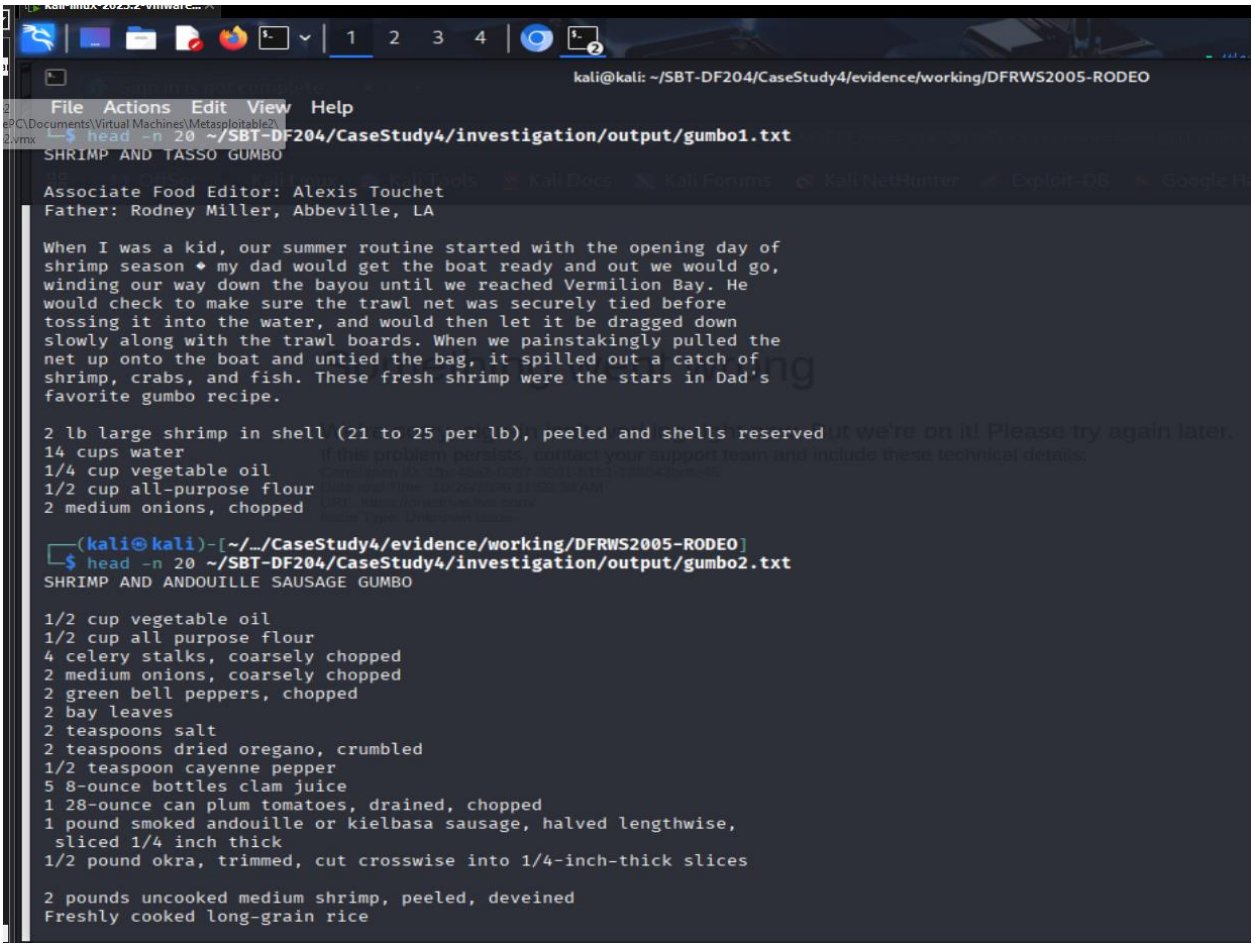


Preliminary Content Examination of Extracted Text Files

The extracted text files were examined using the wc -l and head utilities to establish their line counts and inspect a limited portion of their contents. gumbo1.txt contained 58 lines and began with the heading “SHRIMP AND TASSO GUMBO.” Its visible content included recipe attribution, a narrative describing shrimp fishing in Louisiana, and an ingredient list. gumbo2.txt contained 38 lines and began with the heading “SHRIMP AND ANDOUILLE SAUSAGE GUMBO,” followed by a list of ingredients for a different gumbo recipe.



During the preview of gumbo1.txt, a replacement character appeared within the narrative text. This may indicate a character-encoding or display issue and should not be interpreted as evidence of file corruption without further examination. The observed excerpts are consistent with ordinary culinary recipe content. However, only the first 20 lines of each file were displayed during this stage, so this observation is limited to the portions examined and should not be generalized to the complete files without further review.

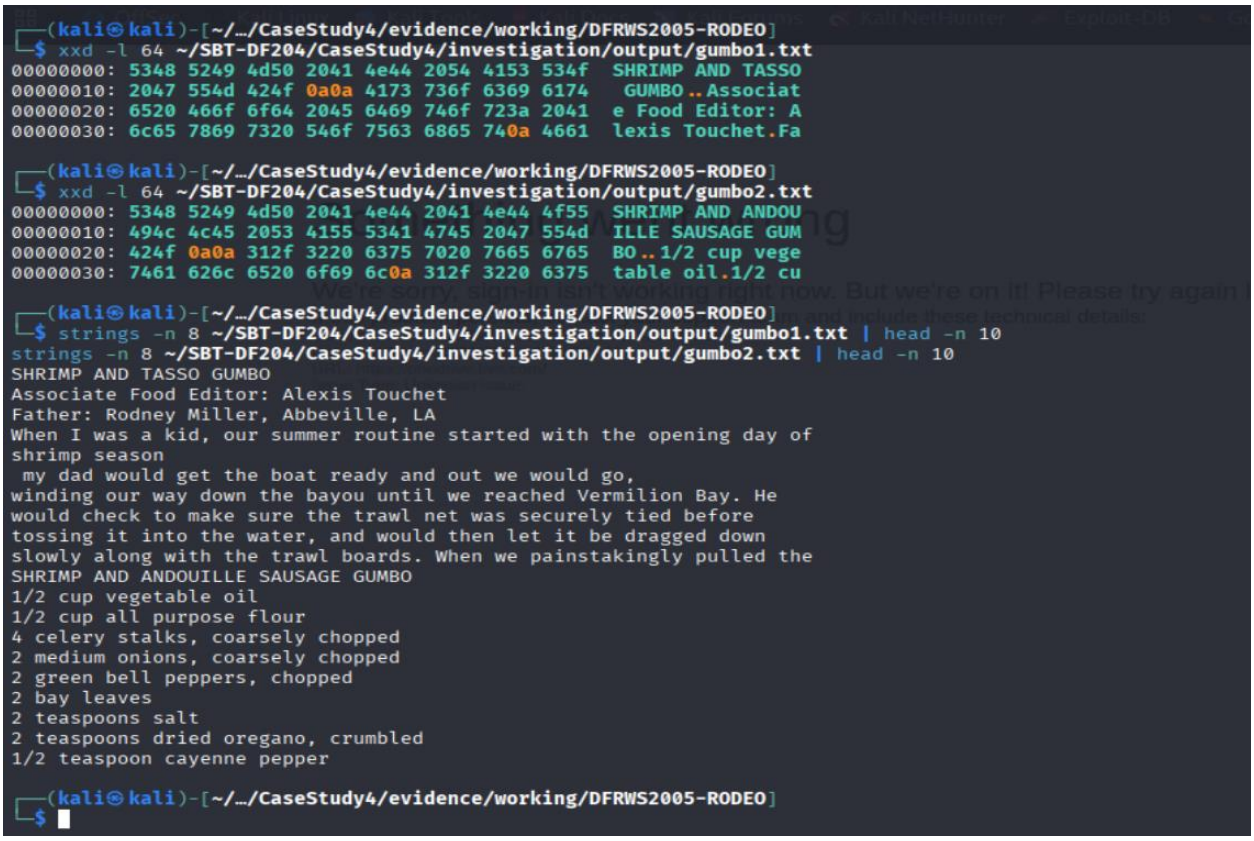


Hexadecimal and Printable-String Examination

The xxd utility was used to inspect the first 64 bytes of each extracted text file in hexadecimal and character format. The output for gumbo1.txt began with the byte sequence corresponding to the text “SHRIMP AND TASSO GUMBO,” while gumbo2.txt began with the sequence corresponding to “SHRIMP AND ANDOUILLE SAUSAGE GUMBO.” Both previews contained readable ASCII characters and newline bytes, consistent with the file-type classifications obtained earlier.

The strings utility was subsequently used with a minimum string length of eight characters to identify readable sequences within each file. The results included the recipe headings, attribution and narrative text from gumbo1.txt, and ingredient descriptions from gumbo2.txt. These findings support the observation that the examined portions contain readable textual data. The tools were used for inspection only, and the extracted files were not modified. Because the analysis covered only the initial hexadecimal bytes and

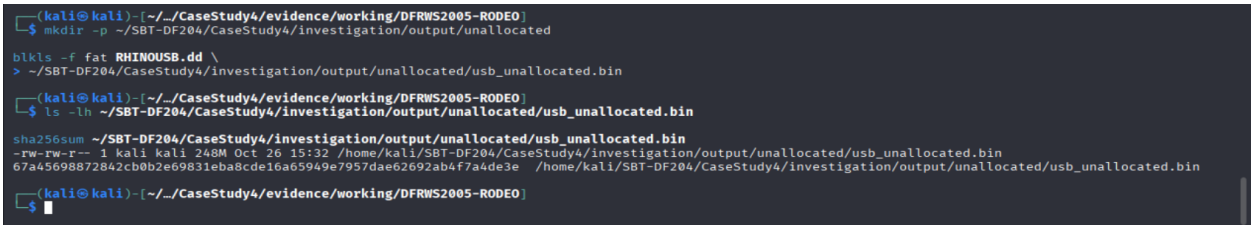
selected printable strings, it does not independently establish that no embedded or non-text data exists elsewhere in either file.



Extraction and Preservation of Unallocated Filesystem Data

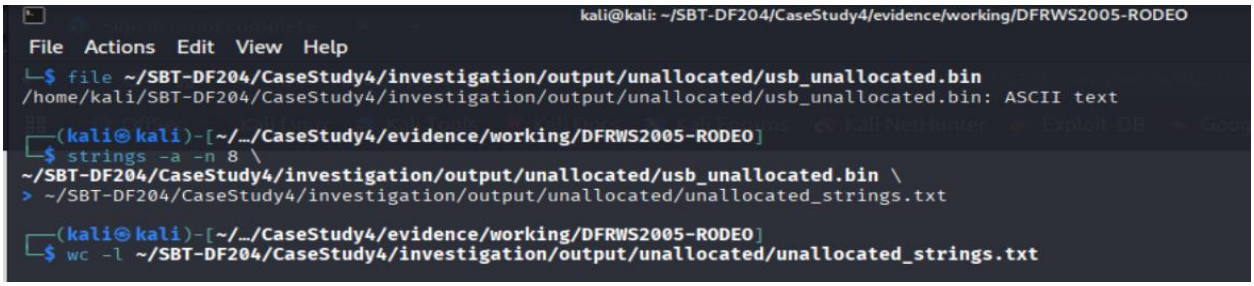
The blkls utility from The Sleuth Kit was used to extract unallocated filesystem data from RHINOUSB.dd. The extracted output was stored separately at investigation/output/unallocated/usb_unallocated.bin, preserving the disk image as the source artefact. The resulting file was reported as approximately 248 MiB, and its SHA-256 hash was calculated as 67a45698872842cb0b2e69831eba8cde16a65949e7957dae62692ab4f7a4de3e. The hash provides a reference value for checking whether this extracted artefact changes during subsequent analysis.

The extraction creates a separate dataset for examining residual data that may not be associated with currently allocated files. The reported file size alone does not establish how much useful residual information is present, whether deleted files can be reconstructed, or whether any particular content is relevant to the investigation. Further examination of the extracted data is required before making such determinations.



Preliminary Examination of Unallocated Data

The unallocated data extracted from the USB disk image was subjected to an initial file-type identification and printable-string examination. The file utility identified usb_unallocated.bin as ASCII text. The strings utility was then used with the -a option to examine the entire extracted file and the -n 8 option to display printable character sequences of at least eight characters. The resulting output contained 583 lines.



The initial review revealed a mixture of readable text and character sequences that did not immediately convey meaningful information. One readable string was copyright 2000 philg@mit.edu. This string was documented as a potential lead for subsequent analysis; however, its presence alone does not establish the identity of the original file, its author, or its relationship to the case. Similarly, identifying the extracted data as ASCII text does not establish that the underlying data consists entirely of ordinary text, since printable fragments can also occur within binary data.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ wc -l ~/SBT-DF204/CaseStudy4/investigation/output/unallocated/unallocated_strings.txt

head -n 30 ~/SBT-DF204/CaseStudy4/investigation/output/unallocated/unallocated_strings.txt
583 /home/kali/SBT-DF204/CaseStudy4/investigation/output/unallocated/unallocated_strings.txt
copyright 2000 philg@mit.edu
 $.' ",#
(7),01444
'9=82<.342
!2222222222222222222222222222222222222222222222222
UE!vI-N=
:ChbP2+r
5LhI3Gnp
FRpO4S@h
v'hBivc^
qOi"l=Txdg
qRwtQuc;]8[
YKc$;cGH
h44YL6lPu
8=+Jl   "V
wbIb}MQ-
zZ9Wd@XFgeI
[#;oCc::
|gXemJf#j
G"K     +4Gp
{WBV3e:^
tQ{rfhcf6
^nGRg6I(
9-#uHss*
wERjFx|(
(4R3L^#(
kH<Xr|?x
2C+yYdEP
P[8POJ_c
Egf|I%By
```

The results of this examination were preserved in investigation/output/unallocated/unallocated_strings.txt for further review. Additional analysis is required to determine whether the extracted data contains remnants of deleted files, recognisable file signatures, or other artefacts relevant to the investigation. No conclusion about the presence or nature of any suspected image should be drawn from this preliminary output alone.

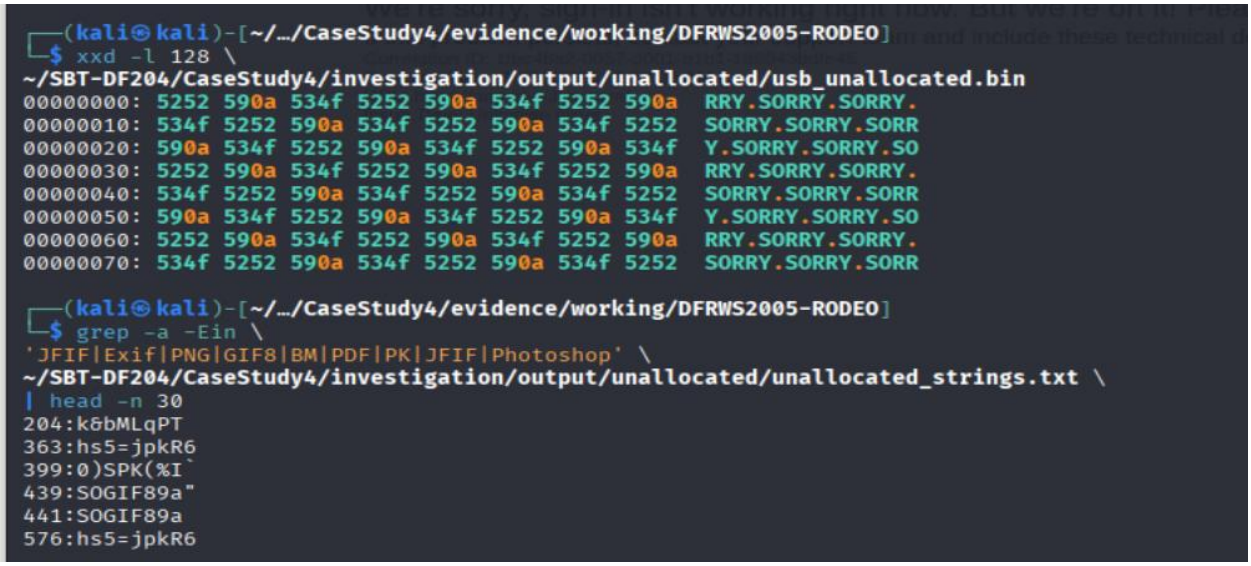
Hexadecimal and Signature-String Examination of Unallocated Data

The extracted unallocated-data file was examined using the xxd utility to inspect its first 128 bytes in hexadecimal and ASCII representation. The output showed repeated occurrences of the ASCII string SORRY, separated by line-feed characters. This establishes that the beginning of the extracted data contains a repeated text pattern, although its original purpose and context have not yet been determined.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ xxd -l 128 \
~/SBT-DF204/CaseStudy4/investigation/output/unallocated/usb_unallocated.bin
00000000: 5252 590a 534f 5252 590a 534f 5252 590a  RRY.SORRY.SORRY.
00000010: 534f 5252 590a 534f 5252 590a 534f 5252  SORRY.SORRY.SORR
00000020: 590a 534f 5252 590a 534f 5252 590a 534f  Y.SORRY.SORRY.SO
00000030: 5252 590a 534f 5252 590a 534f 5252 590a  RRY.SORRY.SORRY.
00000040: 534f 5252 590a 534f 5252 590a 534f 5252  SORRY.SORRY.SORR
00000050: 590a 534f 5252 590a 534f 5252 590a 534f  Y.SORRY.SORRY.SO
00000060: 5252 590a 534f 5252 590a 534f 5252 590a  RRY.SORRY.SORRY.
00000070: 534f 5252 590a 534f 5252 590a 534f 5252  SORRY.SORRY.SORR
```

A preliminary search of the printable-string output was also conducted for terms associated with common file formats. The search returned two entries resembling the GIF89a signature at lines 439 and 441, alongside other incidental character matches.

Since printable-string searches do not preserve the original byte offsets and can identify coincidental sequences, these results are not sufficient to confirm the presence of a GIF file. The potential signature matches require validation against the original unallocated-data file using byte-level examination or a file-carving utility.



The findings were recorded as preliminary leads for further investigation. No file was opened or rendered during this stage, and no conclusion was drawn about the contents of any potential recovered image. Further examination will focus on locating and validating file signatures in the extracted data while preserving the original USB disk image.

Locating Potential GIF Signatures in Unallocated Data

A byte-level search was performed against the extracted unallocated-data file to locate the exact ASCII byte sequence GIF89a, a signature associated with the GIF89a image format. The search identified two candidate offsets: decimal 54,435,840 (hexadecimal 0x33EA000) and decimal 54,448,128 (hexadecimal 0x33ED000). The candidates are separated by 12,288 bytes.

These results provide specific locations for further examination of the unallocated data. At this stage, the findings are recorded as potential GIF signatures rather than confirmed recovered images. The presence of a signature does not by itself demonstrate that a file is complete, valid, or relevant to the case. The surrounding bytes and any available file-structure information must be examined before making a stronger determination. The

original USB disk image remains unchanged, and the analysis has been conducted on the separately extracted unallocated-data artefact.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ python3 - <<'PY'
from pathlib import Path

p = Path.home() / "SBT-DF204/CaseStudy4/investigation/output/unallocated/usb_unallocated.bin"
data = p.read_bytes()
sig = b"GIF89a"

offset = 0
count = 0

while True:
    offset = data.find(sig, offset)
    if offset == -1:
        break
    print(f"GIF89a candidate at decimal offset {offset} (0x{offset:x})")
    offset += 1
    count += 1

print(f"Total exact matches: {count}")
PY
GIF89a candidate at decimal offset 54435840 (0×33ea000)
GIF89a candidate at decimal offset 54448128 (0×33ed000)
Total exact matches: 2
```

The raw-data search confirms two exact occurrences of the byte sequence GIF89a in the extracted unallocated data:

Candidate	Decimal offset	Hexadecimal offset
1	54,435,840	0x33EA000
2	54,448,128	0x33ED000

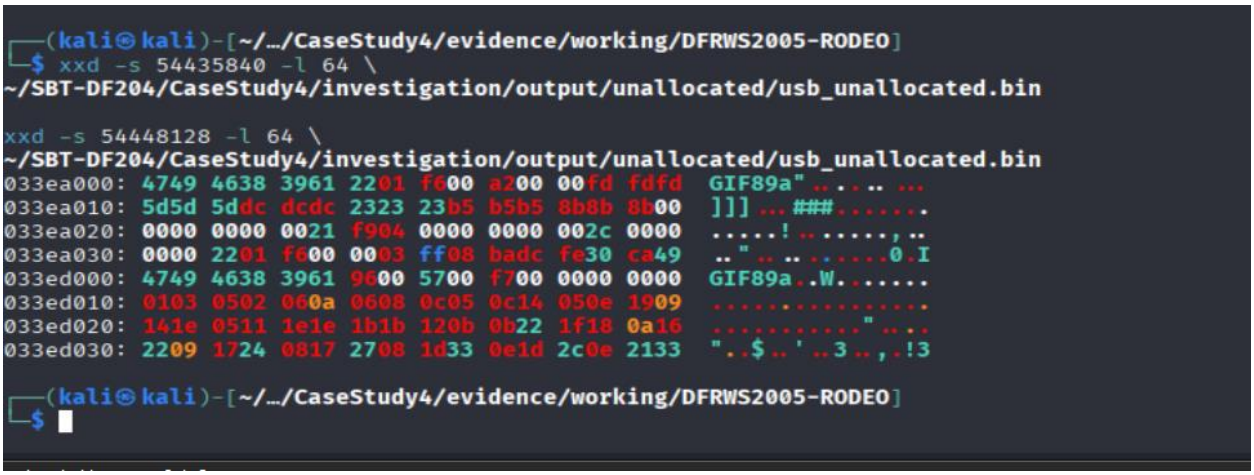
The candidates are 12,288 bytes apart. This is useful forensic evidence, but a GIF signature alone does not establish that either location contains a complete, valid image. The repeated spacing could reflect separate file data, remnants, or another structure; we need to inspect the surrounding bytes and validate the candidates.

Header Examination of Potential GIF Artefacts

A hexadecimal examination was performed at both candidate offsets identified in the extracted unallocated data. The first candidate, located at decimal offset 54,435,840 (0x33EA000), begins with the signature GIF89a. The subsequent bytes indicate a logical screen width of 290 pixels and height of 246 pixels. The second candidate, at decimal offset 54,448,128 (0x33ED000), also begins with GIF89a, followed by dimensions of 150 by 87 pixels.

The presence of the expected signature and plausible logical screen dimensions supports classifying both locations as potential GIF artefacts for further validation. Nevertheless, header examination alone cannot establish that the complete file data is present, that the image structure is valid, or that either artefact is relevant to the alleged activity. The two

candidates were therefore retained for additional structural validation and file-carving analysis, without opening or rendering the images during this stage.



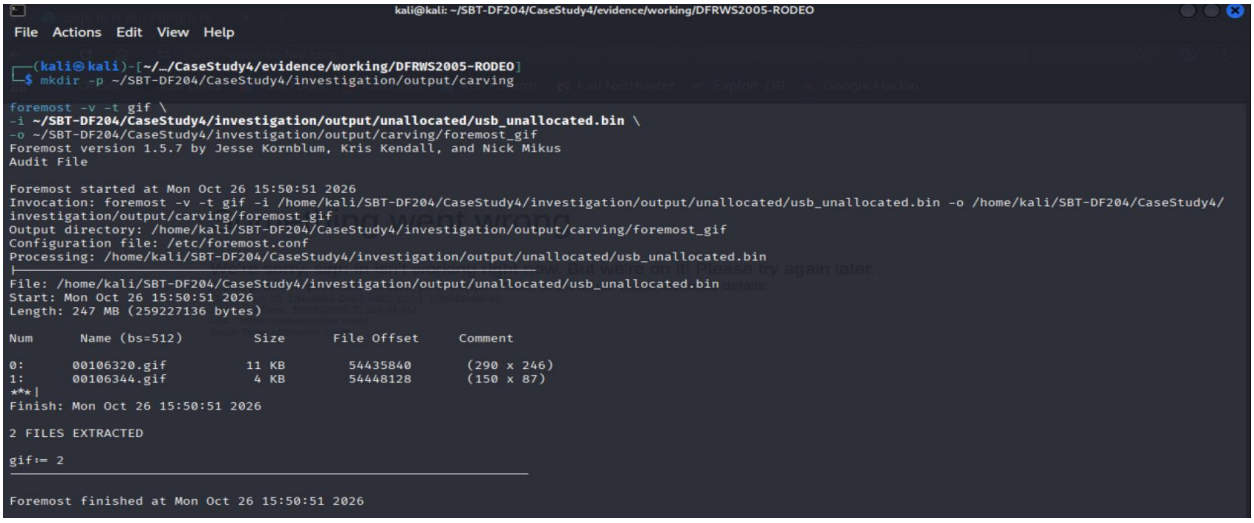
These results provide stronger evidence that the two locations contain data beginning with a GIF89a signature. The bytes at each location, however, look different and should be assessed separately.

- Candidate 1 - 0x33EA000: The header begins GIF89a, followed by bytes 22 01 and f6 00, which decode as a logical screen width of 290 pixels and height of 246 pixels (GIF dimensions are stored in little-endian order). The following bytes are consistent with a packed field, background colour index, and pixel aspect ratio.
- Candidate 2 - 0x33ED000: The header begins GIF89a, followed by 96 00 and 57 00, corresponding to a width of 150 pixels and height of 87 pixels. The following bytes are also consistent with a GIF logical screen descriptor.

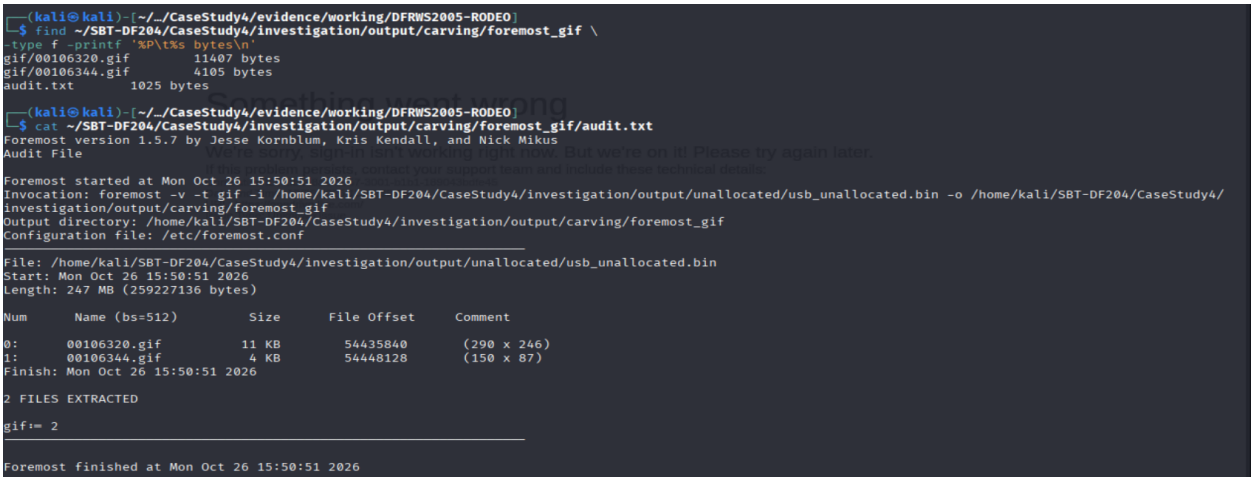
Both candidates therefore have plausible GIF headers and dimensions.

Recovery of GIF Files from Unallocated Data

The Foremost file-carving utility (version 1.5.7) was used to examine the extracted unallocated-data artefact for GIF files. The operation completed successfully and reported two extracted files. The first, 00106320.gif, was recovered from offset 54,435,840 and had a reported size of 11,407 bytes and dimensions of 290 × 246 pixels. The second, 00106344.gif, was recovered from offset 54,448,128 and had a reported size of 4,105 bytes and dimensions of 150 × 87 pixels.



The source offsets correspond to the two GIF89a signature locations previously identified during hexadecimal examination. This agreement between the byte-level search and Foremost's carving report supports the conclusion that two GIF-formatted data candidates were located and extracted from the unallocated-data artefact. The recovered files and Foremost audit report were retained in the investigation output directory for further examination.



Although the carving operation completed successfully, the recovered files have not yet been independently validated for structural completeness or integrity. Their recovery does not establish when or by whom they were created, who may have possessed them, or whether they are related to the alleged activity. These questions require additional artefact analysis and corroboration.

Independent Validation of Recovered GIF Files

The two files recovered by Foremost were examined using ImageMagick's identify utility to verify their recognised file format, dimensions, and byte counts. The utility identified 00106320.gif as GIF format with dimensions of 290 × 246 pixels and a size of 11,407 bytes. It identified 00106344.gif as GIF format with dimensions of 150 × 87 pixels and a size of 4,105 bytes.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ identify -format '%f | format=%m | dimensions=%wx%h | bytes=%b\n' \
~/SBT-DF204/CaseStudy4/investigation/output/carving/foremost_gif/gif/00106320.gif \
~/SBT-DF204/CaseStudy4/investigation/output/carving/foremost_gif/gif/00106344.gif
00106320.gif | format=GIF | dimensions=290×246 | bytes=11407B
00106344.gif | format=GIF | dimensions=150×87 | bytes=4105B

┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ giftext --help
Error in command line parsing - None option Found - '--help'.
Usage: giftext [-v] [-c] [-e] [-z] [-p] [-r] [-h] GifFile
```

These results agree with the dimensions and approximate sizes recorded in the Foremost audit report. The agreement provides independent confirmation that both recovered files are recognised as GIF-formatted data by the installed image-identification utility. However, this check does not conclusively establish that every part of either file is intact or that the files can be fully decoded. Further structural examination is required. The files were assessed through command-line metadata output; neither image was opened or rendered.

The identify results independently confirm that both recovered files are recognised as GIFs, with dimensions and byte counts matching Foremost's report.

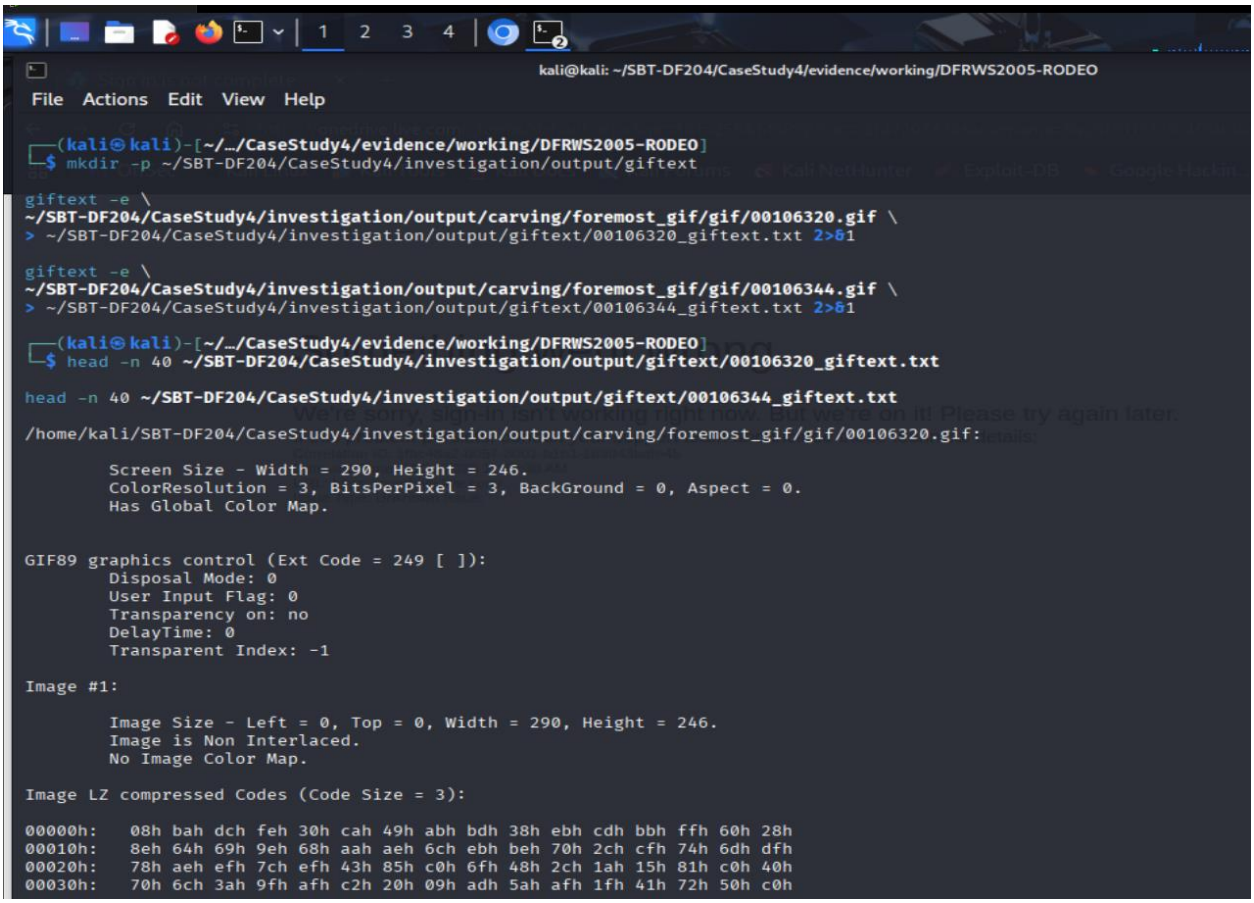
- 00106320.gif — GIF, 290 × 246, 11,407 bytes.
- 00106344.gif — GIF, 150 × 87, 4,105 bytes.

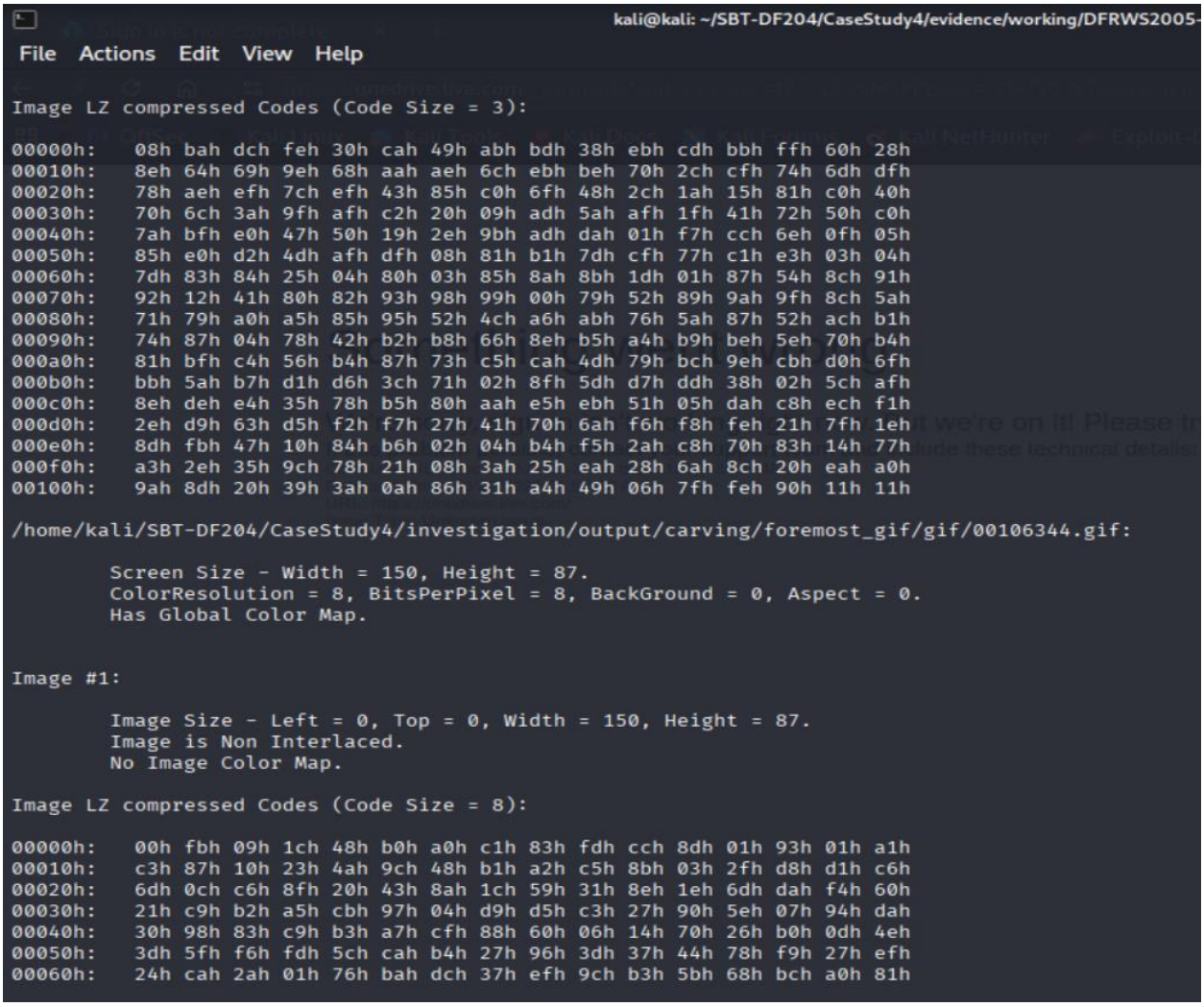
Structural Examination of Recovered GIF Files

The recovered GIF files were examined using the giftext utility, which parsed their headers, logical screen descriptors, image descriptors, and encoded image-data sections. For 00106320.gif, the utility reported a screen size of 290 × 246 pixels, a global colour map, and an image descriptor covering the full screen. It also identified a graphics control extension and reported LZW-compressed image data with a code size of 3 bits.

For 00106344.gif, giftext reported a screen size of 150 × 87 pixels, a global colour map, and an image descriptor matching the full screen dimensions. The utility identified LZW-compressed image data with a code size of 8 bits. These observations are consistent with

GIF-formatted structures and complement the earlier results from Foremost and ImageMagick.




kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-
File Actions Edit View Help
Image LZ compressed Codes (Code Size = 3):
00000h: 08h bah dch feh 30h cah 49h abh bdh 38h ebh cdh bbh ffh 60h 28h
00010h: 8eh 64h 69h 9eh 68h aah aeh 6ch ebh beh 70h 2ch cfh 74h 6dh dfh
00020h: 78h aeh efh 7ch efh 43h 85h c0h 6fh 48h 2ch 1ah 15h 81h c0h 40h
00030h: 70h 6ch 3ah 9fh afh c2h 20h 09h adh 5ah afh 1fh 41h 72h 50h c0h
00040h: 7ah bfh e0h 47h 50h 19h 2eh 9bh adh dah 01h f7h cch 6eh 0fh 05h
00050h: 85h e0h d2h 4dh afh dfh 08h 81h b1h 7dh cfh 77h c1h e3h 03h 04h
00060h: 7dh 83h 84h 25h 04h 80h 03h 85h 8ah 8bh 1dh 01h 87h 54h 8ch 91h
00070h: 92h 12h 41h 80h 82h 93h 98h 99h 00h 79h 52h 89h 9ah 9fh 8ch 5ah
00080h: 71h 79h a0h a5h 85h 95h 52h 4ch a6h abh 76h 5ah 87h 52h ach b1h
00090h: 74h 87h 04h 78h 42h b2h b8h 66h 8eh b5h a4h b9h beh 5eh 70h b4h
000a0h: 81h bfh c4h 56h b4h 87h 73h c5h cah 4dh 79h bch 9eh cbh d0h 6fh
000b0h: bbh 5ah b7h d1h d6h 3ch 71h 02h 8fh 5dh d7h ddh 38h 02h 5ch afh
000c0h: 8eh deh e4h 35h 78h b5h 80h aah e5h ebh 51h 05h dah c8h ech f1h
000d0h: 2eh d9h 63h d5h f2h f7h 27h 41h 70h 6ah f6h f8h feh 21h 7fh 1eh
000e0h: 8dh fbh 47h 10h 84h b6h 02h 04h b4h f5h 2ah c8h 70h 83h 14h 77h
000f0h: a3h 2eh 35h 9ch 78h 21h 08h 3ah 25h eah 28h 6ah 8ch 20h eah a0h
00100h: 9ah 8dh 20h 39h 3ah 0ah 86h 31h a4h 49h 06h 7fh feh 90h 11h 11h
/home/kali/SBT-DF204/CaseStudy4/investigation/output/carving/foremost_gif/gif/00106344.gif:
Screen Size - Width = 150, Height = 87.
ColorResolution = 8, BitsPerPixel = 8, BackGround = 0, Aspect = 0.
Has Global Color Map.
Image #1:
Image Size - Left = 0, Top = 0, Width = 150, Height = 87.
Image is Non Interlaced.
No Image Color Map.
Image LZ compressed Codes (Code Size = 8):
00000h: 00h fbh 09h 1ch 48h b0h a0h c1h 83h fdh cch 8dh 01h 93h 01h a1h
00010h: c3h 87h 10h 23h 4ah 9ch 48h b1h a2h c5h 8bh 03h 2fh d8h d1h c6h
00020h: 6dh 0ch c6h 8fh 20h 43h 8ah 1ch 59h 31h 8eh 1eh 6dh dah f4h 60h
00030h: 21h c9h b2h a5h cbh 97h 04h d9h d5h c3h 27h 90h 5eh 07h 94h dah
00040h: 30h 98h 83h c9h b3h a7h cfh 88h 60h 06h 14h 70h 26h b0h 0dh 4eh
00050h: 3dh 5fh f6h fdh 5ch cah b4h 27h 96h 3dh 37h 44h 78h f9h 27h efh
00060h: 24h cah 2ah 01h 76h bah dch 37h efh 9ch b3h 5bh 68h bch a0h 81h

```
/home/kali/SBT-DF204/CaseStudy4/investigation/output/carving/foremost_gif/gif/00106344.gif:
        Screen Size - Width = 150, Height = 87.
        ColorResolution = 8, BitsPerPixel = 8, BackGround = 0, Aspect = 0.
        Has Global Color Map.


Image #1:

        Image Size - Left = 0, Top = 0, Width = 150, Height = 87.
        Image is Non Interlaced.
        No Image Color Map.

Image LZ compressed Codes (Code Size = 8):

00000h:   00h fbh 09h 1ch 48h b0h a0h c1h 83h fdh cch 8dh 01h 93h 01h a1h
00010h:   c3h 87h 10h 23h 4ah 9ch 48h b1h a2h c5h 8bh 03h 2fh d8h d1h c6h
00020h:   6dh 0ch c6h 8fh 20h 43h 8ah 1ch 59h 31h 8eh 1eh 6dh dah f4h 60h
00030h:   21h c9h b2h a5h cbh 97h 04h d9h d5h c3h 27h 90h 5eh 07h 94h dah
00040h:   30h 98h 83h c9h b3h a7h cfh 88h 60h 06h 14h 70h 26h b0h 0dh 4eh
00050h:   3dh 5fh f6h fdh 5ch cah b4h 27h 96h 3dh 37h 44h 78h f9h 27h efh
00060h:   24h cah 2ah 01h 76h bah dch 37h efh 9ch b3h 5bh 68h bch a0h 81h
00070h:   03h ach d8h b9h 7ah 4dh d3h 62h ach 37h 46h 5bh 1dh 20h 10h ach
00080h:   24h c0h 99h 26h 14h 80h 38h 5ah b0h 9ch 4bh 87h 0eh 9dh bah 75h
00090h:   ech e4h cdh a3h 57h 2fh 5fh c4h 74h 69h 4ch 38h b9h a4h aah 97h
000a0h:   e3h c7h aah a8h b0h b8h 01h 06h 98h 34h 79h feh d4h 6ah 7eh c8h
000b0h:   66h 8fh 36h 2eh 40h 80h 38h e0h 80h 52h 4fh a8h 50h 26h 14h b9h
000c0h:   9ah 34h 25h 4ah 14h 44h 8ah 14h 4dh 9ah 8dh c4h 84h 8eh 35h c1h
000d0h:   a8h cdh fbh 47h f0h 99h 99h c6h 8fh 83h 0bh 77h 8ch ebh 93h 14h
000e0h:   13h 06h b2h c4h 71h 86h ceh f0h 47h 79h cfh e0h 80h 49h 43h 07h
000f0h:   8eh 1ch 63h f4h 36h 47h fch 87h 00h e5h 8ah d0h 00h 1eh 78h 36h
00100h:   72h dah 92h abh f3h e8h d3h abh f7h 84h c8h 08h 85h 2eh c0h 82h
00110h:   e9h 58h 35h bch beh fdh 5eh c5h 9dh 3ch 00h a0h 61h 4dh d9h 67h
00120h:   d5h a0h 23h 0fh 3eh bch 15h f4h 8fh 3ah c5h ach a1h 83h 12h 97h
00130h:   e0h 12h 1ch 2eh a9h 38h d1h 85h 31h 68h 69h 87h 90h 3ch 1dh 70h
00140h:   63h 47h 0ah 21h d4h a6h 48h 15h 4dh 9ch 36h 0ah 29h ach b0h a2h
00150h:   deh 89h 27h 7ah 32h c9h 7dh 2ch deh b7h 83h 2ch a4h a8h a8h 88h
00160h:   13h 46h b0h 10h c2h 03h 31h 5ch f1h 05h 1bh c2h 78h 51h 81h 13h
00170h:   9fh b4h e8h d8h 27h 23h 64h 71h 8eh 85h 08h c1h 81h 01h 05h a7h
```

The output reviewed in this phase demonstrates that giftext could parse the displayed portions of both files. However, because only the first 40 lines of each report were examined, this phase does not independently establish that the complete files terminate correctly or that all compressed image data can be decoded without errors. The files were examined through textual structural output without displaying their visual contents. Further integrity checks are required before classifying either file as fully validated.

Integrity Validation and Hashing of Recovered GIF Files

The two GIF files recovered using Foremost were subjected to an additional validation check using ImageMagick's identify utility with the -regard-warnings option. Both files returned an exit status of 0, and their respective error-output files contained no reported warnings or errors. This supports the conclusion that ImageMagick successfully processed both files during the validation checks.

SHA-256 hashes were subsequently calculated to establish reference values for the recovered artefacts. The hash for 00106320.gif was 8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80, while the hash for 00106344.gif was 71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199. These

values can be used to identify the specific recovered files and verify their integrity during subsequent handling.

The results provide additional technical support for the recoverability and parseability of the two GIF artefacts. They do not independently establish the original files' authenticity, provenance, or relevance to the alleged activity. The hashes and validation outputs should be retained with the investigation records.



Evidence record

File	SHA-256
00106320.gif	8a67d406ed130c9b5447b03fbf9dd0ac6c384a62b5028d6267e92c4abe230a80
00106344.gif	71e9f6b94e0496cc634d5d643387c7c1405e2df9deccda1482a8c7cc5ba0b199

Identification and Verification of Network Capture Evidence

The three files named rhino.log, rhino2.log, and rhino3.log were examined using the Linux file utility. All three were identified as packet capture (PCAP) files using version 2.4 of the PCAP format, with Ethernet link-layer encapsulation and microsecond-resolution timestamps stored in little-endian byte order. This finding establishes that the files contain captured network traffic rather than conventional text-based application logs.

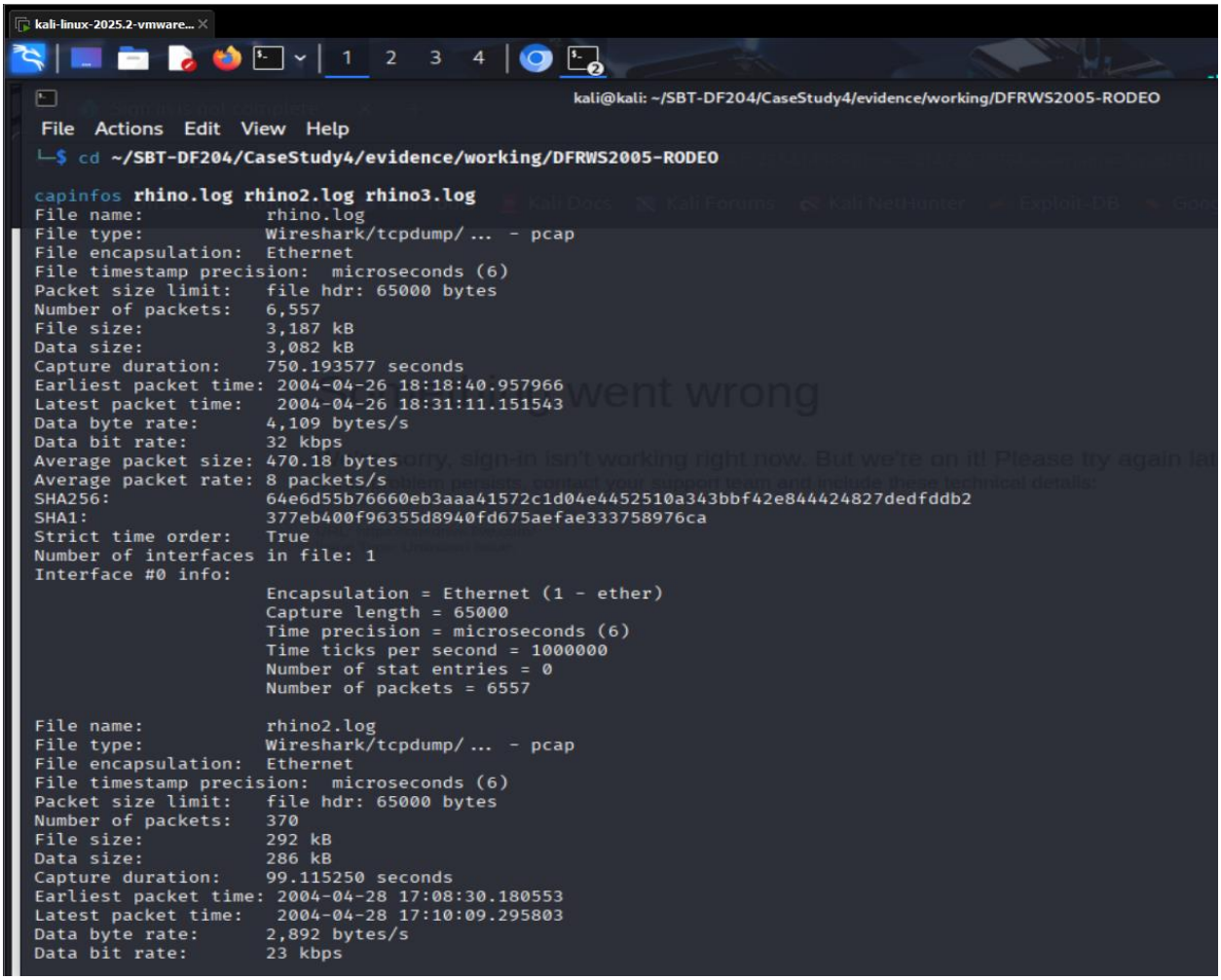
The files have sizes of 3,187,907 bytes, 292,604 bytes, and 226,094 bytes, respectively, giving a combined size of 3,706,605 bytes. SHA-256 hashes were calculated and compared with the values previously recorded for the extracted evidence files; the values were consistent. These captures will be examined to identify relevant network endpoints, communication protocols, connection sequences, timestamps, and any file-transfer or

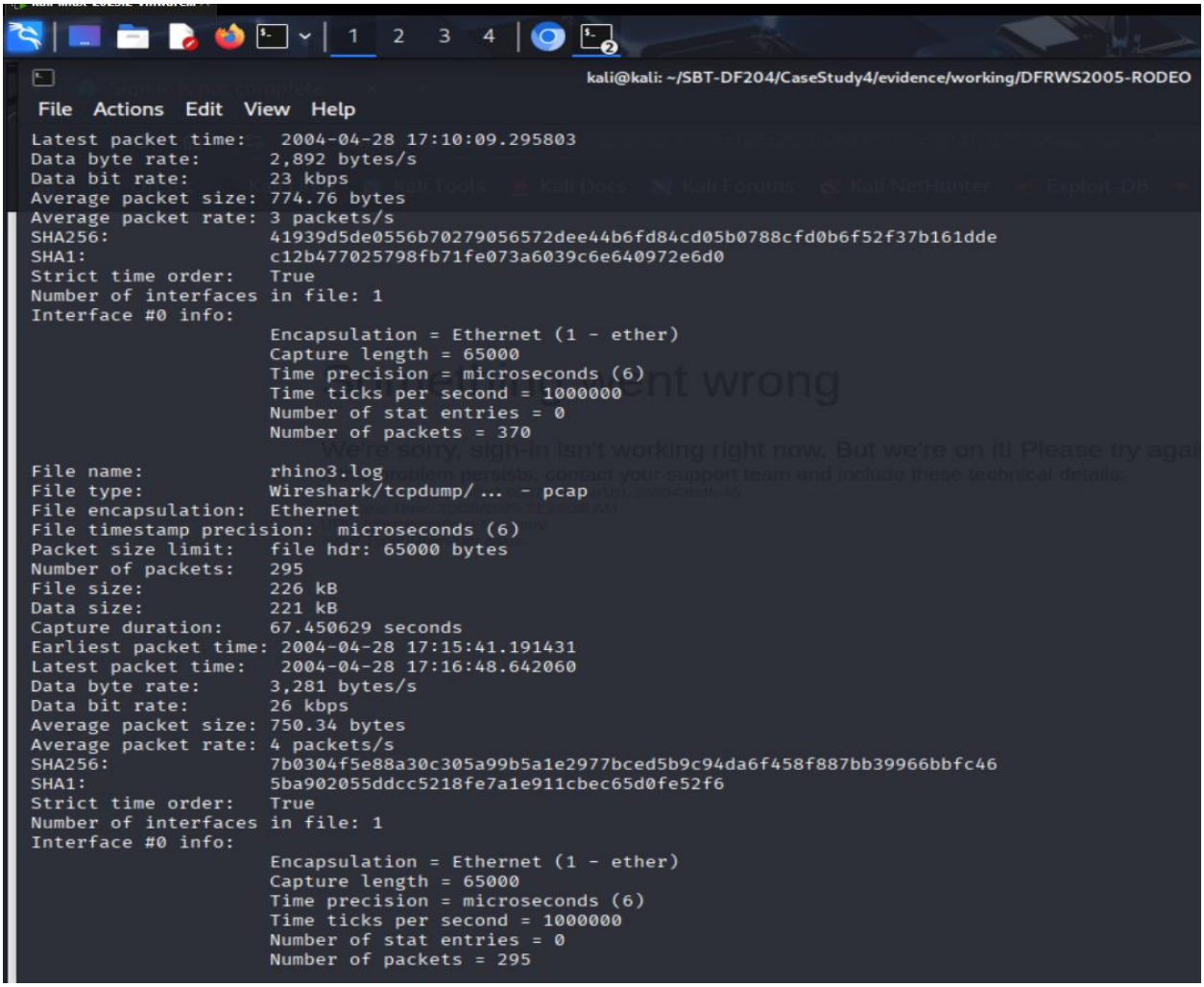
other network activity pertinent to the investigation. No conclusions about the purpose or significance of the captured traffic are drawn solely from file identification and size.

Evidence file	File type	Size	Packets/lines*
rhino.log	PCAP v2.4, Ethernet	3,187,907 bytes	50,040
rhino2.log	PCAP v2.4, Ethernet	292,604 bytes	2,378
rhino3.log	PCAP v2.4, Ethernet	226,094 bytes	1,198
Total	3 PCAP captures	3,706,605 bytes	53,616

Network Capture Metadata and Temporal Scope

The three PCAP files were examined using capinfos to establish their packet counts, capture durations, timestamp ranges, and integrity hashes. The first capture, rhino.log, contains 6,557 packets recorded over approximately 12 minutes and 30 seconds, from 26 April 2004 at 18:18:40.957966 to 18:31:11.151543. The second capture, rhino2.log, contains 370 packets over approximately 99 seconds, from 28 April 2004 at 17:08:30.180553 to 17:10:09.295803. The third capture, rhino3.log, contains 295 packets over approximately 67 seconds, from 28 April 2004 at 17:15:41.191431 to 17:16:48.642060.


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
└─$ cd ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
capinfos rhino.log rhino2.log rhino3.log
File name: rhino.log
File type: Wireshark/tcpdump/ ... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65000 bytes
Number of packets: 6,557
File size: 3,187 kB
Data size: 3,082 kB
Capture duration: 750.193577 seconds
Earliest packet time: 2004-04-26 18:18:40.957966
Latest packet time: 2004-04-26 18:31:11.151543
Data byte rate: 4,109 bytes/s
Data bit rate: 32 kbps
Average packet size: 470.18 bytes
Average packet rate: 8 packets/s
SHA256: 64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
SHA1: 377eb400f96355d8940fd675aefae333758976ca
Strict time order: True
Number of interfaces in file: 1
Interface #0 info:
Encapsulation = Ethernet (1 - ether)
Capture length = 65000
Time precision = microseconds (6)
Time ticks per second = 1000000
Number of stat entries = 0
Number of packets = 6557
File name: rhino2.log
File type: Wireshark/tcpdump/ ... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65000 bytes
Number of packets: 370
File size: 292 kB
Data size: 286 kB
Capture duration: 99.115250 seconds
Earliest packet time: 2004-04-28 17:08:30.180553
Latest packet time: 2004-04-28 17:10:09.295803
Data byte rate: 2,892 bytes/s
Data bit rate: 23 kbps



Collectively, the files contain **7,222 packets** across three separate capture sessions on two calendar dates. Each capture uses Ethernet encapsulation and microsecond-resolution timestamps. capinfos reported that packet timestamps are in strict chronological order within each individual file. The SHA-256 values also matched the previously recorded hashes, supporting the integrity of the working copies during this examination.

The metadata establishes the temporal scope and volume of the captured traffic but does not, by itself, establish what the traffic represents or whether it is related to the USB image contents. Further protocol, endpoint, and conversation analysis is required to identify potentially relevant communications and correlate them with other evidence. The timestamps are reported as displayed by the analysis tools; their time-zone interpretation and relationship to the original system clock have not yet been independently established.

Table 1: Core findings - Network capture analysis

Evidence file	Packets	Capture date	Start time	End time	Duration
rhino.log	6,557	26 Apr 2004	18:18:40.957966	18:31:11.151543	12 min 30.194 sec
rhino2.log	370	28 Apr 2004	17:08:30.180553	17:10:09.295803	1 min 39.115 sec
rhino3.log	295	28 Apr 2004	17:15:41.191431	17:16:48.642060	1 min 7.451 sec
Total	7,222	26–28 Apr 2004	—	—	Three separate capture periods

Note: The total packet count is the sum of the three captures. The durations are not added because the captures represent separate time periods.

Table 2: Core technical findings

Area examined	Finding	Forensic significance
File identification	All three files are PCAP v2.4 packet captures	Establishes that the files contain captured network traffic, not ordinary text logs
Network encapsulation	Ethernet	Provides the link-layer context for examining captured communications
Timestamp precision	Microseconds	Supports fine-grained event sequencing within each capture
Packet count	7,222 packets across three files	Defines the initial scope of network traffic available for analysis
Capture chronology	One capture on 26 April and two on 28 April 2004	Provides the initial time framework for correlation with other evidence
Timestamp ordering	All three captures report strict time order	Supports chronological analysis within each individual capture

File integrity	Recorded SHA-256 hashes match the previously recorded hashes	Supports consistency of the examined files with the earlier extracted copies
Network activity and endpoints	Not yet analysed	Requires protocol, source/destination address, connection and application-level examination

Table 3: Evidence integrity records

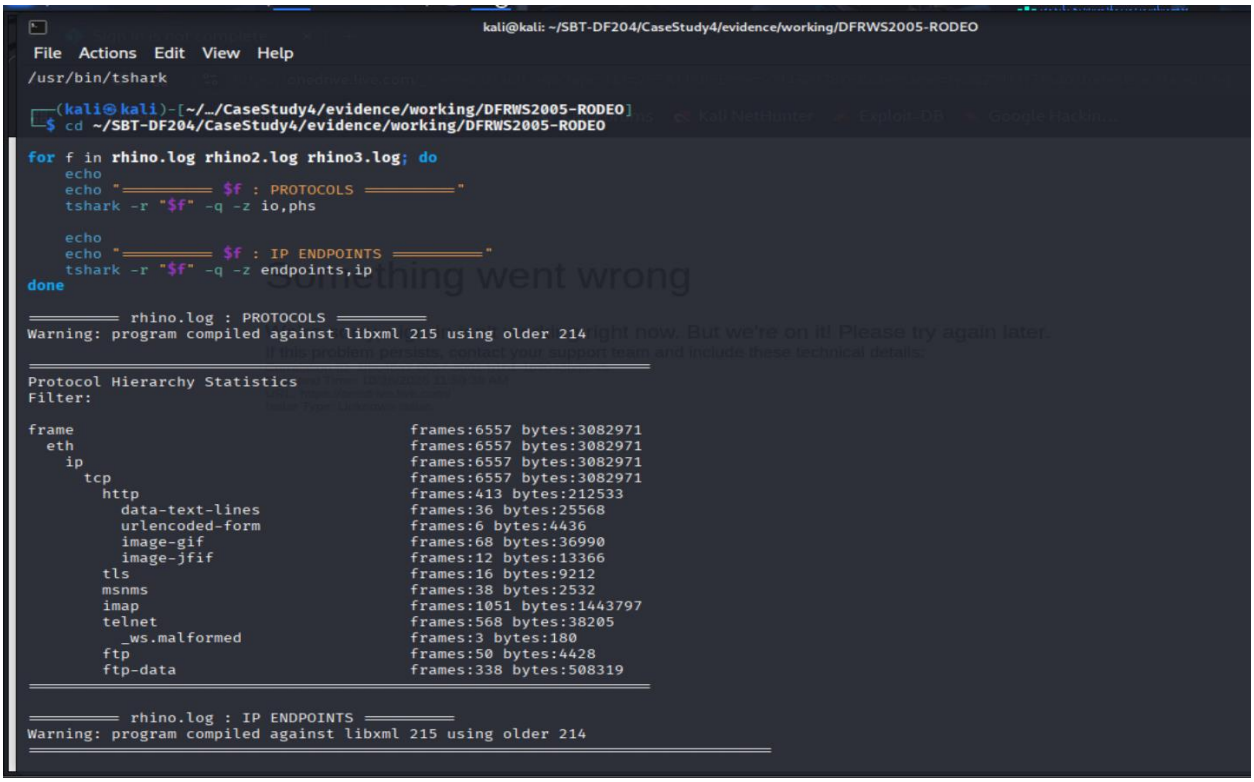
Evidence file	SHA-256
rhino.log	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
rhino2.log	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde
rhino3.log	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46

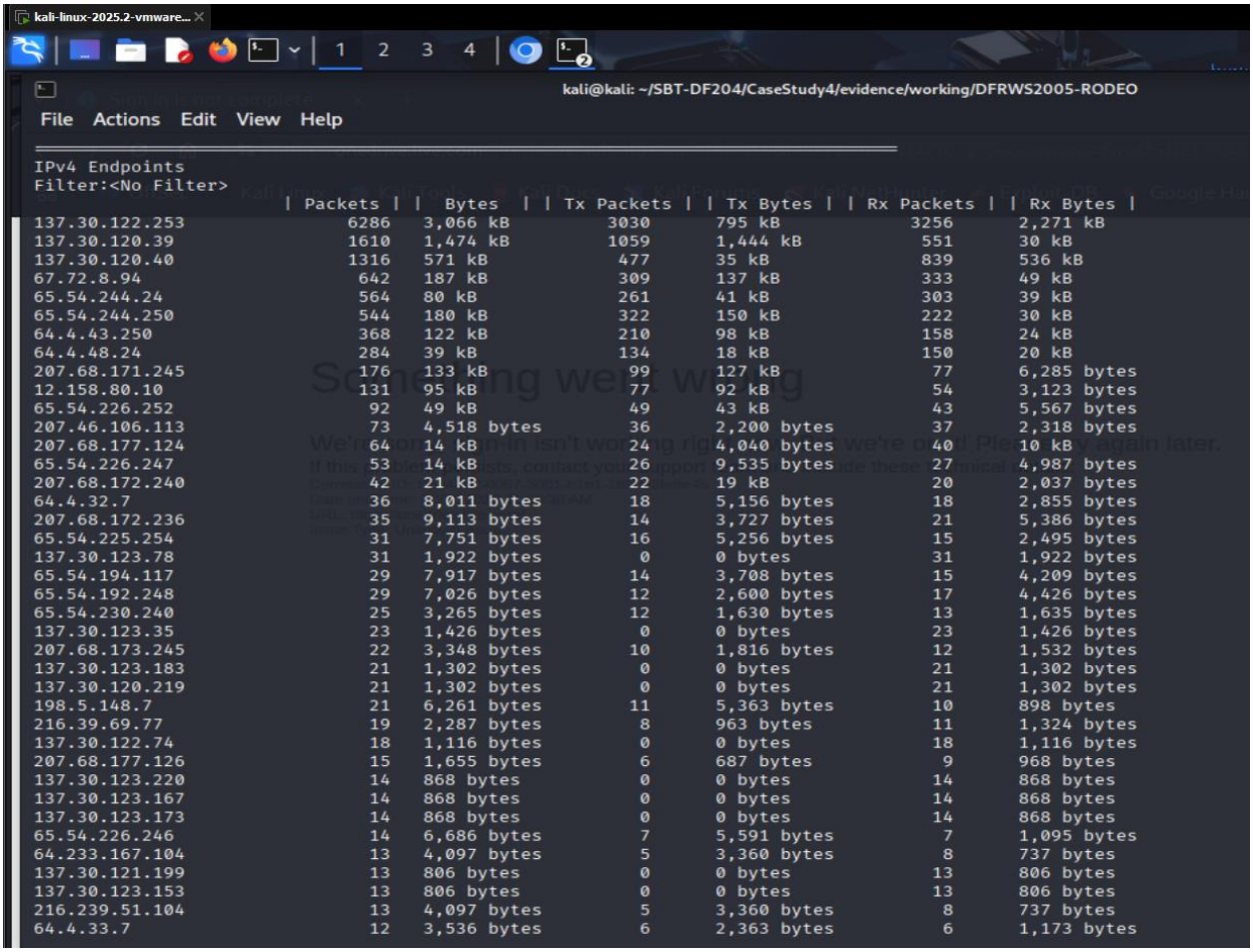
Core Findings from Preliminary Network Analysis

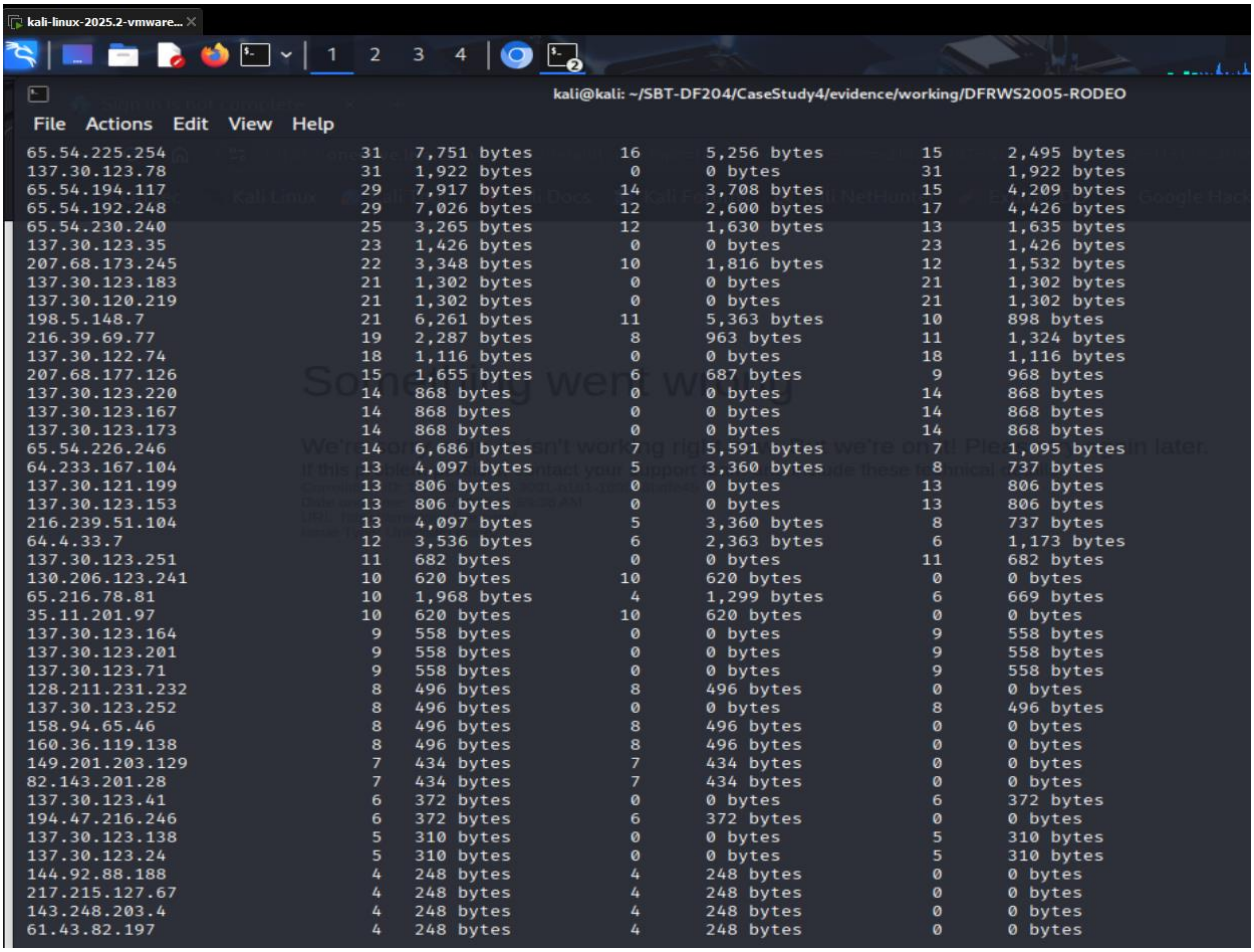
Protocol hierarchy analysis of the three PCAP files identified several application protocols. The largest capture, rhino.log, contains HTTP, IMAP, Telnet, FTP, FTP-DATA, MSN Messenger and TLS traffic. The two later captures, rhino2.log and rhino3.log, predominantly show HTTP and IMAP, with MSN Messenger also identified in rhino3.log. The HTTP dissector identified frames containing GIF image data in all three captures and JPEG/JFIF data in the first two captures.

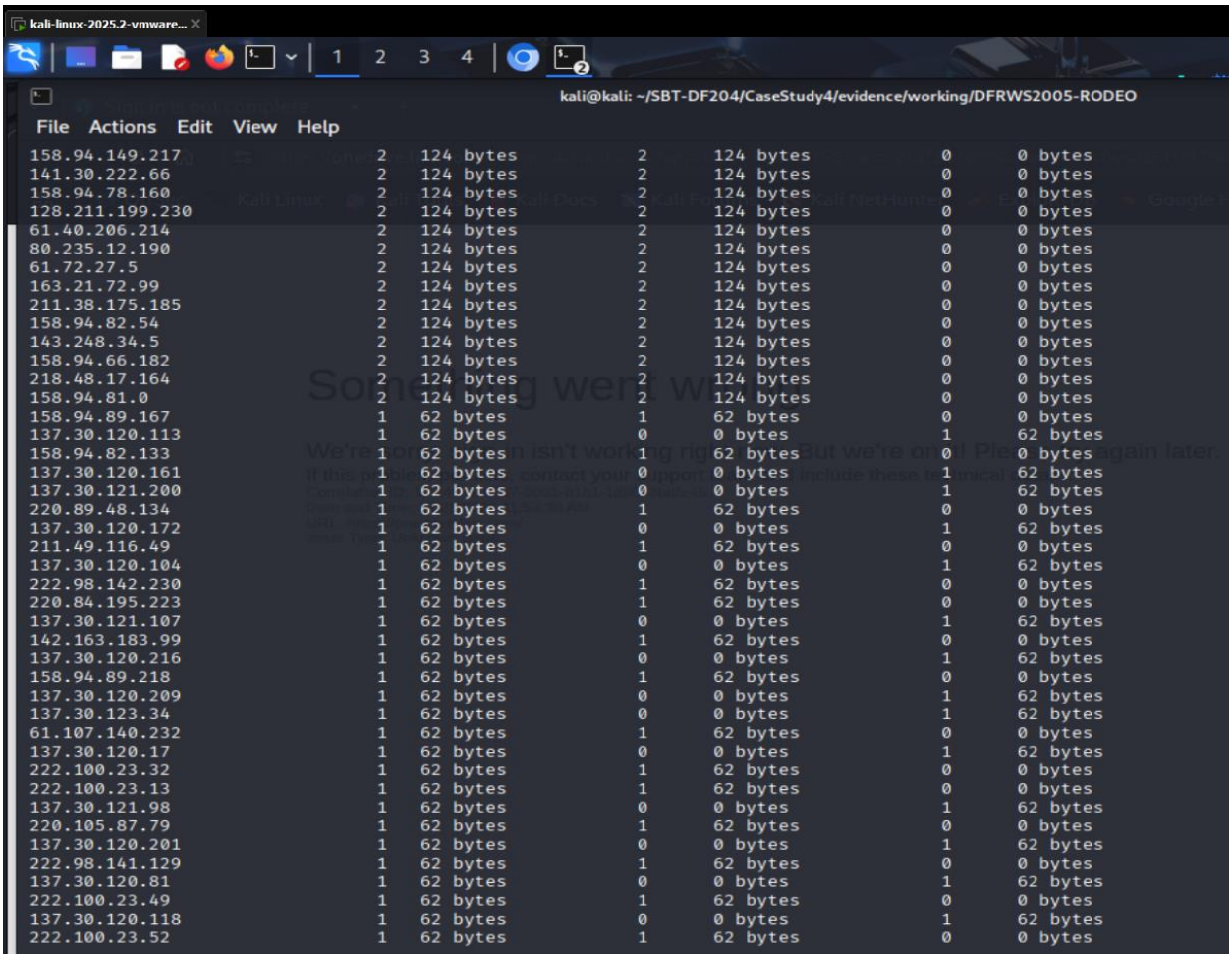
The endpoint summaries identify repeated communications involving IP addresses 137.30.120.39 and 137.30.120.37 across the available captures. The endpoint 137.30.123.234 accounts for all 370 packets in the rhino2.log endpoint summary and all 295 packets in the rhino3.log endpoint summary. These observations provide useful leads for further examination, but the endpoint tables alone do not establish the identity of the systems or users involved, the contents of transferred files, or the purpose of the communications.

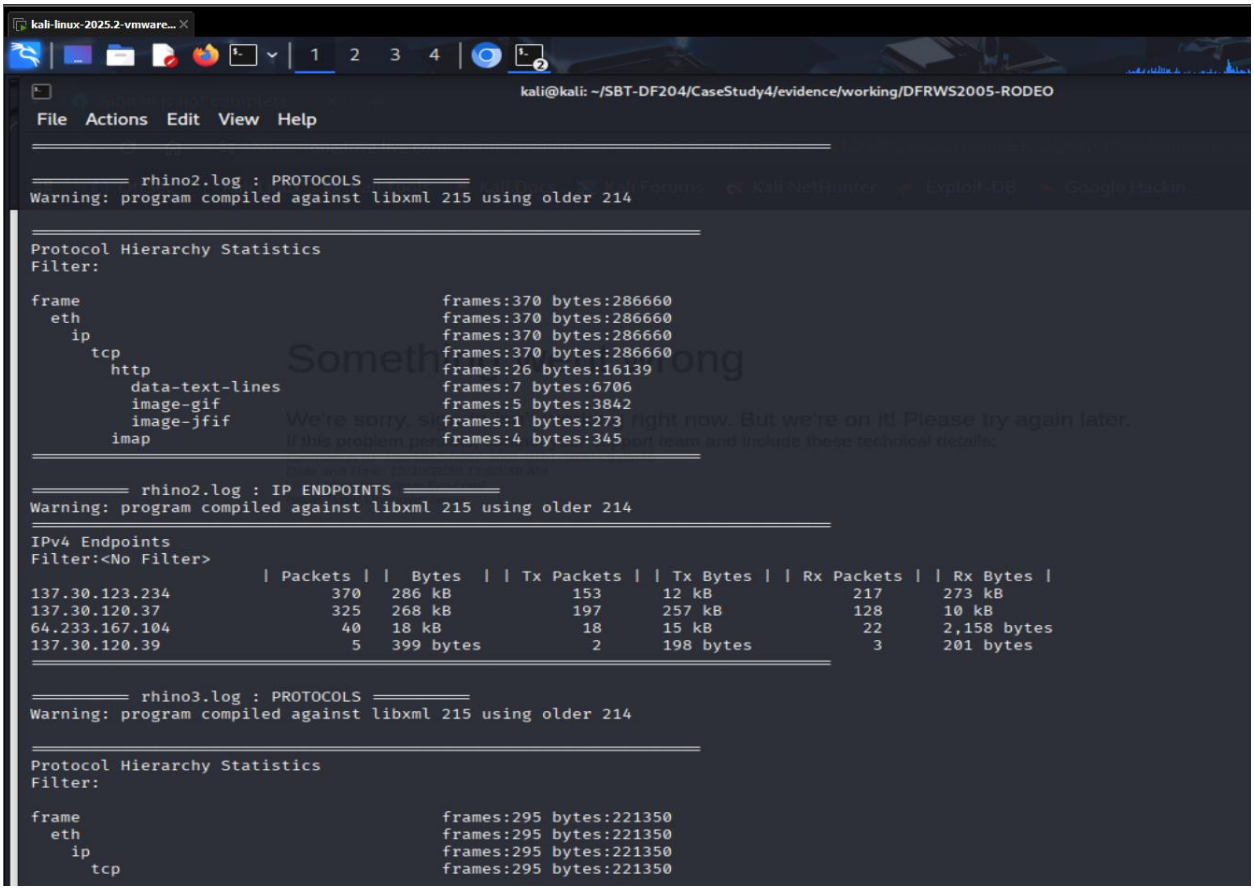
Further analysis will focus on TCP conversations, HTTP request and response metadata, FTP commands and data transfers, and relevant timestamps. Any proposed relationship between the network captures and the USB image will be treated as a hypothesis until supported by specific, corroborating artefacts.

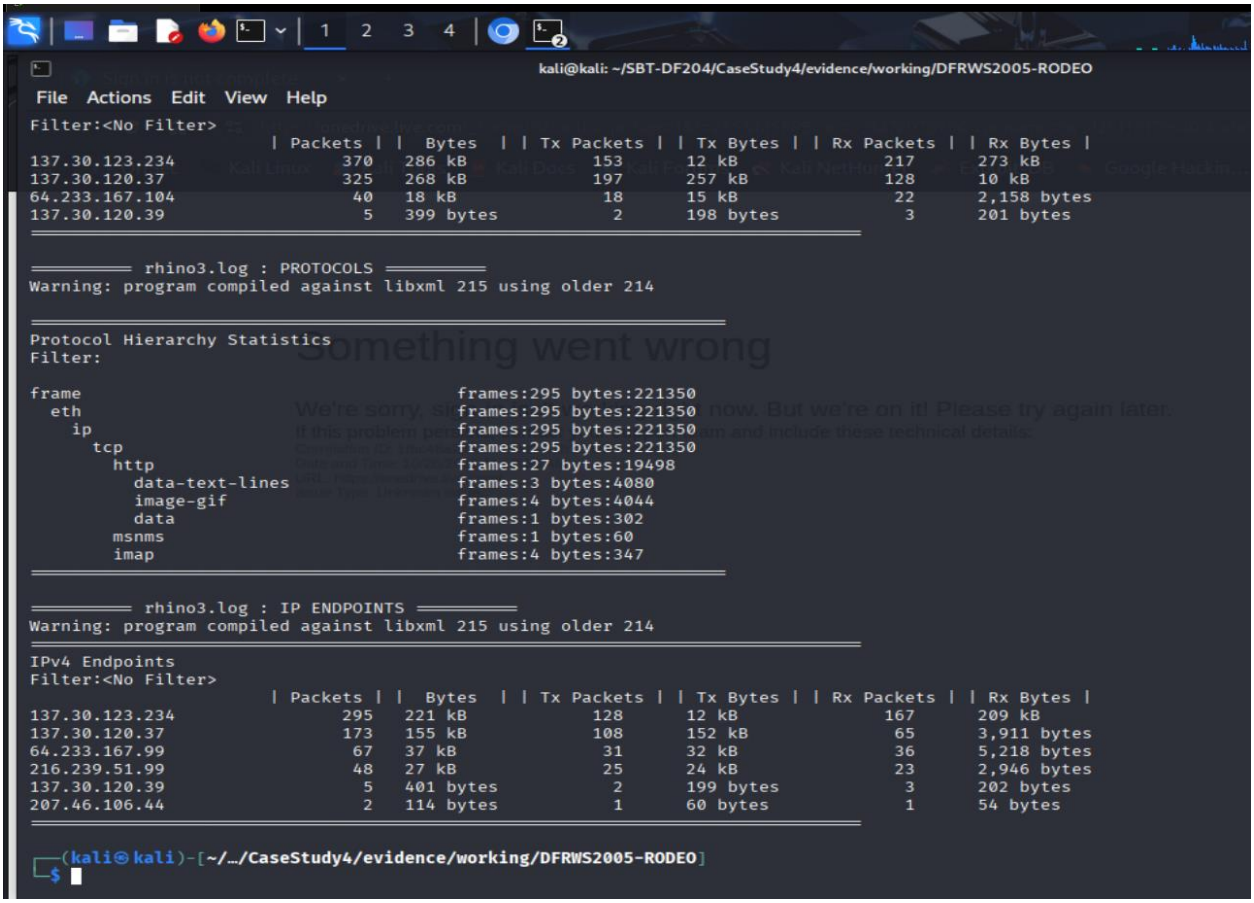



kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
IPv4 Endpoints
Filter:<No Filter>
| Packets | | Bytes | | Tx Packets | | Tx Bytes | | Rx Packets | | Rx Bytes |
137.30.122.253 6286 3,066 kB 3030 795 kB 3256 2,271 kB
137.30.120.39 1610 1,474 kB 1059 1,444 kB 551 30 kB
137.30.120.40 1316 571 kB 477 35 kB 839 536 kB
67.72.8.94 642 187 kB 309 137 kB 333 49 kB
65.54.244.24 564 80 kB 261 41 kB 303 39 kB
65.54.244.250 544 180 kB 322 150 kB 222 30 kB
64.4.43.250 368 122 kB 210 98 kB 158 24 kB
64.4.48.24 284 39 kB 134 18 kB 150 20 kB
207.68.171.245 176 133 kB 99 127 kB 77 6,285 bytes
12.158.80.10 131 95 kB 77 92 kB 54 3,123 bytes
65.54.226.252 92 49 kB 49 43 kB 43 5,567 bytes
207.46.106.113 73 4,518 bytes 36 2,200 bytes 37 2,318 bytes
207.68.177.124 64 14 kB 24 4,040 bytes 40 10 kB
65.54.226.247 53 14 kB 26 9,535 bytes 27 4,987 bytes
207.68.172.240 42 21 kB 22 19 kB 20 2,037 bytes
64.4.32.7 36 8,011 bytes 18 5,156 bytes 18 2,855 bytes
207.68.172.236 35 9,113 bytes 14 3,727 bytes 21 5,386 bytes
65.54.225.254 31 7,751 bytes 16 5,256 bytes 15 2,495 bytes
137.30.123.78 31 1,922 bytes 0 0 bytes 31 1,922 bytes
65.54.194.117 29 7,917 bytes 14 3,708 bytes 15 4,209 bytes
65.54.192.248 29 7,026 bytes 12 2,600 bytes 17 4,426 bytes
65.54.230.240 25 3,265 bytes 12 1,630 bytes 13 1,635 bytes
137.30.123.35 23 1,426 bytes 0 0 bytes 23 1,426 bytes
207.68.173.245 22 3,348 bytes 10 1,816 bytes 12 1,532 bytes
137.30.123.183 21 1,302 bytes 0 0 bytes 21 1,302 bytes
137.30.120.219 21 1,302 bytes 0 0 bytes 21 1,302 bytes
198.5.148.7 21 6,261 bytes 11 5,363 bytes 10 898 bytes
216.39.69.77 19 2,287 bytes 8 963 bytes 11 1,324 bytes
137.30.122.74 18 1,116 bytes 0 0 bytes 18 1,116 bytes
207.68.177.126 15 1,655 bytes 6 687 bytes 9 968 bytes
137.30.123.220 14 868 bytes 0 0 bytes 14 868 bytes
137.30.123.167 14 868 bytes 0 0 bytes 14 868 bytes
137.30.123.173 14 868 bytes 0 0 bytes 14 868 bytes
65.54.226.246 14 6,686 bytes 7 5,591 bytes 7 1,095 bytes
64.233.167.104 13 4,097 bytes 5 3,360 bytes 8 737 bytes
137.30.121.199 13 806 bytes 0 0 bytes 13 806 bytes
137.30.123.153 13 806 bytes 0 0 bytes 13 806 bytes
216.239.51.104 13 4,097 bytes 5 3,360 bytes 8 737 bytes
64.4.33.7 12 3,536 bytes 6 2,363 bytes 6 1,173 bytes


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
65.54.225.254 31 7,751 bytes 16 5,256 bytes 15 2,495 bytes
137.30.123.78 31 1,922 bytes 0 0 bytes 31 1,922 bytes
65.54.194.117 29 7,917 bytes 14 3,708 bytes 15 4,209 bytes
65.54.192.248 29 7,026 bytes 12 2,600 bytes 17 4,426 bytes
65.54.230.240 25 3,265 bytes 12 1,630 bytes 13 1,635 bytes
137.30.123.35 23 1,426 bytes 0 0 bytes 23 1,426 bytes
207.68.173.245 22 3,348 bytes 10 1,816 bytes 12 1,532 bytes
137.30.123.183 21 1,302 bytes 0 0 bytes 21 1,302 bytes
137.30.120.219 21 1,302 bytes 0 0 bytes 21 1,302 bytes
198.5.148.7 21 6,261 bytes 11 5,363 bytes 10 898 bytes
216.39.69.77 19 2,287 bytes 8 963 bytes 11 1,324 bytes
137.30.122.74 18 1,116 bytes 0 0 bytes 18 1,116 bytes
207.68.177.126 15 1,655 bytes 6 687 bytes 9 968 bytes
137.30.123.220 14 868 bytes 0 0 bytes 14 868 bytes
137.30.123.167 14 868 bytes 0 0 bytes 14 868 bytes
137.30.123.173 14 868 bytes 0 0 bytes 14 868 bytes
65.54.226.246 14 6,686 bytes 7 5,591 bytes 7 1,095 bytes
64.233.167.104 13 4,097 bytes 5 3,360 bytes 8 737 bytes
137.30.121.199 13 806 bytes 0 0 bytes 13 806 bytes
137.30.123.153 13 806 bytes 0 0 bytes 13 806 bytes
216.239.51.104 13 4,097 bytes 5 3,360 bytes 8 737 bytes
64.4.33.7 12 3,536 bytes 6 2,363 bytes 6 1,173 bytes
137.30.123.251 11 682 bytes 0 0 bytes 11 682 bytes
130.206.123.241 10 620 bytes 10 620 bytes 0 0 bytes
65.216.78.81 10 1,968 bytes 4 1,299 bytes 6 669 bytes
35.11.201.97 10 620 bytes 10 620 bytes 0 0 bytes
137.30.123.164 9 558 bytes 0 0 bytes 9 558 bytes
137.30.123.201 9 558 bytes 0 0 bytes 9 558 bytes
137.30.123.71 9 558 bytes 0 0 bytes 9 558 bytes
128.211.231.232 8 496 bytes 8 496 bytes 0 0 bytes
137.30.123.252 8 496 bytes 0 0 bytes 8 496 bytes
158.94.65.46 8 496 bytes 8 496 bytes 0 0 bytes
160.36.119.138 8 496 bytes 8 496 bytes 0 0 bytes
149.201.203.129 7 434 bytes 7 434 bytes 0 0 bytes
82.143.201.28 7 434 bytes 7 434 bytes 0 0 bytes
137.30.123.41 6 372 bytes 0 0 bytes 6 372 bytes
194.47.216.246 6 372 bytes 6 372 bytes 0 0 bytes
137.30.123.138 5 310 bytes 0 0 bytes 5 310 bytes
137.30.123.24 5 310 bytes 0 0 bytes 5 310 bytes
144.92.88.188 4 248 bytes 4 248 bytes 0 0 bytes
217.215.127.67 4 248 bytes 4 248 bytes 0 0 bytes
143.248.203.4 4 248 bytes 4 248 bytes 0 0 bytes
61.43.82.197 4 248 bytes 4 248 bytes 0 0 bytes


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
158.94.149.217 2 124 bytes 2 124 bytes 0 0 bytes
141.30.222.66 2 124 bytes 2 124 bytes 0 0 bytes
158.94.78.160 2 124 bytes 2 124 bytes 0 0 bytes
128.211.199.230 2 124 bytes 2 124 bytes 0 0 bytes
61.40.206.214 2 124 bytes 2 124 bytes 0 0 bytes
80.235.12.190 2 124 bytes 2 124 bytes 0 0 bytes
61.72.27.5 2 124 bytes 2 124 bytes 0 0 bytes
163.21.72.99 2 124 bytes 2 124 bytes 0 0 bytes
211.38.175.185 2 124 bytes 2 124 bytes 0 0 bytes
158.94.82.54 2 124 bytes 2 124 bytes 0 0 bytes
143.248.34.5 2 124 bytes 2 124 bytes 0 0 bytes
158.94.66.182 2 124 bytes 2 124 bytes 0 0 bytes
218.48.17.164 2 124 bytes 2 124 bytes 0 0 bytes
158.94.81.0 2 124 bytes 2 124 bytes 0 0 bytes
158.94.89.167 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.113 1 62 bytes 0 0 bytes 1 62 bytes
158.94.82.133 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.161 1 62 bytes 0 0 bytes 1 62 bytes
137.30.121.200 1 62 bytes 0 0 bytes 1 62 bytes
220.89.48.134 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.172 1 62 bytes 0 0 bytes 1 62 bytes
211.49.116.49 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.104 1 62 bytes 0 0 bytes 1 62 bytes
222.98.142.230 1 62 bytes 1 62 bytes 0 0 bytes
220.84.195.223 1 62 bytes 1 62 bytes 0 0 bytes
137.30.121.107 1 62 bytes 0 0 bytes 1 62 bytes
142.163.183.99 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.216 1 62 bytes 0 0 bytes 1 62 bytes
158.94.89.218 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.209 1 62 bytes 0 0 bytes 1 62 bytes
137.30.123.34 1 62 bytes 0 0 bytes 1 62 bytes
61.107.140.232 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.17 1 62 bytes 0 0 bytes 1 62 bytes
222.100.23.32 1 62 bytes 1 62 bytes 0 0 bytes
222.100.23.13 1 62 bytes 1 62 bytes 0 0 bytes
137.30.121.98 1 62 bytes 0 0 bytes 1 62 bytes
220.105.87.79 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.201 1 62 bytes 0 0 bytes 1 62 bytes
222.98.141.129 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.81 1 62 bytes 0 0 bytes 1 62 bytes
222.100.23.49 1 62 bytes 1 62 bytes 0 0 bytes
137.30.120.118 1 62 bytes 0 0 bytes 1 62 bytes
222.100.23.52 1 62 bytes 1 62 bytes 0 0 bytes


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
========== rhino2.log : PROTOCOLS ==========
Warning: program compiled against libxml 215 using older 214
Protocol Hierarchy Statistics
Filter:
frame frames:370 bytes:286660
eth frames:370 bytes:286660
ip frames:370 bytes:286660
tcp frames:370 bytes:286660
http frames:26 bytes:16139
data-text-lines frames:7 bytes:6706
image-gif frames:5 bytes:3842
image-jfif frames:1 bytes:273
imap frames:4 bytes:345
========== rhino2.log : IP ENDPOINTS ==========
Warning: program compiled against libxml 215 using older 214
IPv4 Endpoints
Filter:<No Filter>
| Packets | | Bytes | | Tx Packets | | Tx Bytes | | Rx Packets | | Rx Bytes |
137.30.123.234 370 286 kB 153 12 kB 217 273 kB
137.30.120.37 325 268 kB 197 257 kB 128 10 kB
64.233.167.104 40 18 kB 18 15 kB 22 2,158 bytes
137.30.120.39 5 399 bytes 2 198 bytes 3 201 bytes
========== rhino3.log : PROTOCOLS ==========
Warning: program compiled against libxml 215 using older 214
Protocol Hierarchy Statistics
Filter:
frame frames:295 bytes:221350
eth frames:295 bytes:221350
ip frames:295 bytes:221350
tcp frames:295 bytes:221350



The protocol and endpoint results reveal the first major network findings. We can now summarise the evidence in tables, while keeping interpretations separate from what has actually been established.

The most notable result is that rhino.log contains several types of application traffic, including HTTP, IMAP, Telnet, FTP, FTP data, MSN Messenger traffic, and a small amount of TLS. The two later captures also contain HTTP and IMAP traffic, with GIF image data identified by the protocol dissector.

Table 1: Core network findings

Capture	Packets	Main protocols identified	Key observation
rhino.log	6,557	HTTP, IMAP, Telnet, FTP, FTP-DATA, MSN Messenger, TLS	Broadest range of application protocols and the largest capture
rhino2.log	370	HTTP, IMAP	HTTP traffic includes frames dissected as GIF and JPEG image data

rhino3.log	295	HTTP, IMAP, MSN Messenger	HTTP traffic includes frames dissected as GIF image data
Total	7,222	Multiple TCP-based protocols	Network traffic is available for further application-level examination

These are protocol dissector results, not proof that every detected protocol was used for malicious activity. A packet may also be counted at multiple levels of the protocol hierarchy.

Table 2: Significant protocol counts

The figures below are taken directly from the protocol hierarchy output you supplied.

Protocol or dissected content	**rhino.log**	**rhino2.log**	**rhino3.log**
HTTP	413 frames	26 frames	27 frames
IMAP	1,051 frames	4 frames	4 frames
Telnet	568 frames	Not listed	Not listed
FTP	50 frames	Not listed	Not listed
FTP-DATA	338 frames	Not listed	Not listed
TLS	16 frames	Not listed	Not listed
MSN Messenger	38 frames	Not listed	1 frame
HTTP-dissected GIF content	68 frames	5 frames	4 frames
HTTP-dissected JPEG/JFIF content	12 frames	1 frame	Not listed

Important: “Not listed” means the protocol does not appear in the supplied hierarchy output; it does not establish that the corresponding application or behaviour was impossible outside the captured traffic.

Table 3: Key IP endpoints and repeated communications

Capture	**Endpoint IP**	**Packets involving endpoint**	**Forensic observation**
rhino.log	137.30.122.253	6,286	Highest packet count in the first capture

rhino.log	137.30.120.39	1,610	Also appears in both later captures
rhino.log	137.30.120.40	1,316	High-volume endpoint in the first capture
rhino2.log	137.30.123.234	370	Appears in every packet in this capture's endpoint summary
rhino2.log	137.30.120.37	325	Major communicating endpoint
rhino3.log	137.30.123.234	295	Appears in every packet in this capture's endpoint summary
rhino3.log	137.30.120.37	173	Repeated endpoint across the later captures
rhino3.log	64.233.167.99	67	Additional endpoint requiring connection-level examination
rhino3.log	216.239.51.99	48	Additional endpoint requiring connection-level examination

These are endpoint packet counts from the supplied output. They do not establish which device belongs to a particular person, or whether the communications were malicious. Endpoint direction and service context need to be confirmed through TCP conversations and application-level evidence.

Table 4: Preliminary assessment

Finding	Current assessment	Next verification step
Multiple application protocols in rhino.log	Established	Examine TCP conversations and relevant application streams
HTTP image content detected	Established at the protocol-dissection level	Review HTTP requests and response metadata
FTP and FTP-DATA detected in rhino.log	Established	Identify FTP commands, filenames and transfer direction

Telnet detected in rhino.log	Established	Examine session content for relevant commands, while protecting any credentials
Repeated endpoints across captures	Established	Correlate IPs, ports, timestamps and conversation details
Relationship between network traffic and recovered USB artefacts	Not yet established	Correlate timestamps, filenames and other matching indicators

HTTP Request Analysis

HTTP request metadata was extracted from the three network capture files using Wireshark's command-line utility, tshark. The extraction recorded frame numbers, timestamps, source and destination IP addresses and ports, HTTP methods, hostnames, and request URIs.

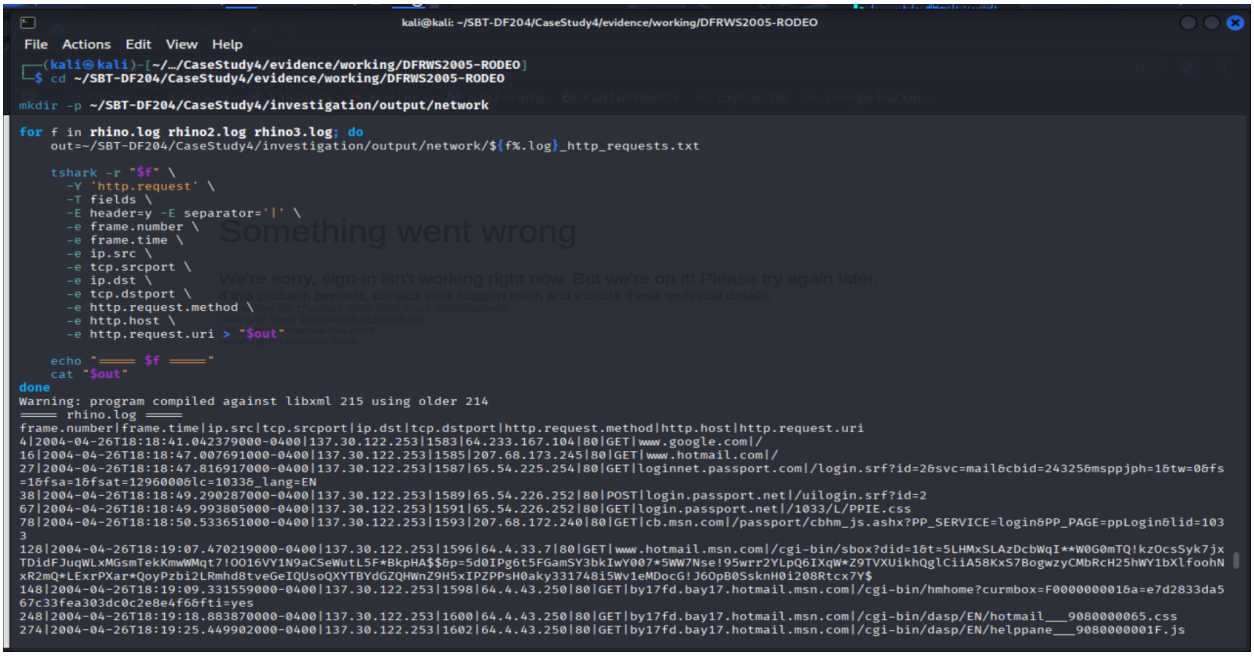
The first capture, rhino.log, contains HTTP activity dated 26 April 2004. The visible requests include access to Google and Hotmail, Microsoft Passport authentication and logout endpoints, and requests associated with Hotmail mailbox functions and web-page resources. These observations establish that web browsing and webmail-related activity occurred within the captured network traffic. However, the request metadata alone does not establish the content of messages or attachments.

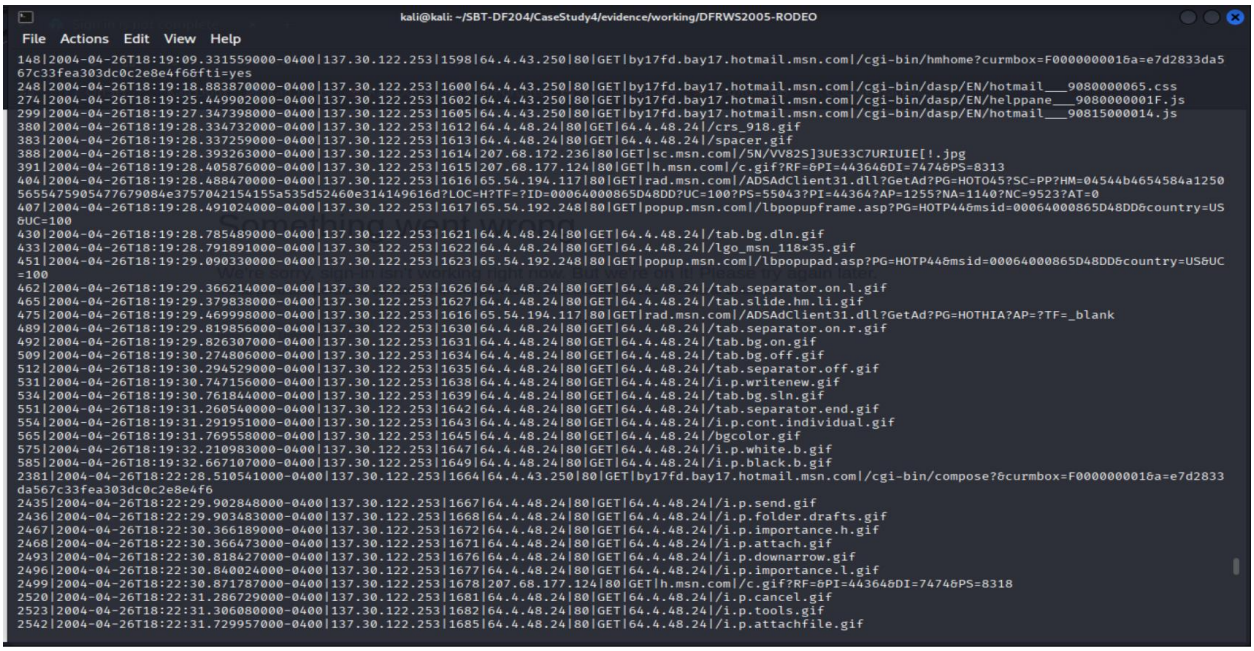
The second capture, rhino2.log, contains HTTP requests dated 28 April 2004. The recorded activity includes access to Google and pages hosted under www.cs.uno.edu, including requests for resources named rhino4.jpg and rhino5.gif. Requests were also made to user-directory paths on that server. These entries provide specific URLs and timestamps for correlation with other evidence.

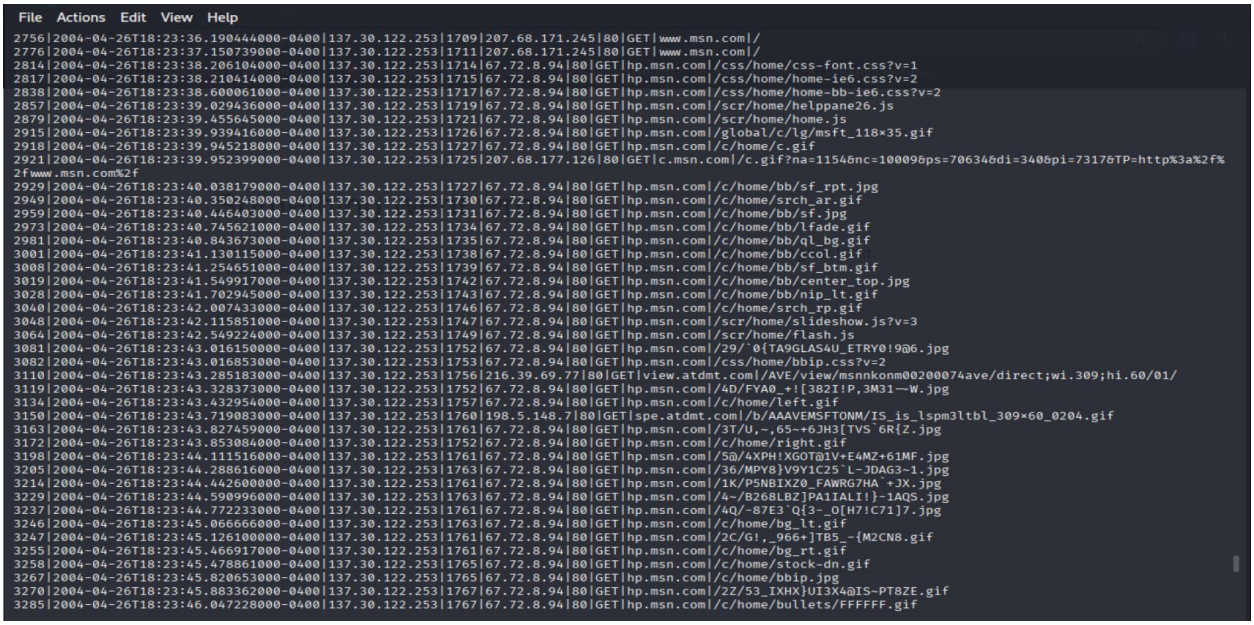
The third capture, rhino3.log, contains a particularly relevant sequence. At approximately 17:15:48 (capture timestamp offset −0400), a request was made to Google for a search containing the term rhino.exe. At approximately 17:15:55, a request was made to Google Groups using the same search term. At approximately 17:16:23, the source address 137.30.123.234 requested /~gnome/rhino.exe from www.cs.uno.edu, whose destination

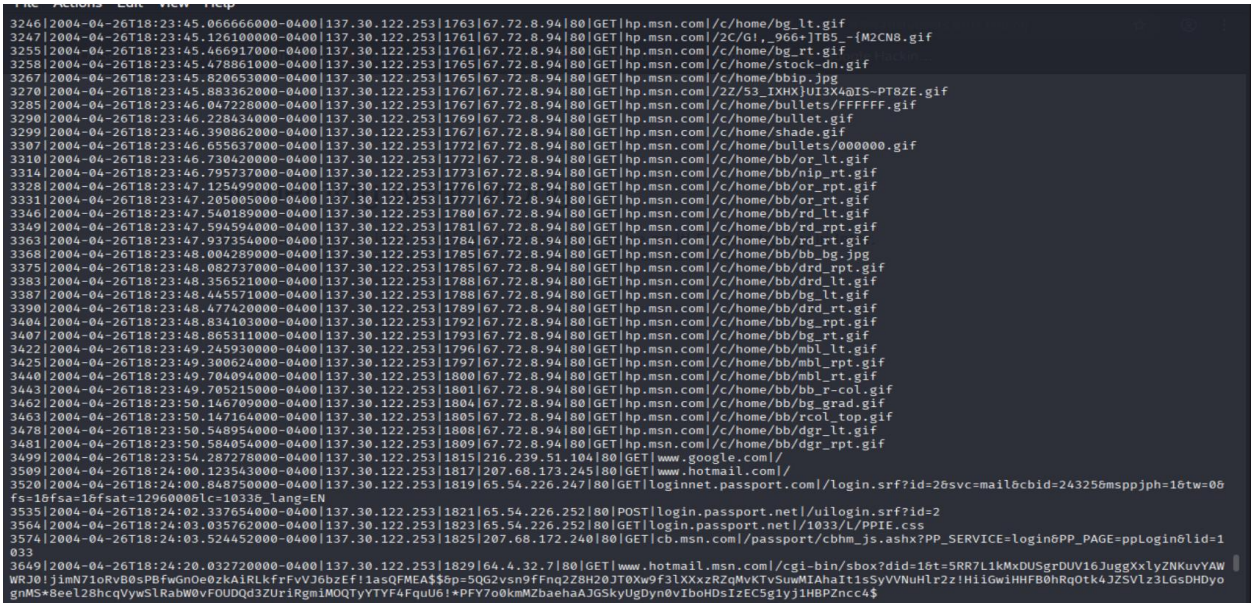
IP address in the capture was 137.30.120.37. The short interval and matching filename make this sequence a significant investigative lead. The HTTP request demonstrates that the resource was requested; confirmation that the file was successfully transferred or executed would require examination of the corresponding response and related packet data.

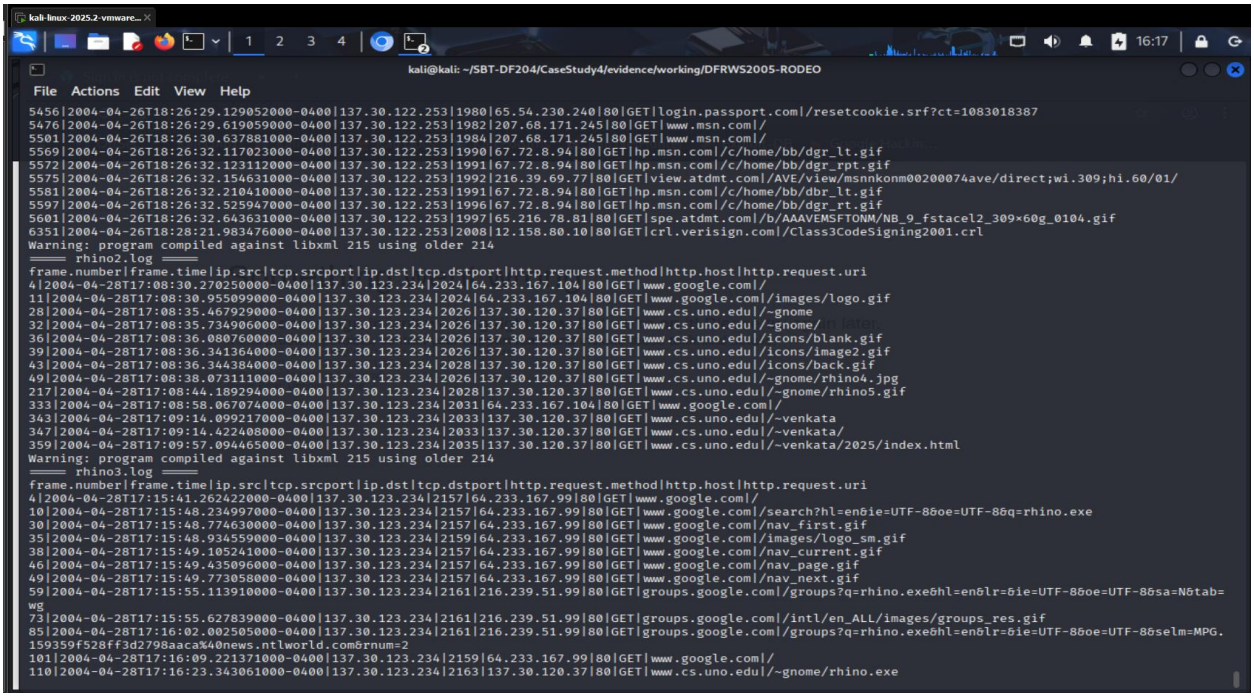
The extracted output should be retained as an investigative artefact. The timestamps, frame numbers, hostnames, and request paths can be used to locate and review the original packets. Conclusions should remain limited to the recorded network evidence and should not, by themselves, attribute the activity to a named individual.




kali@kali: ~/SBT-DF204/CaseStudy4/evidence/working/DFRWS2005-RODEO
File Actions Edit View Help
148|2004-04-26T18:19:09.331559000-0400|137.30.122.253|1598|64.4.43.250|80|GET|by17fd.bay17.hotmail.msn.com|/cgi-bin/hmhome?curmbox=F000000001&a=e7d2833da567c33fea303dc0c2e8e4f6&fti=yes
248|2004-04-26T18:19:18.883870000-0400|137.30.122.253|1600|64.4.43.250|80|GET|by17fd.bay17.hotmail.msn.com|/cgi-bin/dasp/EN/hotmail___9080000065.css
274|2004-04-26T18:19:25.449902000-0400|137.30.122.253|1602|64.4.43.250|80|GET|by17fd.bay17.hotmail.msn.com|/cgi-bin/dasp/EN/helppane___9080000001F.js
299|2004-04-26T18:19:27.347398000-0400|137.30.122.253|1605|64.4.43.250|80|GET|by17fd.bay17.hotmail.msn.com|/cgi-bin/dasp/EN/hotmail___90815000014.js
380|2004-04-26T18:19:28.334732000-0400|137.30.122.253|1612|64.4.48.24|80|GET|64.4.48.24|/crs_918.gif
383|2004-04-26T18:19:28.337259000-0400|137.30.122.253|1613|64.4.48.24|80|GET|64.4.48.24|/spacer.gif
388|2004-04-26T18:19:28.393263000-0400|137.30.122.253|1614|207.68.172.236|80|GET|sc.msn.com|/SN/VV82S]3UE33C7URIUIE[!.jpg
391|2004-04-26T18:19:28.405876000-0400|137.30.122.253|1615|207.68.177.124|80|GET|h.msn.com|/c.gif?RF=&PI=443646DI=74746PS=8313
404|2004-04-26T18:19:28.488470000-0400|137.30.122.253|1616|65.54.194.117|80|GET|rad.msn.com|/ADSAdClient31.dll?GetAd?PG=HOTO45?SC=PP?HM=04544b4654584a1250565547595047767908 4e3757042154155a535d52460e314149616d?LOC=H?TF=?ID=0006400086 5D48DD?UC=100?PS=55043?PI=44364?AP=1255?NA=1140?NC=9523?AT=0
407|2004-04-26T18:19:28.491024000-0400|137.30.122.253|1617|65.54.192.248|80|GET|popup.msn.com|/lbpopupframe.asp?PG=HOTP44&msid=00064000865D48DD&country=US&UC=100
430|2004-04-26T18:19:28.785489000-0400|137.30.122.253|1621|64.4.48.24|80|GET|64.4.48.24|/tab.bg.dln.gif
433|2004-04-26T18:19:28.791891000-0400|137.30.122.253|1622|64.4.48.24|80|GET|64.4.48.24|/lgo_msn_118×35.gif
451|2004-04-26T18:19:29.090330000-0400|137.30.122.253|1623|65.54.192.248|80|GET|popup.msn.com|/lbpopupad.asp?PG=HOTP44&msid=00064000865D48DD&country=US&UC=100
462|2004-04-26T18:19:29.366214000-0400|137.30.122.253|1626|64.4.48.24|80|GET|64.4.48.24|/tab.separator.on.l.gif
465|2004-04-26T18:19:29.379838000-0400|137.30.122.253|1627|64.4.48.24|80|GET|64.4.48.24|/tab.slide.hm.li.gif
475|2004-04-26T18:19:29.469998000-0400|137.30.122.253|1616|65.54.194.117|80|GET|rad.msn.com|/ADSAdClient31.dll?GetAd?PG=HOTHIA?AP=?TF=_blank
489|2004-04-26T18:19:29.819856000-0400|137.30.122.253|1630|64.4.48.24|80|GET|64.4.48.24|/tab.separator.on.r.gif
492|2004-04-26T18:19:29.826307000-0400|137.30.122.253|1631|64.4.48.24|80|GET|64.4.48.24|/tab.bg.on.gif
509|2004-04-26T18:19:30.274806000-0400|137.30.122.253|1634|64.4.48.24|80|GET|64.4.48.24|/tab.bg.off.gif
512|2004-04-26T18:19:30.294529000-0400|137.30.122.253|1635|64.4.48.24|80|GET|64.4.48.24|/tab.separator.off.gif
531|2004-04-26T18:19:30.747156000-0400|137.30.122.253|1638|64.4.48.24|80|GET|64.4.48.24|/i.p.writenew.gif
534|2004-04-26T18:19:30.761844000-0400|137.30.122.253|1639|64.4.48.24|80|GET|64.4.48.24|/tab.bg.sln.gif
551|2004-04-26T18:19:31.260540000-0400|137.30.122.253|1642|64.4.48.24|80|GET|64.4.48.24|/tab.separator.end.gif
554|2004-04-26T18:19:31.291951000-0400|137.30.122.253|1643|64.4.48.24|80|GET|64.4.48.24|/i.p.cont.individual.gif
565|2004-04-26T18:19:31.769558000-0400|137.30.122.253|1645|64.4.48.24|80|GET|64.4.48.24|/bgcolor.gif
575|2004-04-26T18:19:32.210983000-0400|137.30.122.253|1647|64.4.48.24|80|GET|64.4.48.24|/i.p.white.b.gif
585|2004-04-26T18:19:32.667107000-0400|137.30.122.253|1649|64.4.48.24|80|GET|64.4.48.24|/i.p.black.b.gif
2381|2004-04-26T18:22:28.510541000-0400|137.30.122.253|1664|64.4.43.250|80|GET|by17fd.bay17.hotmail.msn.com|/cgi-bin/compose?&curmbox=F000000001&a=e7d2833da567c33fea303dc0c2e8e4f6
2435|2004-04-26T18:22:29.902848000-0400|137.30.122.253|1667|64.4.48.24|80|GET|64.4.48.24|/i.p.send.gif
2436|2004-04-26T18:22:29.903483000-0400|137.30.122.253|1668|64.4.48.24|80|GET|64.4.48.24|/i.p.folder.drafts.gif
2467|2004-04-26T18:22:30.366189000-0400|137.30.122.253|1672|64.4.48.24|80|GET|64.4.48.24|/i.p.importance.h.gif
2468|2004-04-26T18:22:30.366473000-0400|137.30.122.253|1671|64.4.48.24|80|GET|64.4.48.24|/i.p.attach.gif
2493|2004-04-26T18:22:30.818427000-0400|137.30.122.253|1676|64.4.48.24|80|GET|64.4.48.24|/i.p.downarrow.gif
2496|2004-04-26T18:22:30.840024000-0400|137.30.122.253|1677|64.4.48.24|80|GET|64.4.48.24|/i.p.importance.l.gif
2499|2004-04-26T18:22:30.871787000-0400|137.30.122.253|1678|207.68.177.124|80|GET|h.msn.com|/c.gif?RF=&PI=443646DI=74746PS=8318
2520|2004-04-26T18:22:31.286729000-0400|137.30.122.253|1681|64.4.48.24|80|GET|64.4.48.24|/i.p.cancel.gif
2523|2004-04-26T18:22:31.306080000-0400|137.30.122.253|1682|64.4.48.24|80|GET|64.4.48.24|/i.p.tools.gif
2542|2004-04-26T18:22:31.729957000-0400|137.30.122.253|1685|64.4.48.24|80|GET|64.4.48.24|/i.p.attachfile.gif


File Actions Edit View Help
2756|2004-04-26T18:23:36.190444000-0400|137.30.122.253|1709|207.68.171.245|80|GET|www.msn.com|/
2776|2004-04-26T18:23:37.150739000-0400|137.30.122.253|1711|207.68.171.245|80|GET|www.msn.com|/
2814|2004-04-26T18:23:38.206104000-0400|137.30.122.253|1714|67.72.8.94|80|GET|hp.msn.com|/css/home/css-font.css?v=1
2817|2004-04-26T18:23:38.210414000-0400|137.30.122.253|1715|67.72.8.94|80|GET|hp.msn.com|/css/home/home-ie6.css?v=2
2838|2004-04-26T18:23:38.600061000-0400|137.30.122.253|1717|67.72.8.94|80|GET|hp.msn.com|/css/home/home-bb-ie6.css?v=2
2857|2004-04-26T18:23:39.029436000-0400|137.30.122.253|1719|67.72.8.94|80|GET|hp.msn.com|/scr/home/helppane26.js
2879|2004-04-26T18:23:39.455645000-0400|137.30.122.253|1721|67.72.8.94|80|GET|hp.msn.com|/scr/home/home.js
2915|2004-04-26T18:23:39.939416000-0400|137.30.122.253|1726|67.72.8.94|80|GET|hp.msn.com|/global/c/lg/msft_118×35.gif
2918|2004-04-26T18:23:39.945218000-0400|137.30.122.253|1727|67.72.8.94|80|GET|hp.msn.com|/c/home/c.gif
2921|2004-04-26T18:23:39.952399000-0400|137.30.122.253|1725|207.68.177.126|80|GET|c.msn.com|/c.gif?na=1154&nc=10009&ps=70634&di=340&pi=73176TP=http%3a%2f%2fwww.msn.com%2f
2929|2004-04-26T18:23:40.038179000-0400|137.30.122.253|1727|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/sf_rpt.jpg
2949|2004-04-26T18:23:40.350248000-0400|137.30.122.253|1730|67.72.8.94|80|GET|hp.msn.com|/c/home/srch_ar.gif
2959|2004-04-26T18:23:40.446403000-0400|137.30.122.253|1731|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/sf.jpg
2973|2004-04-26T18:23:40.745621000-0400|137.30.122.253|1734|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/lfade.gif
2981|2004-04-26T18:23:40.843673000-0400|137.30.122.253|1735|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/ql_bg.gif
3001|2004-04-26T18:23:41.130115000-0400|137.30.122.253|1738|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/ccol.gif
3008|2004-04-26T18:23:41.254651000-0400|137.30.122.253|1739|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/sf_btm.gif
3019|2004-04-26T18:23:41.549917000-0400|137.30.122.253|1742|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/center_top.jpg
3028|2004-04-26T18:23:41.702945000-0400|137.30.122.253|1743|67.72.8.94|80|GET|hp.msn.com|/c/home/bb/nip_lt.gif
3040|2004-04-26T18:23:42.007433000-0400|137.30.122.253|1746|67.72.8.94|80|GET|hp.msn.com|/c/home/srch_rp.gif
3048|2004-04-26T18:23:42.115851000-0400|137.30.122.253|1747|67.72.8.94|80|GET|hp.msn.com|/scr/home/slideshow.js?v=3
3064|2004-04-26T18:23:42.549224000-0400|137.30.122.253|1749|67.72.8.94|80|GET|hp.msn.com|/scr/home/flash.js
3081|2004-04-26T18:23:43.016150000-0400|137.30.122.253|1752|67.72.8.94|80|GET|hp.msn.com|/29/`0{TA9GLAS4U_ETRY0!9@6.jpg
3082|2004-04-26T18:23:43.016853000-0400|137.30.122.253|1753|67.72.8.94|80|GET|hp.msn.com|/css/home/bbip.css?v=2
3110|2004-04-26T18:23:43.285183000-0400|137.30.122.253|1756|216.39.69.77|80|GET|view.atdmt.com|/AVE/view/msnnkonm00200074ave/direct;wi.309;hi.60/01/
3119|2004-04-26T18:23:43.328373000-0400|137.30.122.253|1752|67.72.8.94|80|GET|hp.msn.com|/4D/FYA0_+![382I!P,3M31—W.jpg
3134|2004-04-26T18:23:43.432954000-0400|137.30.122.253|1757|67.72.8.94|80|GET|hp.msn.com|/c/home/left.gif
3150|2004-04-26T18:23:43.719083000-0400|137.30.122.253|1760|198.5.148.7|80|GET|spe.atdmt.com|/b/AAAVEMSFTONM/IS_is_lspm3ltbl_309×60_0204.gif
3163|2004-04-26T18:23:43.827459000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/3T/U,~,65~+6JH3[TVS`6R{Z.jpg
3172|2004-04-26T18:23:43.853084000-0400|137.30.122.253|1752|67.72.8.94|80|GET|hp.msn.com|/c/home/right.gif
3198|2004-04-26T18:23:44.111516000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/5@/4XPH!XGOT@1V+E4MZ+61MF.jpg
3205|2004-04-26T18:23:44.288616000-0400|137.30.122.253|1763|67.72.8.94|80|GET|hp.msn.com|/36/MPY8}V9Y1C25`L-JDAG3~1.jpg
3214|2004-04-26T18:23:44.442600000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/1K/P5NBIXZ0_FAWRG7HA`+JX.jpg
3229|2004-04-26T18:23:44.590996000-0400|137.30.122.253|1763|67.72.8.94|80|GET|hp.msn.com|/4~/B268LBZ]PA1IALI!}-1AQS.jpg
3237|2004-04-26T18:23:44.772233000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/4Q/-87E3`Q{3-_O[H7!C71]7.jpg
3246|2004-04-26T18:23:45.066666000-0400|137.30.122.253|1763|67.72.8.94|80|GET|hp.msn.com|/c/home/bg_lt.gif
3247|2004-04-26T18:23:45.126100000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/2C/Gi_,_966+]TB5_~{M2CN8.gif
3255|2004-04-26T18:23:45.466917000-0400|137.30.122.253|1761|67.72.8.94|80|GET|hp.msn.com|/c/home/bg_rt.gif
3258|2004-04-26T18:23:45.478861000-0400|137.30.122.253|1765|67.72.8.94|80|GET|hp.msn.com|/c/home/stock-dn.gif
3267|2004-04-26T18:23:45.820653000-0400|137.30.122.253|1765|67.72.8.94|80|GET|hp.msn.com|/c/home/bbip.jpg
3270|2004-04-26T18:23:45.883362000-0400|137.30.122.253|1767|67.72.8.94|80|GET|hp.msn.com|/2Z/53_IXHX}UI3X4@IS~PT8ZE.gif
3285|2004-04-26T18:23:46.047228000-0400|137.30.122.253|1767|67.72.8.94|80|GET|hp.msn.com|/c/home/bullets/FFFFFF.gif





From the visible results, we can already identify several useful leads:

- rhino.log - 26 April 2004: HTTP requests from 137.30.122.253 include Google and Hotmail access, Microsoft Passport login and logout requests, and Hotmail mailbox actions. There are also requests for images and other web-page resources.
- rhino2.log - 28 April 2004: HTTP requests from 137.30.123.234 include Google and a request to www.cs.uno.edu/~gnome

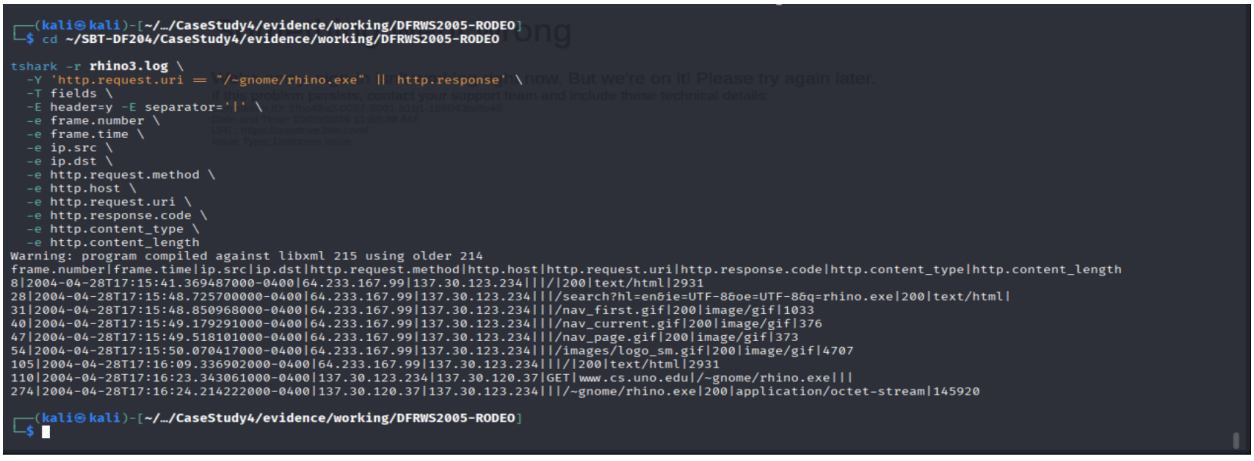
- The logs include HTTP POST requests, but a POST alone does not establish what information was submitted or whether it was successfully received.
- The request paths and timestamps are leads for further investigation, not proof of a particular person's actions.

HTTP Retrieval of rhino.exe

Further examination of the rhino3.log packet capture identified an HTTP GET request for /~gnome/rhino.exe on the host www.cs.uno.edu. The request was recorded in frame 110 at 17:16:23.343 on 28 April 2004 (UTC offset −0400). The source IP address was 137.30.123.234, and the destination IP address was 137.30.120.37, communicating over HTTP.

A corresponding server response was recorded in frame 274 at 17:16:24.214. The response status was **200 OK**, with the content type reported as application/octet-stream and a content length of **145,920 bytes**. The interval between the recorded request and response was approximately 0.871 seconds. These packet-level observations indicate that the server returned data in response to the request for rhino.exe.

This is a significant network artefact because it establishes a recorded request for an executable file and a corresponding successful HTTP response. However, the capture alone does not establish whether the received data was written to disk, whether the file was subsequently executed, or which individual was responsible for the network activity. Those questions require corroboration from other evidence sources, such as endpoint artefacts, file-system records, or execution-related logs.



The output confirms a significant finding: the capture records an HTTP request for rhino.exe, followed by a successful HTTP response (200 OK) containing 145,920 bytes of application/octet-stream data.

The request occurred on 28 April 2004 at 17:16:23.343 (-0400). The response was recorded at 17:16:24.214 (-0400), approximately 0.871 seconds later. This supports the conclusion that the server returned data for the requested executable. It does not, by itself, prove that the file was saved to disk or executed on the requesting system.

TCP Stream Correlation

To validate the relationship between the HTTP request and response identified in the preceding analysis, frames 110 and 274 of rhino3.log were examined for TCP stream identifiers, port numbers, TCP payload lengths, and HTTP metadata.

Both frames were assigned to **TCP stream 3**. Frame 110 records a GET request from source port 2163 to destination port 80 for /~gnome/rhino.exe. Frame 274 records traffic in the reverse direction, from source port 80 to destination port 2163, with an HTTP status of 200, content type application/octet-stream, and a reported content length of 145,920 bytes.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ tshark -r rhino3.log \
  -Y 'frame.number == 110 || frame.number == 274' \
  -T fields \
  -E header=y -E separator='|' \
  -e frame.number \
  -e tcp.stream \
  -e tcp.srcport \
  -e tcp.dstport \
  -e tcp.len \
  -e tcp.flags \
  -e http.request.method \
  -e http.request.uri \
  -e http.response.code \
  -e http.content_type \
  -e http.content_length
Warning: program compiled against libxml 215 using older 214
frame.number|tcp.stream|tcp.srcport|tcp.dstport|tcp.len|tcp.flags|http.request.method|http.request.uri|http.response.code|http.content_type|http.content_l
ength
110|3|2163|80|393|0x0018|GET|/~gnome/rhino.exe|||
274|3|80|2163|248|0x0018||/~gnome/rhino.exe|200|application/octet-stream|145920
```

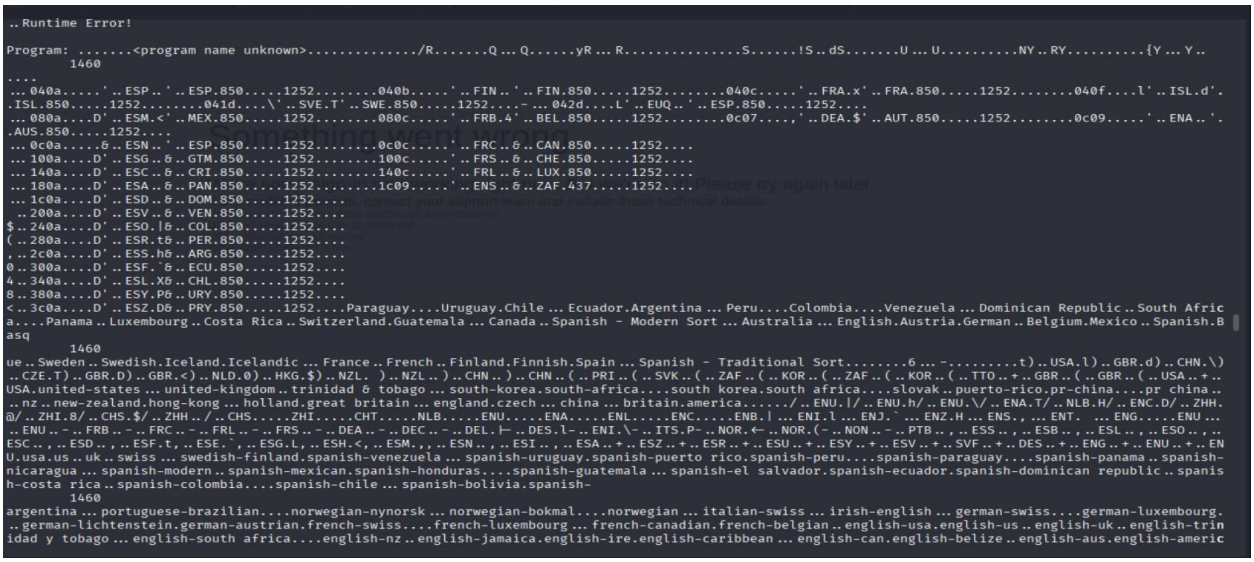
The matching stream identifier, reversed port direction, and matching requested resource support the interpretation that the two frames belong to the same TCP conversation and are associated with the attempted retrieval of rhino.exe. The reported tcp.len values of 393 and 248 represent TCP payload lengths in the respective frames; they are not the total size of the HTTP object. The HTTP content-length field provides the reported object size, but these two selected frames alone do not establish that every byte of the object was captured or reassembled.

Executable Content Identification

The ASCII representation of TCP stream 3 was examined to further characterize the data returned in response to the request for rhino.exe. The HTTP response contains an MZ signature at the beginning of the displayed object data, followed by the familiar DOS-stub message, “This program cannot be run in DOS mode.” The stream also contains strings referencing Windows libraries, including KERNEL32.dll, ADVAPI32.dll, and USER32.dll, as well as Microsoft Visual C++ runtime messages.

These observations are consistent with Windows executable content and provide additional support for identifying the HTTP response as a transfer of executable data. The HTTP headers report a content length of 145,920 bytes. However, the ASCII stream display is a representation of binary data and is not, by itself, a complete integrity or malware analysis. No conclusion is made about the executable's maliciousness or whether it was run on the destination computer. The executable was not executed as part of this examination.







Preliminary Telnet and FTP Traffic Analysis

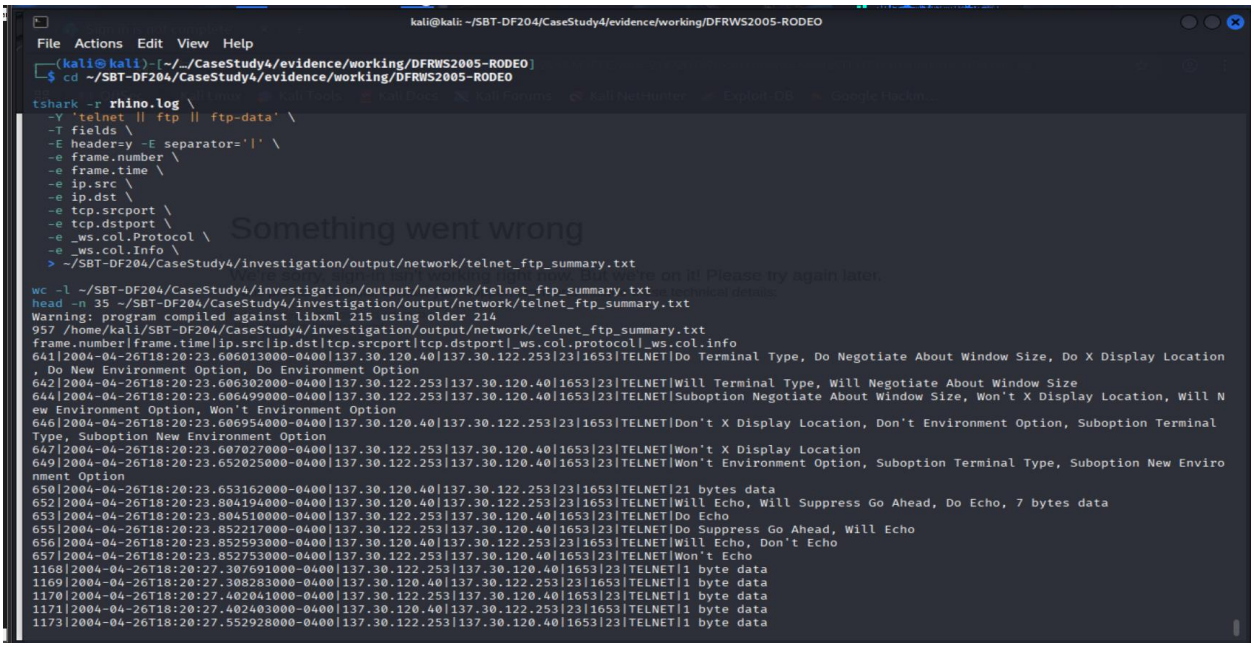
The rhino.log packet capture was filtered for Telnet, FTP, and FTP-DATA traffic. The resulting summary contained 957 lines, including the column header. The first 35 lines were reviewed to identify the initial session activity.

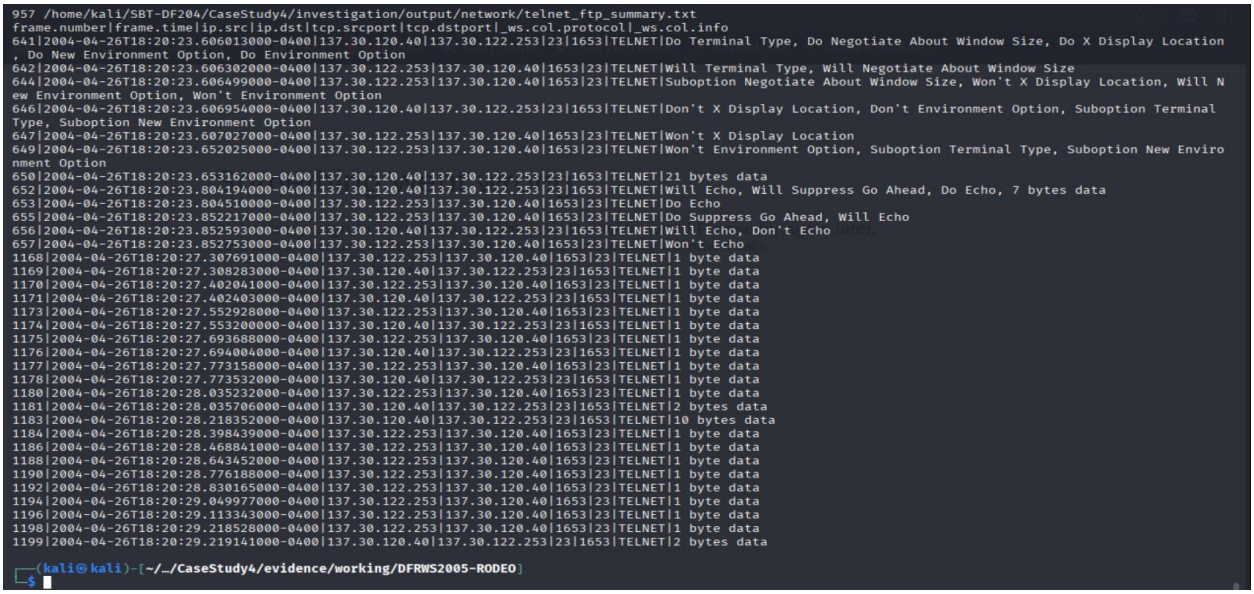
The displayed packets show a Telnet connection between 137.30.122.253 and 137.30.120.40, using TCP port 1653 on one endpoint and TCP port 23 on the other. The session begins with Telnet option negotiation, including terminal type, window size, echo,

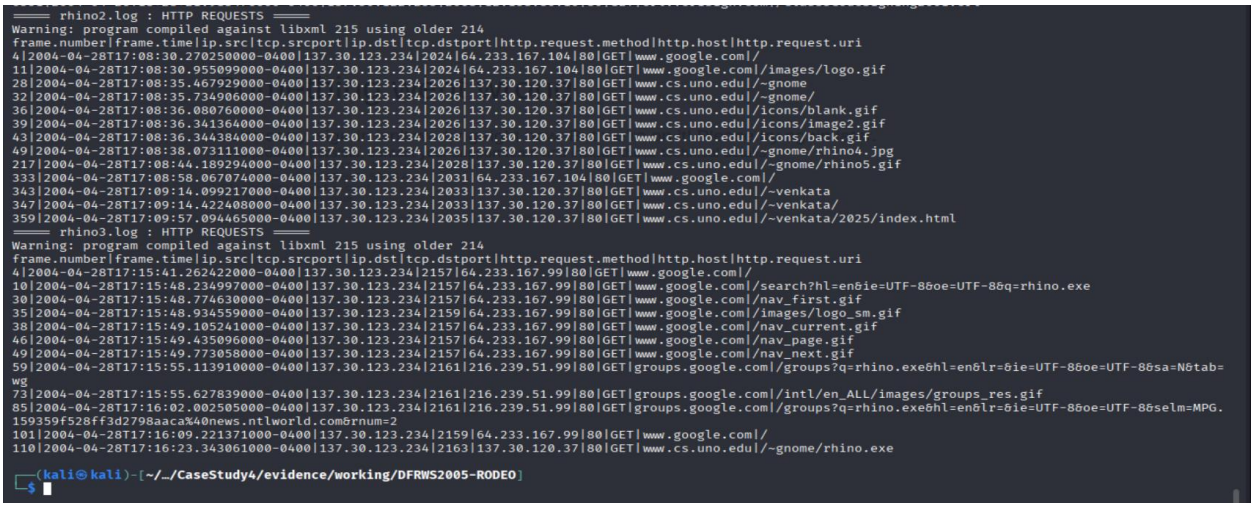
and environment-related options. This is followed by bidirectional exchanges of small amounts of data between the endpoints.

These observations establish that Telnet communication occurred during the captured period. However, the displayed summary does not reveal the full meaning of the exchanged data or establish what commands, if any, were successfully executed. Because Telnet is an unencrypted protocol, session contents may be visible in the packet capture; any credentials or other sensitive information must be handled carefully and excluded from screenshots and report excerpts unless specifically required by the investigation.

Further examination of the session and the FTP-related packets is necessary before making claims about authentication, commands, or file transfers.







Telnet and FTP Session Enumeration

A protocol-specific examination of rhino.log was conducted to identify the number of TCP streams associated with Telnet and FTP traffic and to summarize the FTP commands and server response codes.

The analysis identified three distinct TCP streams containing Telnet traffic and three containing FTP traffic. The FTP command summary recorded four STOR commands, four PORT commands, three USER commands, three PASS commands, three TYPE commands, and three QUIT commands. The response summary included three 230 responses, indicating successful FTP login responses, alongside 331 responses requesting password submission, 150 responses indicating that a data connection was

opening for a transfer, and four 226 responses indicating that a transfer or related file action completed successfully.

```
echo "== Telnet TCP streams =="
tshark -r rhino.log -Y telnet -T fields -e tcp.stream \
  | sort -n -u | wc -l

echo "== FTP TCP streams =="
tshark -r rhino.log -Y ftp -T fields -e tcp.stream \
  | sort -n -u | wc -l

echo "== FTP command types =="
tshark -r rhino.log -Y 'ftp.request.command' \
  -T fields -e ftp.request.command \
  | sort | uniq -c | sort -nr

echo "== FTP response codes =="
tshark -r rhino.log -Y 'ftp.response.code' \
  -T fields -e ftp.response.code \
  | sort | uniq -c | sort -nr
== Telnet TCP streams ==
Warning: program compiled against libxml 215 using older 214
3
== FTP TCP streams ==
Warning: program compiled against libxml 215 using older 214
3
== FTP command types ==
Warning: program compiled against libxml 215 using older 214
      4 STOR
      4 PORT
      3 USER
      3 TYPE
      3 QUIT
      3 PASS
== FTP response codes ==
Warning: program compiled against libxml 215 using older 214
      7 200
      6 221
      4 226
      4 150
      3 331
      3 230
      3 220
```

The presence of four STOR commands is a significant lead because this command is used to request storage of a file on the FTP server. The associated 150 and 226 responses warrant further correlation with individual commands, filenames, and data connections. However, aggregate command counts alone do not establish which files were transferred, whether every STOR operation succeeded, or the nature of the transferred content. These questions require examination of the FTP control conversations and their associated data streams.

Protocol Hierarchy Analysis

The protocol hierarchy of rhino.log was examined using Wireshark's TShark analysis functionality. The capture contains 6,557 frames and 3,082,971 bytes. The protocol

hierarchy identifies HTTP (413 frames), TLS (16 frames), MSN Messenger (38 frames), IMAP (1,051 frames), Telnet (568 frames), FTP (50 frames), and FTP-DATA (338 frames). The FTP-DATA branch accounts for 508,319 bytes, indicating a substantial volume of data associated with FTP data-channel traffic. This observation is consistent with the earlier identification of FTP file-storage commands, including four STOR requests. The capture also contains HTTP traffic classified as GIF and JPEG image content, as well as text and URL-encoded data.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ tshark -r rhino.log -q -z io,phs
Warning: program compiled against libxml 215 using older 214

===================================================================
Protocol Hierarchy Statistics
Filter:

frame                                    frames:6557 bytes:3082971
  eth                                    frames:6557 bytes:3082971
    ip                                   frames:6557 bytes:3082971
      tcp                                frames:6557 bytes:3082971
        http                             frames:413 bytes:212533
          data-text-lines                frames:36 bytes:25568
          urlencoded-form                frames:6 bytes:4436
          image-gif                      frames:68 bytes:36990
          image-jfif                     frames:12 bytes:13366
        tls                              frames:16 bytes:9212
        msnms                            frames:38 bytes:2532
        imap                             frames:1051 bytes:1443797
        telnet                           frames:568 bytes:38205
          _ws.malformed                  frames:3 bytes:180
        ftp                              frames:50 bytes:4428
        ftp-data                         frames:338 bytes:508319
===================================================================
```

These statistics provide an overview of the protocols present and help prioritize subsequent examination. They do not independently establish the purpose of the communication, the identity of a user, or the content and outcome of individual transfers. Further analysis of FTP control commands and their associated data connections is required to identify specific file-transfer activity.

FTP Session and File Transfer Analysis

The FTP control-channel analysis identified three sessions between 137.30.122.253 and the FTP server at 137.30.120.40. Each session began with the server returning response code 220, indicating that the FTP service was ready. The client then submitted the username gnome, received response code 331 requesting a password, and subsequently received response code 230, indicating successful authentication. The capture contains the password submitted during these sessions in plaintext. This demonstrates that the

FTP session did not protect the login credentials from interception within the captured traffic.

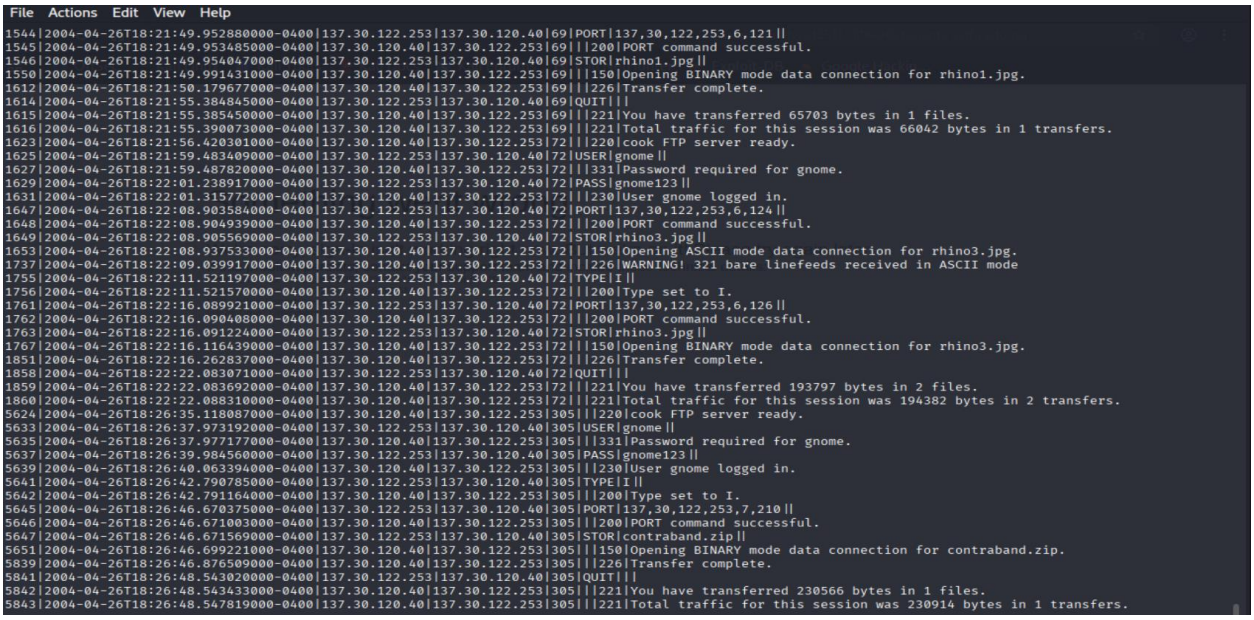
The first session, TCP stream 69, occurred on 26 April 2004 at approximately 18:21:43–18:21:55 (capture-local time, UTC−04:00). Following authentication, the client issued a STOR command for rhino1.jpg. The server returned response code 150, indicating that the data connection was opening, followed by response code 226, indicating completion of the transfer. The server's closing response reported that 65,703 bytes had been transferred in one file.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ tshark -r rhino.log \
  -Y 'ftp.request.command || ftp.response.code' \
  -T fields \
  -E header=y -E separator='|' \
  -e frame.number \
  -e frame.time \
  -e ip.src \
  -e ip.dst \
  -e tcp.stream \
  -e ftp.request.command \
  -e ftp.request.arg \
  -e ftp.response.code \
  -e ftp.response.arg
Warning: program compiled against libxml 215 using older 214
frame.number|frame.time|ip.src|ip.dst|tcp.stream|ftp.request.command|ftp.request.arg|ftp.response.code|ftp.response.arg
1529|2004-04-26T18:21:39.998180000-0400|137.30.120.40|137.30.122.253|69|||220|cook FTP server ready.
1532|2004-04-26T18:21:43.598613000-0400|137.30.122.253|137.30.120.40|69|USER|gnome||
1534|2004-04-26T18:21:43.602936000-0400|137.30.120.40|137.30.122.253|69|||331|Password required for gnome.
1536|2004-04-26T18:21:45.625720000-0400|137.30.122.253|137.30.120.40|69|PASS|gnome123||
1538|2004-04-26T18:21:45.706912000-0400|137.30.120.40|137.30.122.253|69|||230|User gnome logged in.
1540|2004-04-26T18:21:46.560519000-0400|137.30.122.253|137.30.120.40|69|TYPE|I||
1541|2004-04-26T18:21:46.560784000-0400|137.30.120.40|137.30.122.253|69|||200|Type set to I.
1544|2004-04-26T18:21:49.952880000-0400|137.30.122.253|137.30.120.40|69|PORT|137,30,122,253,6,121||
1545|2004-04-26T18:21:49.953485000-0400|137.30.120.40|137.30.122.253|69|||200|PORT command successful.
1546|2004-04-26T18:21:49.954047000-0400|137.30.122.253|137.30.120.40|69|STOR|rhino1.jpg||
1550|2004-04-26T18:21:49.991431000-0400|137.30.120.40|137.30.122.253|69|||150|Opening BINARY mode data connection for rhino1.jpg.
1612|2004-04-26T18:21:50.179677000-0400|137.30.120.40|137.30.122.253|69|||226|Transfer complete.
1614|2004-04-26T18:21:55.384845000-0400|137.30.122.253|137.30.120.40|69|QUIT|||
1615|2004-04-26T18:21:55.385450000-0400|137.30.120.40|137.30.122.253|69|||221|You have transferred 65703 bytes in 1 files.
1616|2004-04-26T18:21:55.390073000-0400|137.30.120.40|137.30.122.253|69|||221|Total traffic for this session was 66042 bytes in 1 transfers.
1623|2004-04-26T18:21:56.420301000-0400|137.30.120.40|137.30.122.253|72|||220|cook FTP server ready.
1625|2004-04-26T18:21:59.483409000-0400|137.30.122.253|137.30.120.40|72|USER|gnome||
1627|2004-04-26T18:21:59.487820000-0400|137.30.120.40|137.30.122.253|72|||331|Password required for gnome.
1629|2004-04-26T18:22:01.238917000-0400|137.30.122.253|137.30.120.40|72|PASS|gnome123||
1631|2004-04-26T18:22:01.315772000-0400|137.30.120.40|137.30.122.253|72|||230|User gnome logged in.
1647|2004-04-26T18:22:08.903584000-0400|137.30.122.253|137.30.120.40|72|PORT|137,30,122,253,6,124||
1648|2004-04-26T18:22:08.904939000-0400|137.30.120.40|137.30.122.253|72|||200|PORT command successful.
1649|2004-04-26T18:22:08.905569000-0400|137.30.122.253|137.30.120.40|72|STOR|rhino3.jpg||
1653|2004-04-26T18:22:08.937533000-0400|137.30.120.40|137.30.122.253|72|||150|Opening ASCII mode data connection for rhino3.jpg.
1737|2004-04-26T18:22:09.039917000-0400|137.30.120.40|137.30.122.253|72|||226|WARNING! 321 bare linefeeds received in ASCII mode
1755|2004-04-26T18:22:11.521197000-0400|137.30.122.253|137.30.120.40|72|TYPE|I||
1756|2004-04-26T18:22:11.521570000-0400|137.30.120.40|137.30.122.253|72|||200|Type set to I.
```

The second session, TCP stream 72, occurred at approximately 18:21:59–18:22:22. The client issued STOR commands for rhino3.jpg twice. The first transfer was conducted in ASCII mode, and the server returned a completion message noting 321 bare line feeds received in ASCII mode. The client subsequently switched to binary mode using the TYPE I command and initiated another transfer of rhino3.jpg. The server then returned 226 Transfer complete. Its session summary reported two files transferred, totalling 193,797 bytes. The capture therefore records two successful transfer completions associated with the same filename, although the control-channel output alone does not establish whether the transferred file contents were identical.

The third session, TCP stream 305, occurred at approximately 18:26:35–18:26:48. After authentication, the client switched to binary mode and issued STOR contraband.zip. The server returned 150, followed by 226 Transfer complete. The closing session message

reported that 230,566 bytes had been transferred in one file. This is evidence of a completed FTP upload transaction for a file named contraband.zip; the filename alone does not establish the archive's actual contents.

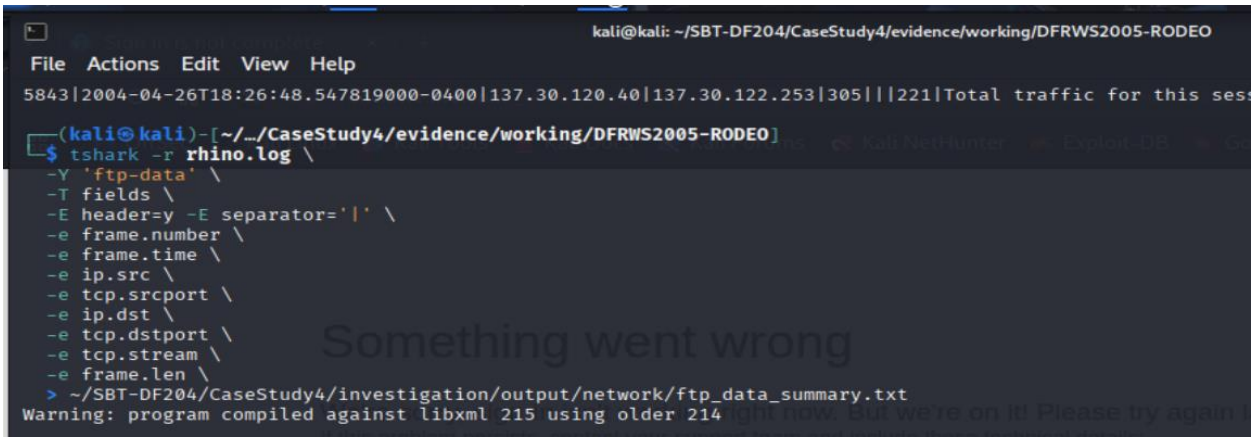


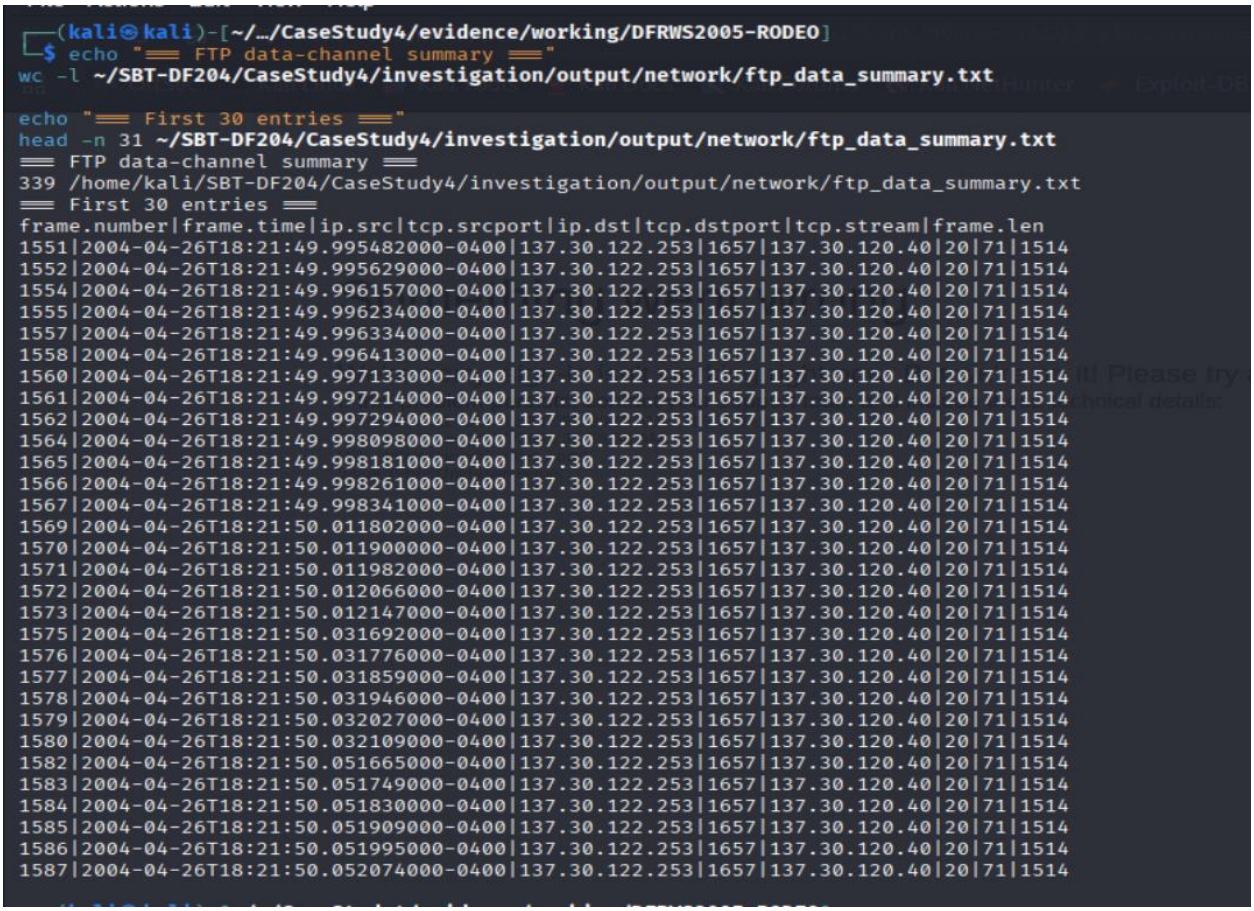
Overall, the captured FTP traffic records three authenticated sessions and four STOR commands: one for rhino1.jpg, two for rhino3.jpg, and one for contraband.zip. The server responses support the conclusion that the corresponding transfer operations completed. However, attributing the activity to a particular individual, determining the actual contents of each transferred file, or confirming how the files were subsequently used would require corroborating evidence.

FTP Data-Channel Analysis

The FTP data-channel extraction produced 338 packet entries, excluding the header, from rhino.log. These entries were identified using Wireshark's ftp-data display filter and recorded with their frame numbers, timestamps, source and destination IP addresses and ports, TCP stream identifiers, and frame lengths.

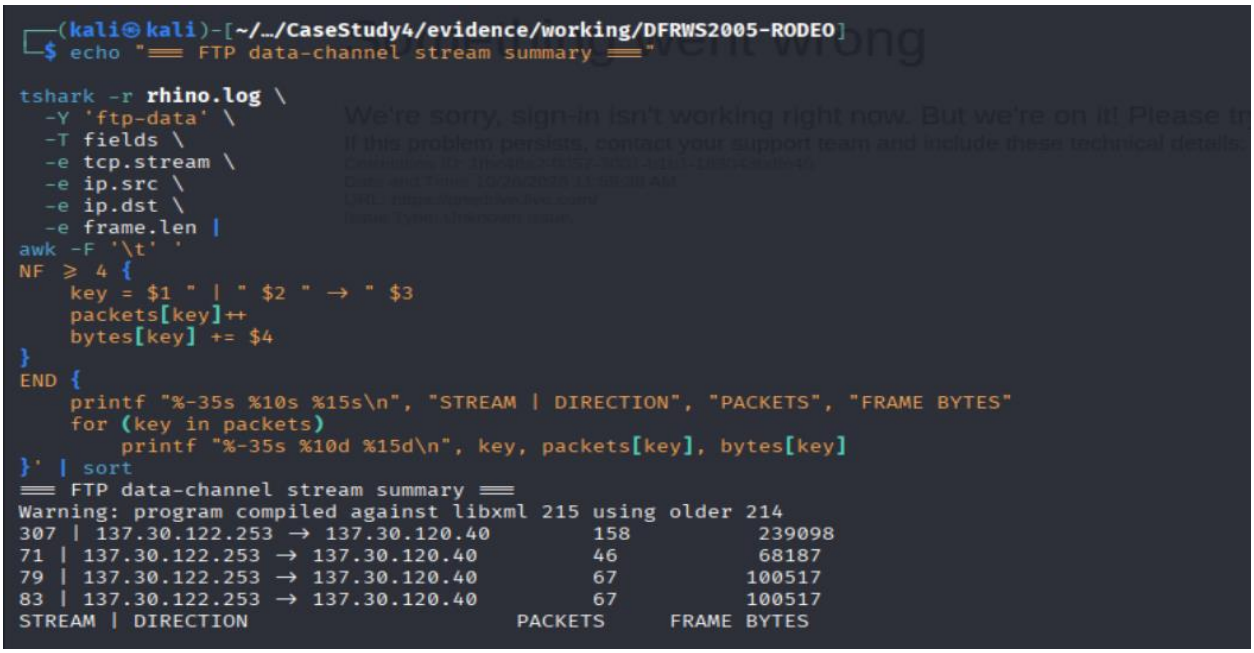
The first entries shown relate to TCP stream 71, with traffic from 137.30.122.253:1657 to 137.30.120.40:20. The packets occur around 18:21:49–18:21:50 on 26 April 2004 (UTC−04:00), corresponding closely to the FTP session in which the control channel recorded a STOR rhino1.jpg request. This temporal and endpoint correlation is consistent with the data channel carrying the associated upload.





The output also contains many frames with a length of 1,514 bytes. These are individual captured frame lengths; they should not be treated as file sizes or added together without accounting for TCP retransmissions, protocol overhead, and data-stream reassembly. The FTP data-channel listing is useful for correlating traffic, but it does not independently establish the complete contents of each transferred file.

FTP Data-Stream Correlation



The stream summary identified four TCP streams carrying traffic classified as FTP data, all in the direction from 137.30.122.253 to 137.30.120.40.

TCP stream	Packet entries	Total captured frame bytes
71	46	68,187
79	67	100,517
83	67	100,517
307	158	239,098

The repeated packet count and byte total for streams 79 and 83 are noteworthy. However, identical totals alone do not establish that the streams contain identical file data. Likewise, the frame-byte totals include captured protocol overhead and are not verified file sizes.

The four streams are consistent with multiple FTP data transfers. The earlier control-channel evidence recorded four STOR commands, but the data streams must be correlated with their respective control sessions before assigning filenames to individual streams. The present summary does not identify the transferred file associated with each stream by itself.

FTP Data-Stream Timeline and Endpoint Correlation

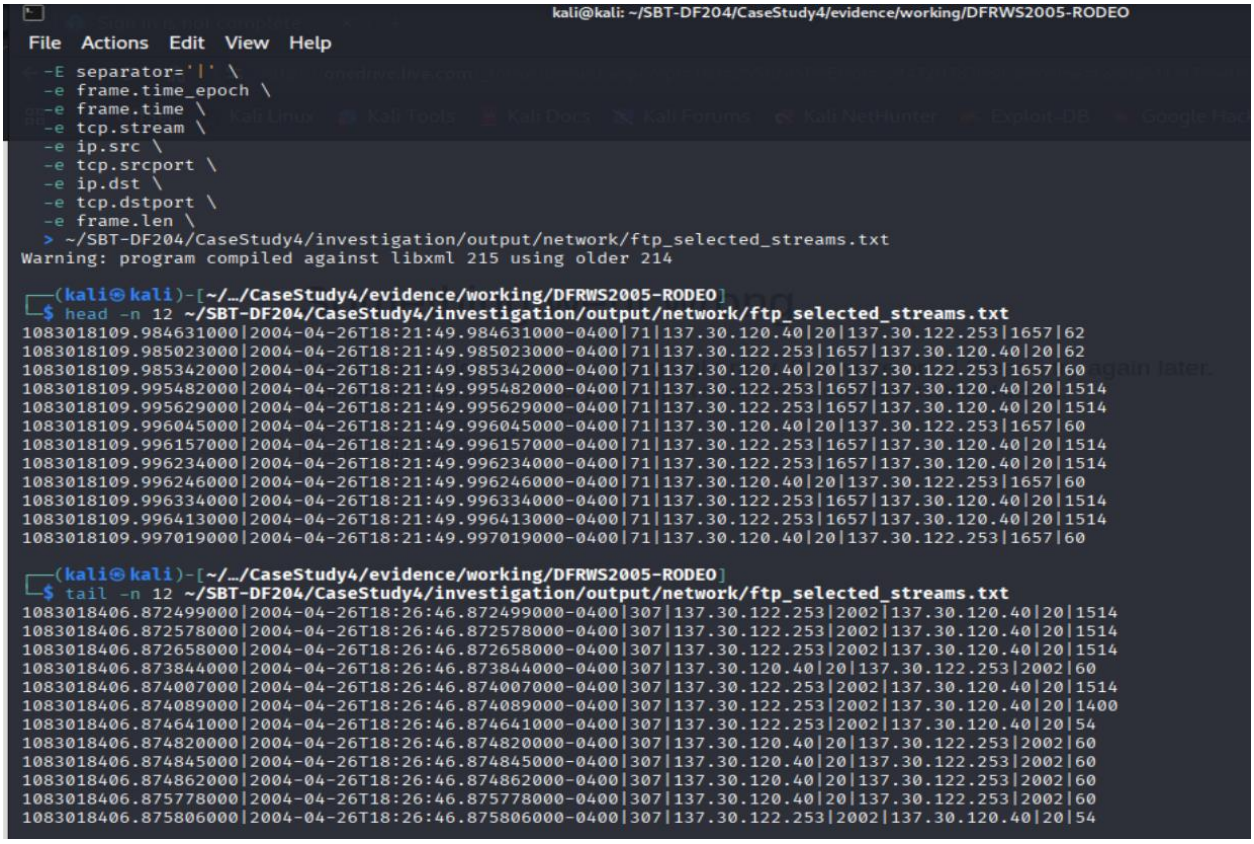
The selected TCP streams were examined to establish their timestamps, endpoint addresses, port numbers, and frame lengths. The output confirms that TCP stream 71 began at approximately 18:21:49.984 on 26 April 2004, with traffic exchanged between 137.30.120.40:20 and 137.30.122.253:1657. This is consistent with the FTP control-channel record for the STOR rhino1.jpg command, issued shortly before the data connection began. The observed data traffic is therefore strongly correlated in time and endpoints with the first recorded file upload.

The final entries supplied show TCP stream 307 at approximately 18:26:46.872–18:26:46.875, with data flowing from 137.30.122.253:2002 to 137.30.120.40:20. This timing closely matches the FTP control-channel record in which the client issued STOR contraband.zip at 18:26:46.671, received a 150 response indicating that the data connection was opening, and later received 226 Transfer complete. The timing and endpoint information strongly support associating stream 307 with the contraband.zip upload.

The displayed beginning and end of the file do not show the complete timelines for streams 79 and 83. Their filenames should therefore not yet be assigned based on this output alone. The frame lengths also represent captured network frames rather than the verified size of any transferred file.

The timestamps and ports give us a useful correlation point.

- TCP stream 71 uses data port 1657 on 137.30.122.253 and server port 20 on 137.30.120.40. Its activity begins at approximately 18:21:49.984, matching the recorded STOR rhino1.jpg transaction.
- TCP stream 307 uses data port 2002 and begins around 18:26:46, matching the recorded STOR contraband.zip transaction.
- The output from head and tail does not show the start and end times for streams 79 and 83, so we should not assign those streams to particular filenames yet.



- Stream 71: Strongly correlated with the rhino1.jpg upload.
- Stream 307: Strongly correlated with the contraband.zip upload.
- Streams 79 and 83: Require further correlation with the remaining FTP control-channel events.
- Evidence limitation: These network records establish transfer activity, but do not independently establish the files' complete contents or subsequent use.

FTP Data-Stream Timeline Correlation

The timeline summary establishes the start and end times of all four identified FTP data streams and provides additional support for correlating them with the FTP control-channel events recorded in rhino.log.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ awk -F '|' '
{
    s=$3
    if (!(s in first)) {
        first[s]=$2
        src[s]=$4 ":" $5
        dst[s]=$6 ":" $7
    }
    last[s]=$2
    count[s]++
}
END {
    printf "%-8s %-30s %-30s %-8s\n",
           "STREAM", "FIRST PACKET", "LAST PACKET", "PACKETS"
    for (s in first)
        printf "%-8s %-30s %-30s %-8d\n",
               s, first[s], last[s], count[s]
}' ~/SBT-DF204/CaseStudy4/investigation/output/network/ftp_selected_streams.txt |
sort -n
STREAM   FIRST PACKET                   LAST PACKET                    PACKETS
71       2004-04-26T18:21:49.984631000-0400 2004-04-26T18:21:50.093830000-0400 63
79       2004-04-26T18:22:08.929029000-0400 2004-04-26T18:22:08.997384000-0400 85
83       2004-04-26T18:22:16.109635000-0400 2004-04-26T18:22:16.262129000-0400 85
307      2004-04-26T18:26:46.690952000-0400 2004-04-26T18:26:46.875806000-0400 189
```

TCP stream	First packet (UTC−04:00)	Last packet (UTC−04:00)	Packet entries
71	18:21:49.984	18:21:50.093	63
79	18:22:08.929	18:22:08.997	85
83	18:22:16.109	18:22:16.262	85
307	18:26:46.690	18:26:46.875	189

Stream 71 aligns with the rhino1.jpg upload, while stream 307 aligns with the contraband.zip upload. The remaining two streams began shortly after the respective STOR rhino3.jpg commands: stream 79 at approximately 18:22:08.929 and stream 83 at approximately 18:22:16.109. These timings support associating the streams with the two rhino3.jpg transfer attempts recorded in TCP stream 72 of the FTP control channel.

The data-stream timeline therefore provides a consistent correlation between the four observed data streams and the four recorded STOR commands. This correlation is based on matching timestamps and endpoints; it does not establish the exact content of the transferred files. The packet-entry totals also differ from the earlier summary because that summary filtered specifically for packets dissected as ftp-data, whereas this timeline includes all packets in the selected TCP streams.

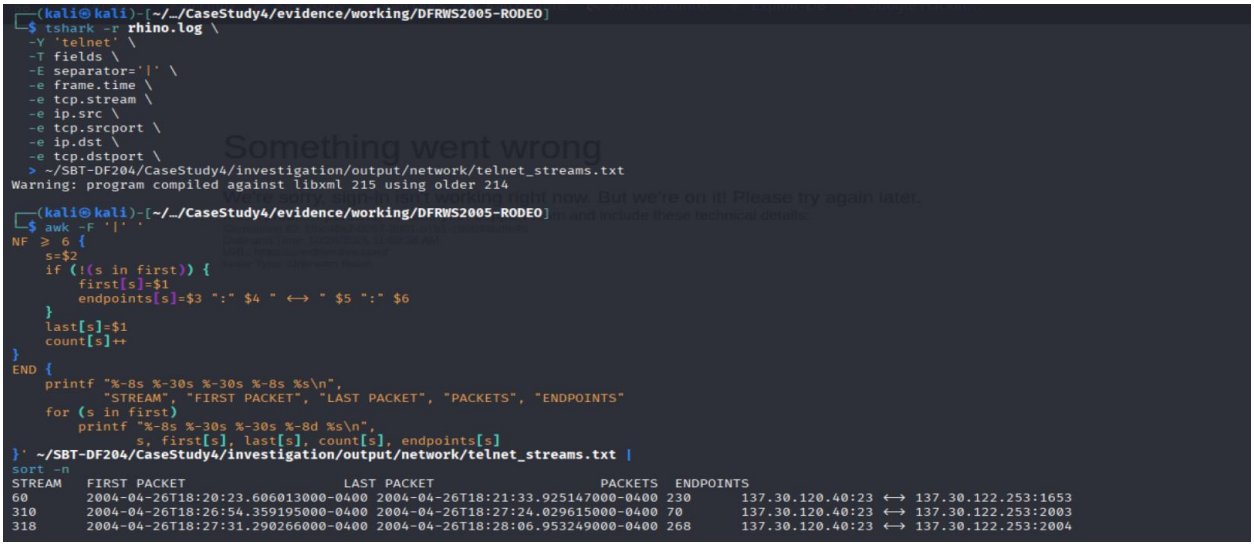
Telnet Session Timeline and Endpoint Analysis

The Telnet traffic extracted from rhino.log was grouped by TCP stream to establish the duration, packet count, and communicating endpoints of each session. The analysis

identified three TCP streams containing Telnet traffic, all involving 137.30.120.40 on TCP port 23 and 137.30.122.253 using different client-side ports.

TCP stream	Start time (UTC−04:00)	End time (UTC−04:00)	Packet entries
60	18:20:23.606	18:21:33.925	230
310	18:26:54.359	18:27:24.029	70
318	18:27:31.290	18:28:06.953	268

Stream 60 spans approximately 70 seconds, stream 310 approximately 30 seconds, and stream 318 approximately 36 seconds. The packet counts indicate that stream 318 contains the largest number of frames classified as Telnet, followed by stream 60 and stream 310.



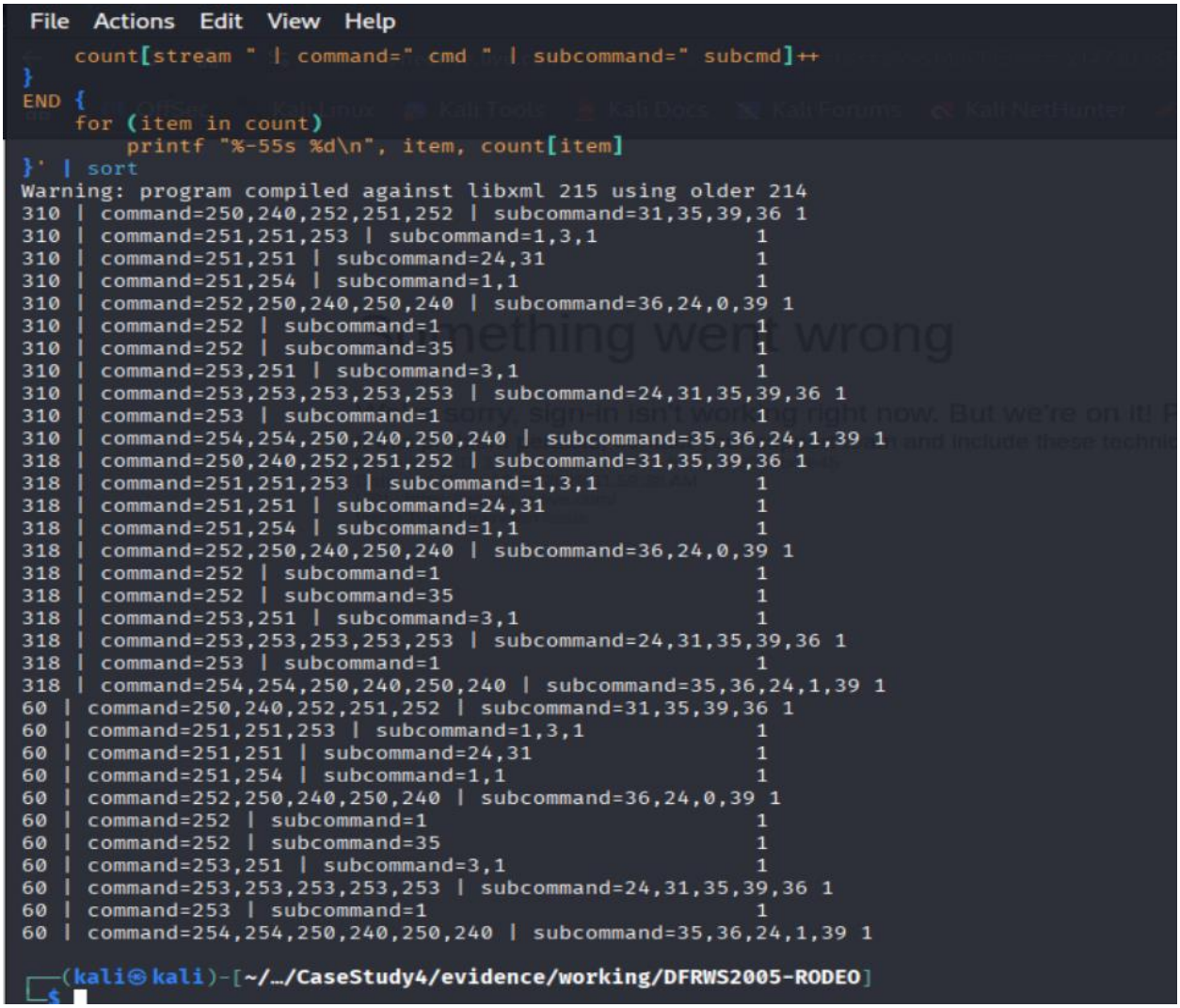
The repeated use of 137.30.120.40:23 establishes that the same network endpoint participated in all three captured Telnet sessions. However, endpoint information and packet counts alone do not establish what commands were entered, whether authentication succeeded, or what actions were performed during the sessions. Because Telnet transmits data without encryption, the capture may contain readable session content, including sensitive information. Any further examination should therefore be limited to relevant evidence and should avoid reproducing credentials in the report.

Preliminary Telnet Negotiation Analysis

The preliminary analysis identified Telnet negotiation activity in TCP streams 60, 310, and 318. All three streams contain the same sets of decoded command and subcommand

sequences. This repeated pattern is consistent with similar Telnet negotiation behaviour across the three sessions.

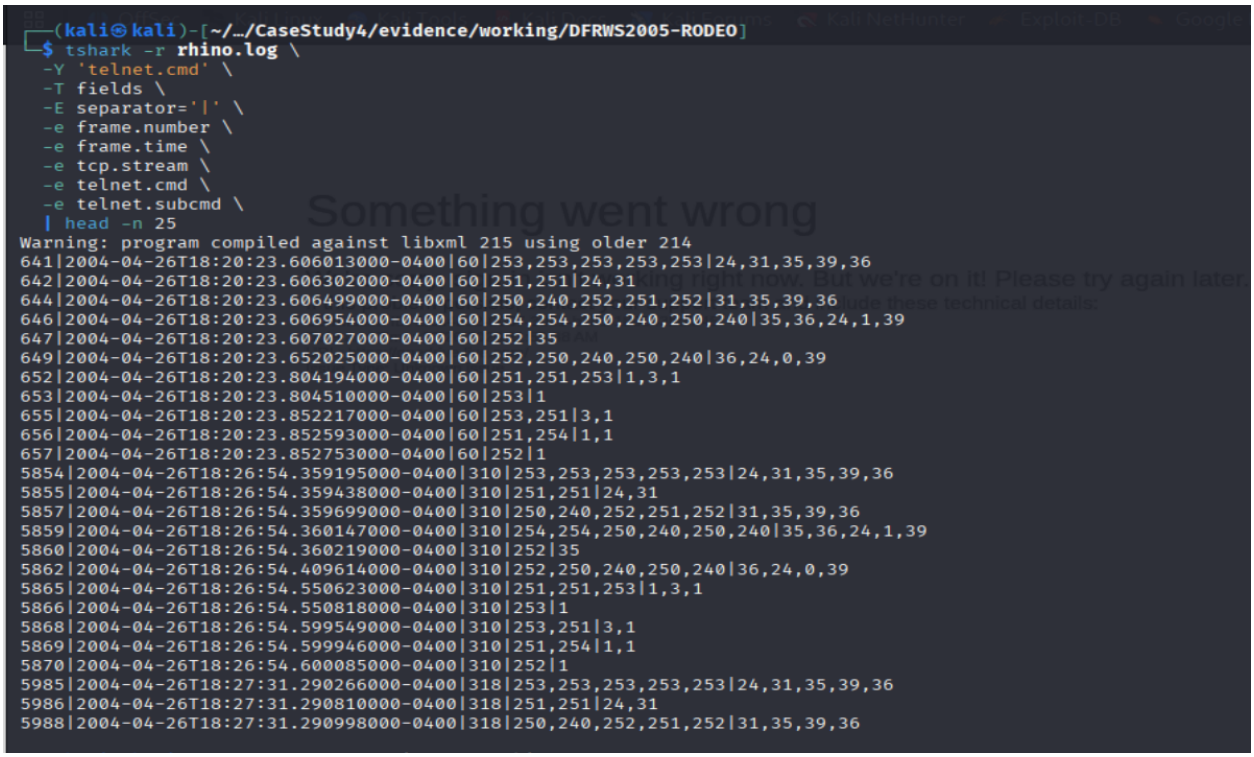
The output includes numeric command values such as 250, 240, 251, 252, 253, and 254. These values are associated with Telnet protocol control and option negotiation. The results indicate protocol negotiation activity but do not, by themselves, establish the commands entered by a user, the success of any login, or the actions performed during a session. Further interpretation requires examining the decoded fields at packet level.



Telnet Negotiation Analysis

The packet-level examination identified Telnet negotiation activity in three TCP streams between 137.30.122.253 and 137.30.120.40, using server port 23. The initial negotiation packets in each stream contain repeated command and subcommand sequences, suggesting that the sessions followed a similar negotiation pattern. The available output

establishes the presence and timing of Telnet negotiation, but does not yet establish the commands entered during interactive use or whether any particular user action occurred.



```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ tshark -r rhino.log \
  -Y 'telnet.cmd' \
  -T fields \
  -E separator='|' \
  -e frame.number \
  -e frame.time \
  -e tcp.stream \
  -e telnet.cmd \
  -e telnet.subcmd \
  | head -n 25
Warning: program compiled against libxml 215 using older 214
641|2004-04-26T18:20:23.606013000-0400|60|253,253,253,253,253|24,31,35,39,36
642|2004-04-26T18:20:23.606302000-0400|60|251,251|24,31
644|2004-04-26T18:20:23.606499000-0400|60|250,240,252,251,252|31,35,39,36
646|2004-04-26T18:20:23.606954000-0400|60|254,254,250,240,250,240|35,36,24,1,39
647|2004-04-26T18:20:23.607027000-0400|60|252|35
649|2004-04-26T18:20:23.652025000-0400|60|252,250,240,250,240|36,24,0,39
652|2004-04-26T18:20:23.804194000-0400|60|251,251,253|1,3,1
653|2004-04-26T18:20:23.804510000-0400|60|253|1
655|2004-04-26T18:20:23.852217000-0400|60|253,251|3,1
656|2004-04-26T18:20:23.852593000-0400|60|251,254|1,1
657|2004-04-26T18:20:23.852753000-0400|60|252|1
5854|2004-04-26T18:26:54.359195000-0400|310|253,253,253,253,253|24,31,35,39,36
5855|2004-04-26T18:26:54.359438000-0400|310|251,251|24,31
5857|2004-04-26T18:26:54.359699000-0400|310|250,240,252,251,252|31,35,39,36
5859|2004-04-26T18:26:54.360147000-0400|310|254,254,250,240,250,240|35,36,24,1,39
5860|2004-04-26T18:26:54.360219000-0400|310|252|35
5862|2004-04-26T18:26:54.409614000-0400|310|252,250,240,250,240|36,24,0,39
5865|2004-04-26T18:26:54.550623000-0400|310|251,251,253|1,3,1
5866|2004-04-26T18:26:54.550818000-0400|310|253|1
5868|2004-04-26T18:26:54.599549000-0400|310|253,251|3,1
5869|2004-04-26T18:26:54.599946000-0400|310|251,254|1,1
5870|2004-04-26T18:26:54.600085000-0400|310|252|1
5985|2004-04-26T18:27:31.290266000-0400|318|253,253,253,253,253|24,31,35,39,36
5986|2004-04-26T18:27:31.290810000-0400|318|251,251|24,31
5988|2004-04-26T18:27:31.290998000-0400|318|250,240,252,251,252|31,35,39,36
```

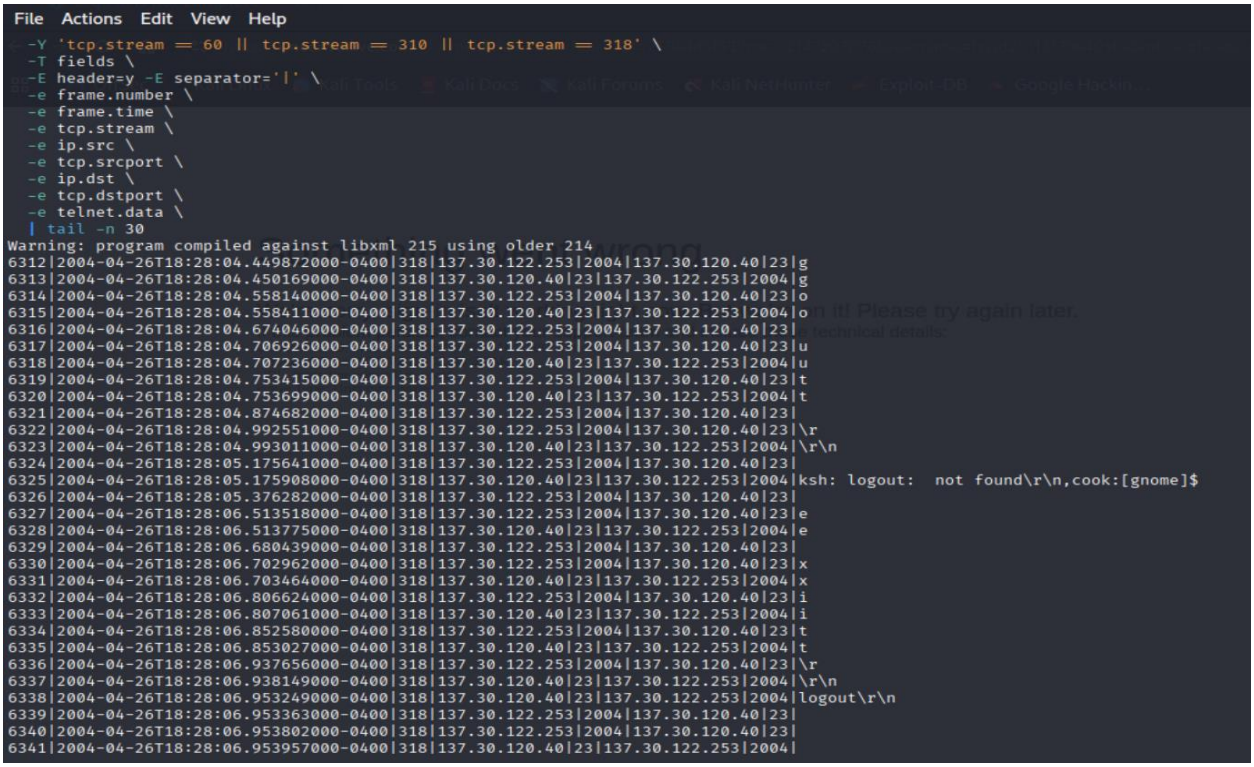
Telnet Interactive Session Content Analysis

The Telnet session data associated with TCP streams 60, 310, and 318 was examined using TShark to extract timestamps, stream identifiers, endpoint addresses, port numbers, and decoded Telnet data. The results show that the communication was not limited to protocol negotiation; readable interactive data was also present in the captured session.

In TCP stream 318, the client endpoint 137.30.122.253 communicated with the Telnet server at 137.30.120.40 on TCP port 23. The extracted data near the end of the session contains individual character exchanges, followed by the server-side message ksh: logout: not found and a shell prompt identifying the session context as cook:[gnome]$. The final packets include the text logout, followed by a carriage return and line feed. The last recorded packet in this stream occurred on 26 April 2004 at approximately 18:28:06.954 (capture timestamp, UTC−04:00).

The message ksh: logout: not found indicates that the shell reported the entered command as unavailable or unrecognised in that context. The subsequent appearance

of the shell prompt suggests that the session returned to an interactive command prompt before the later logout sequence. These observations establish the presence of readable shell-related interaction within the captured Telnet traffic. However, the extracted tail alone does not establish the full sequence of commands entered earlier in the session or the outcome of any preceding operations.



Because Telnet traffic is unencrypted, application data may be recoverable directly from the packet capture. Any credentials or other sensitive values encountered during further examination should be excluded from screenshots and report excerpts. The observed username-like text and endpoint addresses are artefacts within the capture; they do not independently establish the real-world identity of the person operating the connection.

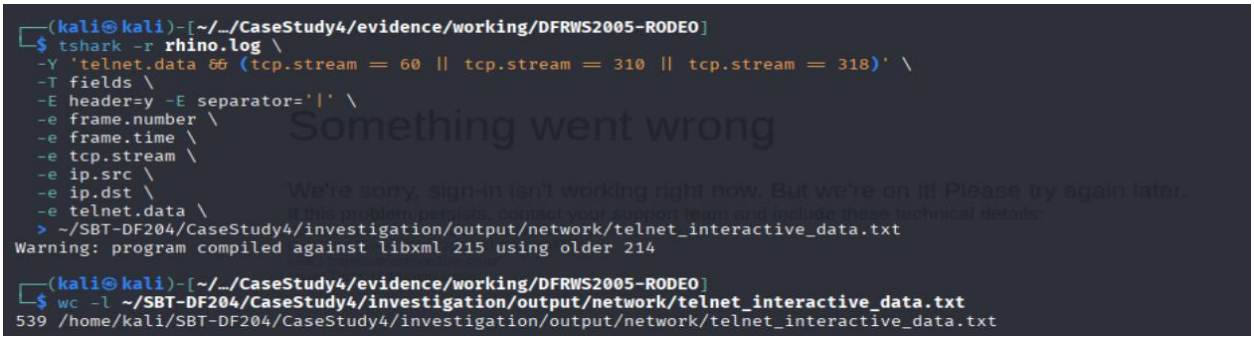
The extracted output should be retained as an investigation artefact, and the relevant packets should be correlated with the complete session data before drawing conclusions about the activities performed during the connection.

Extraction of Readable Telnet Session Data

TShark was used to extract decoded Telnet application data from TCP streams 60, 310, and 318 within the rhino.log packet capture. The extraction included the frame number, capture timestamp, TCP stream identifier, source and destination IP addresses, and

decoded Telnet data. The results were saved to investigation/output/network/telnet_interactive_data.txt for further examination and documentation.

The resulting file contains 539 lines, comprising one header line and 538 extracted data records. This output provides a structured view of the readable data associated with the three Telnet streams and supports further examination of the communication sequence and interactive session activity.



The presence of decoded application data is consistent with the recovery of readable content from the captured Telnet communications. However, the number of extracted records does not represent the number of unique commands, login attempts, or successful operations. Individual records may contain partial characters, control-related content, or responses exchanged during an interactive session. Consequently, the extracted data must be interpreted in sequence and correlated with the surrounding packets before making conclusions about the activities performed.

The extraction was performed against the existing packet capture and did not involve connecting to the remote endpoint or executing commands on the remote system. Any credentials or other sensitive information present in the decoded traffic should be redacted from report screenshots and shared excerpts.

Telnet Stream and Communication Direction Analysis

Following the extraction of readable Telnet data from rhino.log, an awk analysis was performed on telnet_interactive_data.txt to count the extracted records by TCP stream and communication direction. The analysis excluded the header row and grouped records using the TCP stream identifier, source IP address, and destination IP address.

The results identified 538 extracted data records across three TCP streams: stream 60 contained 220 records, stream 310 contained 60 records, and stream 318 contained 258

records. The records were exchanged between the client endpoint 137.30.122.253 and the Telnet server endpoint 137.30.120.40.

For stream 60, 123 records were directed from the client to the server, while 97 were directed from the server to the client. Stream 310 contained 35 client-to-server records and 25 server-to-client records. Stream 318 contained 128 client-to-server records and 130 server-to-client records. These counts demonstrate that decoded data was observed in both directions across all three streams.

```
┌──(kali㉿kali)-[~/…/CaseStudy4/evidence/working/DFRWS2005-RODEO]
└─$ awk -F'|' '
NR > 1 && NF >= 6 {
    stream[$3]++
    direction[$3 "|" $4 " → " $5]++
}
END {
    print "Records by TCP stream:"
    for (s in stream)
        printf "Stream %s: %d records\n", s, stream[s]

    print "\nRecords by stream and direction:"
    for (d in direction)
        printf "%s: %d records\n", d, direction[d]
}' ~/SBT-DF204/CaseStudy4/investigation/output/network/telnet_interactive_data.txt
Records by TCP stream:
Stream 60: 220 records
Stream 310: 60 records
Stream 318: 258 records

Records by stream and direction:
318|137.30.122.253 → 137.30.120.40: 128 records
310|137.30.122.253 → 137.30.120.40: 35 records
60|137.30.122.253 → 137.30.120.40: 123 records
318|137.30.120.40 → 137.30.122.253: 130 records
310|137.30.120.40 → 137.30.122.253: 25 records
60|137.30.120.40 → 137.30.122.253: 97 records
```

The record counts represent extracted data-bearing frame records rather than unique commands, login attempts, or application-level transactions. Differences between the directional counts may reflect the nature of the interactive exchange, including client input, server responses, and terminal-related data. The counts alone do not establish whether a particular command succeeded or what activity occurred during the entire session.

This analysis provides a structured basis for further interpretation of the Telnet communications. The results should be considered alongside packet timestamps and decoded session content, while avoiding disclosure of credentials or other sensitive information recovered from the traffic.

LIMITATIONS OF THE INVESTIGATION

The investigation was limited to the supplied USB disk image and three network packet captures. The analysis could only establish findings supported by these evidence sources

and the outputs generated using the selected tools. The absence of a finding in a particular search or listing does not necessarily establish that the corresponding artefact or activity never existed.

The USB image examination identified a FAT16 filesystem and enabled the extraction of selected allocated files and unallocated data. However, signature-based file carving does not independently establish the original filename, source, ownership, or complete historical context of a recovered file. The structural checks performed on the recovered GIF candidates do not establish their visual contents, origin, or purpose.

The network packet captures provide evidence of communications recorded during the captured periods. They do not establish activity outside those periods. The filename rhino.exe and the executable-like response data are consistent with a Windows executable being transferred over HTTP, but the analysis did not establish that the file was executed or that it caused any particular activity. Similarly, FTP transfer commands and filenames establish recorded transfer activity but do not independently prove the complete contents or purpose of every transferred file.

Telnet data was readable within the captured sessions, but record counts alone do not represent unique commands or successful operations. Any user-like identifiers or IP addresses observed in the evidence cannot, without additional corroboration, establish the real-world identity of an individual.

The investigation was performed using the tools and methods documented in this report. Any conclusions are subject to the completeness of the supplied evidence, the interpretation of tool output, and the scope of the examination.

CONCLUSION

The investigation successfully applied digital forensic techniques to examine the supplied USB disk image and network packet captures as part of the ICDFA Basic Computer Skills for Digital Forensics case study. The examination covered evidence integrity verification, filesystem analysis, file extraction, unallocated data analysis, file carving, and network traffic investigation using tools available in Kali Linux.

The USB disk image was identified as a FAT16 filesystem containing two allocated text files, which were extracted and examined. Analysis of unallocated data identified two GIF-signature candidates that were subsequently carved and subjected to structural checks.

The network investigation examined three packet captures containing HTTP, FTP, and Telnet communications. The analysis identified web requests and responses, FTP file-upload transactions involving several filenames, and readable data exchanged during three Telnet sessions. An HTTP transaction requesting rhino.exe was also identified, with the response data containing characteristics consistent with a Windows executable.

The investigation demonstrates how filesystem artefacts, file signatures, timestamps, hashes, and network communications can be used together to reconstruct aspects of digital activity. It also highlights the importance of preserving evidence, documenting analytical procedures, and distinguishing observed facts from assumptions. Although the findings establish that relevant file and network activity was present in the supplied evidence, they do not independently prove that the executable was run, establish the complete contents or purpose of every transferred file, or identify the real-world individual responsible for the recorded activity.

Overall, the objectives of the assignment were addressed within the scope of the available evidence and the methods applied. The investigation provided practical experience in digital evidence handling, forensic analysis, network traffic examination, and evidence-based reporting. The findings and supporting artefacts have been documented to provide a transparent account of the procedures performed and the results obtained.